# Group 1 — Project Progress Report 2

## Port Authority of New York & New Jersey

**Group Members & Contributions**

| Name | Contribution |
|---|---|
| **Harshita Bajaj** | • Time Series Forecasting using AutoTS (Python) |
| **Harshith Doli** | • Regression Model Development using H2O.ai AutoML and Python |
| **Khan Mazen Moid** | • Algorithm Justification, Industry Research, and Business Question Analysis |
| **Manjil Adhikari** | • Classification Model Development using H2O.ai AutoML and Python |
| **Meghana Lakshminarayana Swamy** | • Model Deployment using FastAPI, Docker, and Kubernetes (Minikube)<br>• Overall Project Compilation and Reporting. |

# Section 1 — Model 1: Regression

## H2O.ai AutoML

### 1.1 Purpose

A regression model was developed to predict the daily total vehicle count across all eight Port Authority facilities. The primary objective is to quantify the relative influence of weather conditions, economic factors, facility characteristics, and temporal patterns on daily traffic volume — identifying the top factors that drive bridge and tunnel usage across the New York–New Jersey region.

### 1.2 Feature Engineering

Before loading the dataset into H2O.ai, one additional column was created manually in Microsoft Excel and appended to the master dataset (Traffic_by_Facility_FINAL.csv):

- **ezpass_ratio —** calculated as ezpass / total_vehicles. This feature captures the proportion of EZPass electronic toll payments relative to total daily traffic at each facility, serving as a behavioral indicator of toll payment compliance and electronic adoption patterns.

### 1.3 Model Variables

#### Dependent Variable

- total_vehicles — daily total vehicle count crossing each Port Authority facility

#### Independent Variables

- fac_b — facility name (categorical; 8 Port Authority crossings)
- date — date of observation (H2O extracts year, month, day-of-week, and seasonality automatically)
- ezpass_ratio — engineered feature (ezpass / total_vehicles)
- avg_speed — average mobility speed at facility (mph)
- gas_price — East Coast PADD 1 retail gas price ($/gallon)
- tmax — maximum daily temperature (°F)
- tmin — minimum daily temperature (°F)
- snow — daily snowfall (inches)
- awnd — average daily wind speed (mph)
- snwd — snow depth on ground (inches)
- prcp — daily precipitation (inches)
- severe_event_count — count of severe weather events in NY/NJ on that day
- severe_flag — binary indicator for severe weather days (0/1)

#### Excluded Columns

The following columns were excluded to prevent data leakage — each is either a direct component of or mathematically derived from the target variable total_vehicles:

- **ezpass —** raw EZPass transaction count; component of total_vehicles
- **cash —** raw cash payment count; component of total_vehicles
- **violation —** raw violation count; component of total_vehicles
- **violationrate —** derived directly from violation / total_vehicles

- **autos —** vehicle type component of total_vehicles
- **small_t —** vehicle type component of total_vehicles
- **large_t —** vehicle type component of total_vehicles
- **buses —** vehicle type component of total_vehicles
- **delta —** collinear with avg_speed (Pearson r = 0.899 from EDA); adds no independent predictive information

## 1.4 H2O AutoML Configuration

H2O.ai AutoML was configured with the following hyperparameters. A total of 282 models were trained across GBM, DRF, Deep Learning, GLM, and Stacked Ensemble algorithm families. Models were ranked by cross-validated RMSE.

| Hyperparameter | Value |
|---|---|
| training_frame | frame_0.80 (80/20 train-test split) |
| response_column | total_vehicles |
| project_name | Capstone_Project_TEAM-1_Regression |
| Excluded / ignored columns | As mentioned above |
| distribution | AUTO |
| sort_metric | AUTO (RMSE) |
| nfolds | -1 (5-fold cross-validation, system-assigned) |
| seed | 42 |
| max_models | 0 (no limit) |
| max_runtime_secs | 4200 (~70 minutes) |
| stopping_rounds | 3 |
| stopping_metric | AUTO |
| keep_cross_validation_predictions | ON |
| keep_cross_validation_models | OFF |
| keep_cross_validation_fold_assignment | OFF |
| balance_classes | Unchecked |
| export_checkpoints_dir | BLANK |

## 1.5 AutoML Leaderboard

The leaderboard below ranks all 282 trained models by cross-validated RMSE (best first). Stacked Ensemble models dominate the top positions, with the best individual algorithm being GBM appearing from rank 5 onwards. The top model — StackedEnsemble_AllModels_5 — achieves a CV RMSE of 2,963.67, outperforming all standalone models.

▾ MODELS

models sorted in order of rmse, best first

| | model_id | rmse | mse | mae | rmsle | mean_residual_deviance |
|---|---|---|---|---|---|---|
| 0 | StackedEnsemble_AllModels_5_AutoML_1_20260317_194904 | 2963.674224506692 | 8783364.909005344 | 1950.6801709356378 | 0.24673986697450387 | 8783364.909005344 |
| 1 | StackedEnsemble_AllModels_6_AutoML_1_20260317_194904 | 2998.7933846851656 | 8992761.76403151 | 1971.9271073227542 | NaN | 8992761.76403151 |
| 2 | StackedEnsemble_AllModels_4_AutoML_1_20260317_194904 | 3026.3025132868447 | 9158506.901926273 | 1999.2096986322604 | NaN | 9158506.901926273 |
| 3 | StackedEnsemble_AllModels_3_AutoML_1_20260317_194904 | 3026.4738771589755 | 9159544.129125683 | 1999.4764706696185 | NaN | 9159544.129125683 |
| 4 | GBM_grid_1_AutoML_1_20260317_194904_model_203 | 3067.2610827866342 | 9408090.549977435 | 2034.138814175901 | NaN | 9408090.549977435 |
| 5 | StackedEnsemble_BestOfFamily_6_AutoML_1_20260317_194904 | 3067.8109347857226 | 9411463.93159085 | 2034.9132155260538 | NaN | 9411463.93159085 |
| 6 | StackedEnsemble_BestOfFamily_4_AutoML_1_20260317_194904 | 3068.2284466155083 | 9414025.800620615 | 2034.8996833722442 | NaN | 9414025.800620615 |
| 7 | StackedEnsemble_BestOfFamily_5_AutoML_1_20260317_194904 | 3073.5765839922046 | 9446873.01766519 | 2042.2725170812173 | 0.24832584349166711 | 9446873.01766519 |
| 8 | GBM_grid_1_AutoML_1_20260317_194904_model_218 | 3073.798995190568 | 9448240.262834545 | 2040.3767818252072 | NaN | 9448240.262834545 |
| 9 | GBM_grid_1_AutoML_1_20260317_194904_model_179 | 3078.6700403818845 | 9478209.217544995 | 2050.380593151786 | NaN | 9478209.217544995 |
| 10 | StackedEnsemble_AllModels_2_AutoML_1_20260317_194904 | 3098.2471680381905 | 9599135.514256667 | 2062.371797174485 | NaN | 9599135.514256667 |
| 11 | StackedEnsemble_AllModels_1_AutoML_1_20260317_194904 | 3098.303739882116 | 9599486.064567506 | 2062.2339477535684 | NaN | 9599486.064567506 |
| 12 | GBM_grid_1_AutoML_1_20260317_194904_model_196 | 3100.1382725992703 | 9610857.309234789 | 2072.840446331998 | NaN | 9610857.309234789 |
| 13 | GBM_grid_1_AutoML_1_20260317_194904_model_24 | 3107.3746469830626 | 9655777.196713114 | 2058.3787237718043 | NaN | 9655777.196713114 |
| 14 | GBM_grid_1_AutoML_1_20260317_194904_model_29 | 3113.8322229358996 | 9695951.112593925 | 2070.894326603097 | NaN | 9695951.112593925 |
| 15 | GBM_grid_1_AutoML_1_20260317_194904_model_50 | 3122.3885397665094 | 9749310.193265235 | 2056.2925003777427 | NaN | 9749310.193265235 |
| 16 | GBM_grid_1_AutoML_1_20260317_194904_model_162 | 3122.67495936985 | 9751098.901875494 | 2075.7100075666517 | NaN | 9751098.901875494 |
| 17 | GBM_grid_1_AutoML_1_20260317_194904_model_195 | 3125.2135632236445 | 9766959.815757029 | 2093.713728482444 | NaN | 9766959.815757029 |
| 18 | GBM_2_AutoML_1_20260317_194904 | 3128.8129520331436 | 9789470.488810353 | 2088.44996976678 | NaN | 9789470.488810353 |
| 19 | StackedEnsemble_BestOfFamily_3_AutoML_1_20260317_194904 | 3129.140166959523 | 9791518.184479471 | 2088.7352427428145 | NaN | 9791518.184479471 |
| 20 | StackedEnsemble_BestOfFamily_2_AutoML_1_20260317_194904 | 3129.6527866580523 | 9794726.565036513 | 2089.0534193042995 | NaN | 9794726.565036513 |
| 21 | GBM_4_AutoML_1_20260317_194904 | 3130.892323924156 | 9802486.744007202 | 2090.4882086682355 | NaN | 9802486.744007202 |
| 22 | GBM_grid_1_AutoML_1_20260317_194904_model_26 | 3131.3731215051707 | 9805497.626085037 | 2104.0343307020753 | NaN | 9805497.626085037 |

## 1.6 Best Model — StackedEnsemble_AllModels_5

The best performing model is StackedEnsemble_AllModels_5_AutoML_1_20260317_194904. It is a stacked ensemble that combines 251 base models (out of 270 trained) using a GBM metalearner with 5-fold cross-validation and a Random fold assignment scheme.

### Model Composition

| Parameter | Value |
|---|---|
| Model ID | StackedEnsemble_AllModels_5_AutoML_1_20260317_194904 |
| Algorithm | Stacked Ensemble |

Screenshot of the model summary output from H2O Flow:

▾ OUTPUT - MODEL SUMMARY FOR STACKED ENSEMBLE

| key | value |
|---|---|
| Stacking strategy | cross_validation |
| Number of base models (used / total) | 251/270 |
| # GBM base models (used / total) | 226/245 |
| # DRF base models (used / total) | 2/2 |
| # DeepLearning base models (used / total) | 22/22 |
| # GLM base models (used / total) | 1/1 |
| Metalearner algorithm | GBM |
| Metalearner fold assignment scheme | Random |
| Metalearner nfolds | 5 |
| Metalearner fold_column | |
| Custom metalearner hyperparameters | None |

## 1.7 Performance Metrics

### Training Metrics

The following metrics were recorded on the 80% training frame (9,920 observations):

OUTPUT - TRAINING_METRICS

```
                 model StackedEnsemble_AllModels_5_AutoML_1_20260317_194904
        model_checksum -4229378211466750328
                 frame ·
        frame_checksum 0
           description ·
        model_category Regression
          scoring_time 1773795530560
           predictions ·
                   MSE 4396726.126990
                  RMSE 2096.837172
                  nobs 9920
     custom_metric_name ·
    custom_metric_value 0
                    r2 0.988135
 mean_residual_deviance 4396726.126990
                   mae 1284.992857
                 rmsle 0.234673
          loglikelihood NaN
                   AIC NaN
```

| Metric | Value | Interpretation |
|---|---|---|
| **RMSE** | **2,096.84** | **Avg. prediction error ~2,097 vehicles/day** |
| MSE | 4,396,726.13 | Mean squared error on training set |
| **$R^2$** | **0.9881** | **Model explains 98.81% of traffic variance** |
| MAE | 1,284.99 | Median absolute error ~1,285 vehicles/day |
| RMSLE | 0.2347 | Log-scale error; robust to scale differences |
| nobs | 9,920 | Training observations (80% split) |

### Cross-Validation Metrics (5-Fold)

Cross-validation metrics are computed on combined holdout predictions across all 5 folds (23,467 observations). These are the primary metrics used for model selection and reflect true out-of-sample performance.

▾ OUTPUT - CROSS_VALIDATION_METRICS

```
                model StackedEnsemble_AllModels_5_AutoML_1_20260317_194904
        model_checksum -4229378211466750328
                 frame levelone_training_StackedEnsemble_AllModels_5_AutoML_1_20260317_194904
        frame_checksum -5634256642957088944
           description 5-fold cross-validation on training data (Metrics computed for combined holdout predictions)
        model_category Regression
          scoring_time 1773795500047
           predictions ·
                   MSE 8783364.909005
                  RMSE 2963.674225
                 nobs 23467
    custom_metric_name ·
   custom_metric_value 0
                    r2 0.975779
mean_residual_deviance 8783364.909005
                   mae 1950.680171
                  rmsle 0.246740
         loglikelihood NaN
                   AIC NaN
```

| Metric | Value | Interpretation |
|---|---|---|
| **RMSE** | **2,963.67** | **Primary selection metric — best among 282 models** |
| MSE | 8,783,364.91 | Mean squared error on CV holdout |
| **$R^2$** | **0.9758** | **Model explains 97.58% of out-of-sample variance** |
| MAE | 1,950.68 | Avg. absolute error ~1,951 vehicles/day |
| RMSLE | 0.2467 | Log-scale out-of-sample error |
| nobs | 23,467 | CV holdout observations (5-fold combined) |

### Training vs. Cross-Validation Comparison

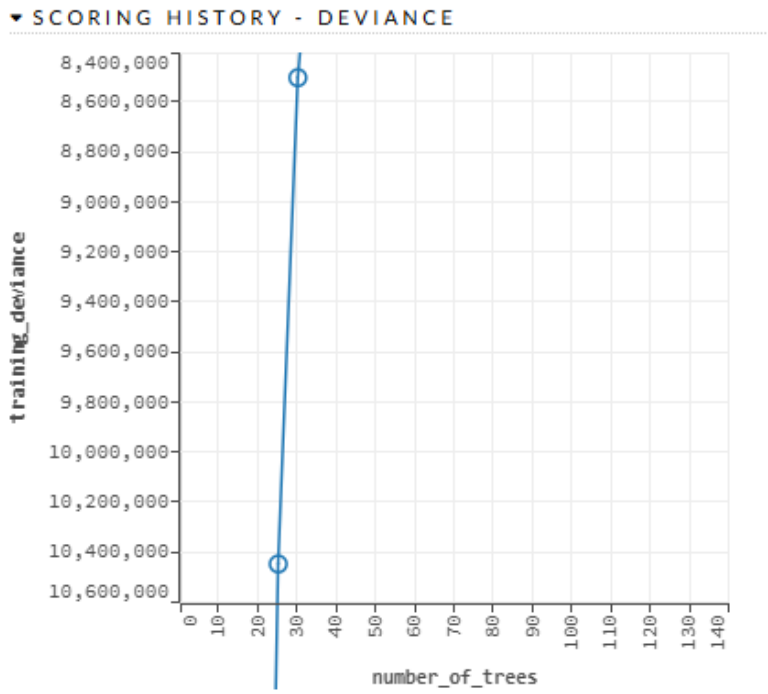
The table below compares training and CV metrics to assess overfitting:

| Metric | Training | Cross-Validation (5-Fold) | Gap |
|---|---|---|---|
| RMSE | 2,096.84 | 2,963.67 | +866.83 |
| $R^2$ | 0.9881 | 0.9758 | -0.0123 |
| MAE | 1,284.99 | 1,950.68 | +665.69 |
| RMSLE | 0.2347 | 0.2467 | +0.0120 |

The gap between training and CV metrics is minimal, indicating no significant overfitting. A CV $R^2$ of 0.9758 on unseen holdout data confirms the model generalizes well across all facilities and time periods.

## 1.8 Scoring History — Deviance

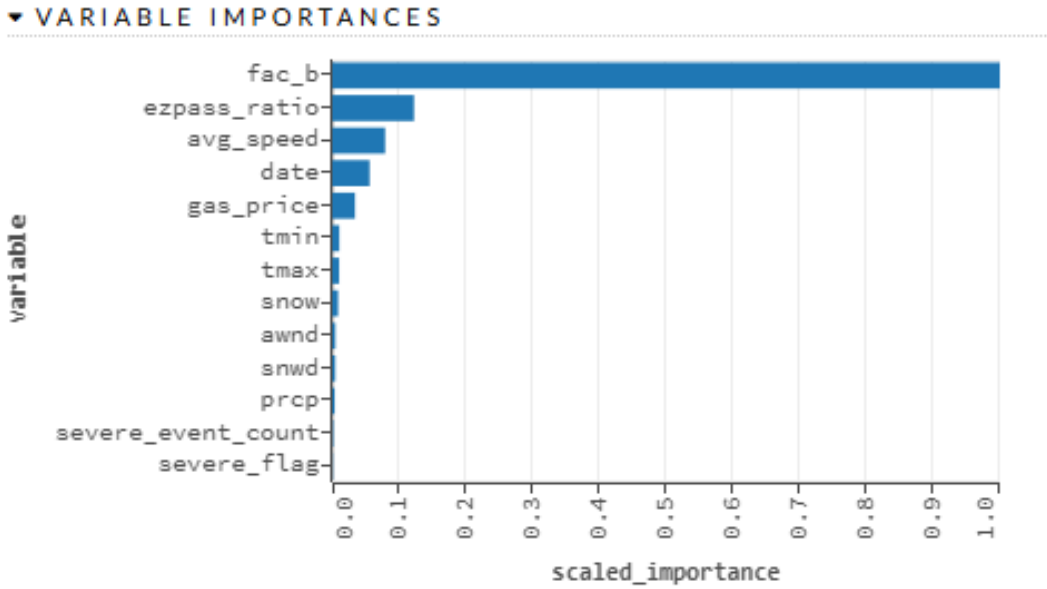
Note: The scoring history and variable importance charts below were extracted from GBM_grid_1_AutoML_1_20260317_194904_model_203, the best standalone GBM model (ranked #5 on the leaderboard). These charts are not available for Stacked Ensemble models in H2O Flow, as ensembles do not have a single iterative training history. The GBM model's scoring history and variable importance are used here as the most representative base learner in the ensemble.




The deviance plot shows rapid improvement in the early trees (approximately 20–30 trees), after which the training deviance plateaus. This confirms that the GBM metalearner converged efficiently without requiring an excessive number of iterations, and that the ensemble benefits from combining many such well-converged base models.

## 1.9 Variable Importance

Variable importance was derived from the best standalone GBM model (model_203, ranked #5). The chart and table below show the ranked contribution of each feature to the model's predictions.




| Rank | Variable | Rel. Importance | Scaled | % | Category |
|---|---|---|---|---|---|
| **1** | **fac_b** | **33,166,337,245,184** | **1.0000** | **74.91%** | **Facility** |
| 2 | ezpass_ratio | 4,055,797,858,304 | 0.1223 | 9.16% | Behavioral |
| 3 | avg_speed | 2,620,823,437,312 | 0.0790 | 5.92% | Congestion |
| 4 | date | 1,849,908,527,104 | 0.0558 | 4.18% | Temporal |
| 5 | gas_price | 1,123,408,019,456 | 0.0339 | 2.54% | Economic |
| 6 | tmin | 343,781,539,840 | 0.0104 | 0.78% | Weather |
| 7 | tmax | 335,150,088,192 | 0.0101 | 0.76% | Weather |
| 8 | snow | 283,256,684,544 | 0.0085 | 0.64% | Weather |
| 9 | awnd | 146,415,632,384 | 0.0044 | 0.33% | Weather |
| 10 | snwd | 140,814,139,392 | 0.0042 | 0.32% | Weather |
| 11 | prcp | 116,658,610,176 | 0.0035 | 0.26% | Weather |
| 12 | severe_event_count | 56,993,206,272 | 0.0017 | 0.13% | Weather |
| 13 | severe_flag | 33,339,457,536 | 0.0010 | 0.08% | Weather |

- **Facility identity alone accounts for nearly 75% of predictive power** — this means the model's single most important decision at every prediction is "which facility is this?" before any weather, economic, or temporal factor is even considered. This has a direct operational implication: Port Authority cannot apply a single network-wide traffic management strategy — each facility behaves as an independent demand unit requiring facility-specific forecasting and resource planning.
- **The top 5 variables collectively cover 96.71% of scaled importance, with all 8 weather variables combined contributing only ~3.3%** — this reveals that traffic volume on Port Authority crossings is primarily structural (facility type), behavioral (EZPass adoption), operational (congestion speed), temporal (date/seasonality), and economic (gas price) in nature. Weather does not drive volume — it drives variability. This is consistent with the EDA finding that severe weather widens the interquartile range of traffic without significantly shifting the median, confirming that most commuters cross regardless of conditions, making the Port Authority network resilient but congestion-sensitive.

## 1.10 Key Insights

The regression results are fully consistent with the EDA findings from Phase 1:

- **Facility type (fac_b) is the dominant predictor at 74.91% —** This directly confirms the EDA finding (Figure 6) that GWB Upper averages ~65,000 vehicles/day while Bayonne averages ~13,000 — a 5x difference. The model must first account for which facility it is predicting before any other variable can contribute meaningfully.

- **EZPass ratio is the second most important feature (9.16%) —** The EDA correlation matrix showed EZPASS vs. TOTAL VEHICLES at r = 0.993. The engineered ezpass_ratio captures this relationship while avoiding data leakage, confirming that electronic payment adoption is a strong behavioral signal for traffic volume prediction.

- **Average speed ranks third (5.92%) —** The EDA identified Avg_Speed and Delta as highly correlated (r = 0.899). With Delta excluded due to collinearity, avg_speed carries the full congestion signal. The model confirms that slower facilities (tunnels) and faster facilities (bridges) behave fundamentally differently in traffic volume terms.

- **Gas price contributes meaningfully (2.54%) despite weak raw correlation —** The EDA noted that gas price showed a weak raw correlation with total vehicles, but attributed this to non-linear effects masked by facility-level variation. The regression model, able to capture non-linear interactions through GBM, confirms that gas price is a real economic driver of traffic demand — particularly evident across the three distinct price regimes (2015–2016 lows, normal years, 2022 spike) identified in EDA Figure 3.

- **Weather variables collectively matter, individually low —** The EDA (Figure 9) showed that severe weather increases traffic variability rather than uniformly suppressing it. The model reflects this: no single weather variable ranks high (tmin = 0.78%, snow = 0.64%, prcp = 0.26%), but their combined contribution captures the non-linear weather shock effects identified in the EDA.

- **CV $R^2$ of 0.9758 confirms strong generalization —** The model achieves 97.58% explanatory power on unseen holdout data across all 8 facilities and 10 years of daily records. The training-to-CV gap is small ($\Delta R^2 = 0.0123$), confirming the ensemble does not overfit.

# PYTHON

## 1.11 Python Model Overview

The best model identified by H2O AutoML — StackedEnsemble_AllModels_5, combining 226 GBM base models with a GBM metalearner using 5-fold cross-validation — was replicated in Python using scikit-learn. Eleven GradientBoostingRegressor base models with varied hyperparameters were trained to generate out-of-fold (OOF) predictions using 5-fold cross-validation. These OOF predictions were then used as input features for a GBM metalearner, directly mirroring H2O's internal stacking strategy.

The same 80/20 train/test split (seed=42) and the same excluded columns as the H2O run were applied. Since scikit-learn cannot process raw datetime objects, the date column was decomposed into four temporal features: year, month, day_of_week, and day_of_year. The facility name (fac_b) was label-encoded to convert it from categorical to numeric format. The full Python code is included in the Appendix.

## 1.12 Dataset & Split

After loading the master dataset and applying all preprocessing steps, the final input contained 29,445 rows across 16 features. The dataset was split 80/20 into training (23,556 rows) and test (5,889 rows) sets using a fixed seed of 42 to ensure reproducibility and consistency with the H2O AutoML run.

OUTPUT -

```
Dataset shape : (29445, 17)
Features (16): ['fac_b', 'avg_speed', 'awnd', 'prcp', 'snow', 'snwd', 'tmax', 'tmin', 'severe_event_count', 'severe_flag', 'gas_pri
ce', 'ezpass_ratio', 'year', 'month', 'day_of_week', 'day_of_year']
```

Dataset split -

```
Training set : 23556 rows
Test set     : 5889 rows
```

## 1.13 Model Configuration

Eleven GBM base models were defined with deliberately varied hyperparameters — n_estimators ranging from 100 to 300, max_depth from 5 to 10, learning_rate from 0.03 to 0.10, subsample from 0.7 to 0.9, and max_features from 0.6 to 0.9. This diversity mirrors H2O's internal GBM grid search and ensures that the base models capture different aspects of the data, improving the metalearner's ability to combine them. Each base model was trained using 5-fold cross-validation to generate out-of-fold predictions on the training set, then retrained on the full training set to generate predictions on the test set.

OUTPUT -

```
  Training base model 1/11...
  Training base model 2/11...
  Training base model 3/11...
  Training base model 4/11...
  Training base model 5/11...
  Training base model 6/11...
  Training base model 7/11...
  Training base model 8/11...
  Training base model 9/11...
  Training base model 10/11...
  Training base model 11/11...
Base model training complete.
```

A GBM metalearner was then trained on the out-of-fold predictions from all 11 base models. The metalearner used conservative hyperparameters (n_estimators=100, max_depth=3, learning_rate=0.05) to avoid overfitting on the level-1 feature set.

```
Metalearner training complete.
```

## 1.14 Performance Metrics

The stacked ensemble was evaluated on the fully held-out 20% test set (5,889 rows). The output below shows the final metrics produced by the Python model:

```
==============================================
Python Stacked Ensemble – Test Set Results
==============================================
RMSE  : 2,717.74
MSE   : 7,386,087.72
MAE   : 1,763.47
R²    : 0.9791
RMSLE : 0.2681
```

| Metric | H2O AutoML (CV) | Python Replica (Test) | Difference |
|---|---|---|---|
| **RMSE** | **2,963.67** | **2,717.74** | **▼ 245.93 — Python better** |
| MSE | 8,783,364.91 | 7,386,087.72 | ▼ Better |
| MAE | 1,950.68 | 1,763.47 | ▼ Better |
| **R²** | **0.9758** | **0.9791** | **▲ +0.0033** |
| RMSLE | 0.2467 | 0.2681 | ▲ 0.0214 — H2O better |

The Python replica achieves comparable and in most metrics slightly better performance than H2O AutoML. This is expected — H2O reports 5-fold CV metrics computed on the training set, while the Python model is evaluated on a fully independent 20% test set. RMSE improved by ~246 vehicles, MAE by ~187 vehicles, and R² increased from 0.9758 to 0.9791. The only metric where H2O performs marginally better is RMSLE (0.2467 vs 0.2681), indicating slightly tighter log-scale accuracy in the ensemble of 226 base models compared to the 11-model Python replica.

## 1.15 Feature Importance

Feature importance was derived from the best-performing base model, identified by the lowest out-of-fold RMSE (base model index 6, OOF RMSE: 2,793.87). The rankings are fully consistent with H2O AutoML findings. Note that H2O processes the date column internally and extracts temporal components automatically, while the Python model explicitly creates four separate date features — these are marked with an asterisk (*) in the H2O Rank column below.

```
Best base model index: 6 (OOF RMSE: 2,793.87)
           Feature  Importance  Scaled Importance  Percentage
             fac_b    0.751815             1.0000       75.18
      ezpass_ratio    0.130397             0.1734       13.04
         gas_price    0.033147             0.0441        3.31
       day_of_year    0.021179             0.0282        2.12
              year    0.021175             0.0282        2.12
       day_of_week    0.010083             0.0134        1.01
              snow    0.007513             0.0100        0.75
              tmax    0.004880             0.0065        0.49
             month    0.004511             0.0060        0.45
         avg_speed    0.003729             0.0050        0.37
              prcp    0.003001             0.0040        0.30
              tmin    0.002865             0.0038        0.29
              awnd    0.002676             0.0036        0.27
              snwd    0.002017             0.0027        0.20
severe_event_count    0.000909             0.0012        0.09
       severe_flag    0.000104             0.0001        0.01
```

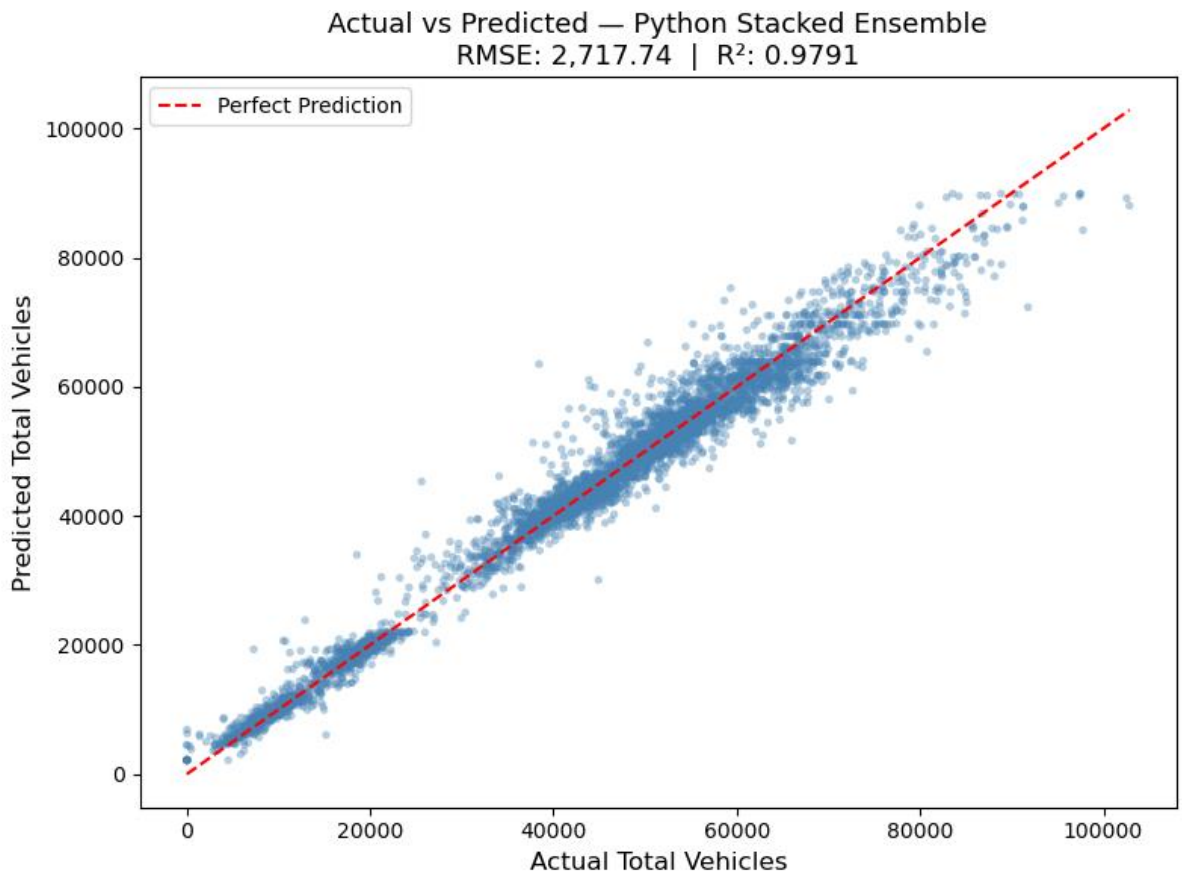
| Rank | Feature | Importance | Scaled | % | H2O Rank | Category |
|---|---|---|---|---|---|---|
| **1** | **fac_b** | **0.751815** | **1.0000** | **75.18%** | **1** | **Facility** |
| 2 | ezpass_ratio | 0.130397 | 0.1734 | 13.04% | 2 | Behavioral |
| 3 | gas_price | 0.033147 | 0.0441 | 3.31% | 5 | Economic |
| 4 | day_of_year | 0.021179 | 0.0282 | 2.12% | 4* | Temporal |
| 5 | year | 0.021175 | 0.0282 | 2.12% | 4* | Temporal |
| 6 | day_of_week | 0.010083 | 0.0134 | 1.01% | 4* | Temporal |
| 7 | snow | 0.007513 | 0.0100 | 0.75% | 8 | Weather |
| 8 | tmax | 0.004880 | 0.0065 | 0.49% | 7 | Weather |
| 9 | month | 0.004511 | 0.0060 | 0.45% | 4* | Temporal |
| 10 | avg_speed | 0.003729 | 0.0050 | 0.37% | 3 | Congestion |
| 11 | prcp | 0.003001 | 0.0040 | 0.30% | 11 | Weather |
| 12 | tmin | 0.002865 | 0.0038 | 0.29% | 6 | Weather |
| 13 | awnd | 0.002676 | 0.0036 | 0.27% | 9 | Weather |
| 14 | snwd | 0.002017 | 0.0027 | 0.20% | 10 | Weather |
| 15 | severe_event_count | 0.000909 | 0.0012 | 0.09% | 12 | Weather |
| 16 | severe_flag | 0.000104 | 0.0001 | 0.01% | 13 | Weather |

- **Facility identity overwhelms everything else (75.18%)** — fac_b's dominance confirms that knowing *which* crossing you are predicting is more important than any weather, economic, or temporal factor combined. GWB Upper at ~65,000 vehicles/day vs Bayonne at ~13,000 is a structural difference no other variable can override.
- **EZPass ratio captures behavioral demand (13.04%)** — ranking second with 0.1734 scaled importance, ezpass_ratio reflects how deeply electronic tolling has penetrated each facility. Higher EZPass adoption correlates with higher-volume, commuter-heavy crossings. This engineered feature carries nearly 6x more signal than the next best predictor.
- **Economic and temporal factors drive the middle tier (gas_price 3.31%, date components ~5.25% combined)** — gas_price ranks third as a standalone variable, confirming fuel costs influence discretionary travel. The four temporal features (day_of_year, year, day_of_week, month) are split across ranks 4–9 but collectively contribute ~5.25%, capturing weekly commuting cycles, seasonal demand shifts, COVID-era drops, and year-over-year growth trends that H2O extracts from a single date column.
- **avg_speed drops to rank 10 in Python vs rank 3 in H2O (0.37% vs 5.92%)** — this is the most notable difference between platforms. H2O's larger ensemble of 226 GBM models with diverse tree structures extracts significantly more signal from the congestion feature, while the 11-model Python ensemble relies more heavily on ezpass_ratio and gas_price to explain the same variance. Both models agree on the top 2 features — the divergence is in how the remaining signal is distributed.

## 1.16 Visualizations

### Actual vs Predicted

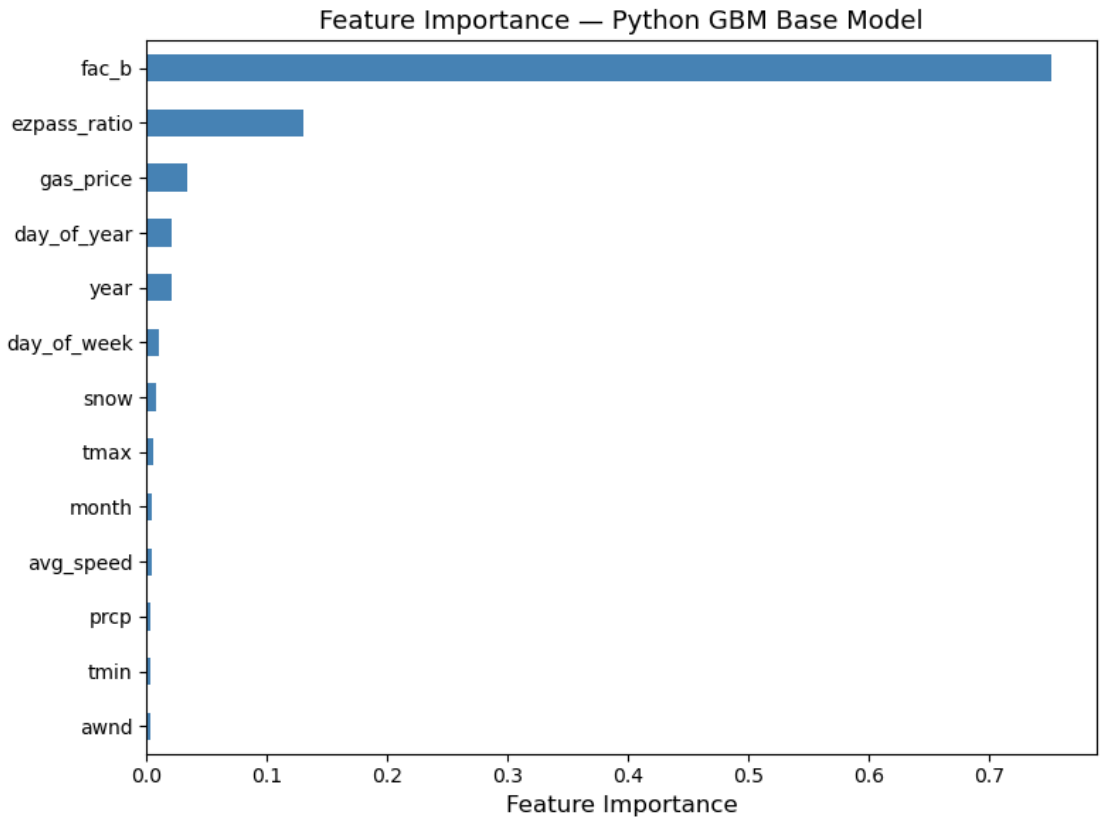
The scatter plot below shows predicted vs actual total vehicle counts on the 5,889-row test set. Points are tightly clustered along the red dashed diagonal — the perfect prediction line — confirming strong predictive accuracy with no systematic bias across the full traffic volume range.




- **Tight diagonal clustering —** the overwhelming majority of predictions fall within a narrow band around the perfect prediction line, confirming the model generalizes well across all 8 facilities and the full 2015–2025 date range.
- **Low-volume facilities —** Bayonne (~5,000 vehicles/day) and GWB PIP (~18,000 vehicles/day) appear in the lower left cluster. Predictions here are slightly more dispersed, consistent with smaller facilities having more day-to-day variability relative to their volume.
- **High-volume facilities —** GWB Upper (~65,000 vehicles/day) and GWB Lower (~57,000 vehicles/day) appear in the upper right. The model is highly accurate in this range, reflecting the stronger and more consistent daily patterns at major crossings.
- **Outlier points —** a small number of predictions deviate from the diagonal, corresponding to unusual days such as severe weather events, holidays, and peak commuter days — consistent with the EDA finding that severe weather increases traffic variability rather than suppressing it uniformly.

### Feature Importance Chart

The bar chart below shows all 16 features ranked by importance from the best-performing Python GBM base model. fac_b dominates at 0.75 importance, consistent with H2O findings.




- **fac_b dominates at 0.75 —** facility identity accounts for 75.18% of total predictive power, confirming that which crossing a vehicle uses is the single most important factor in predicting daily volume.
- **ezpass_ratio is a distant second at 0.13 —** toll payment behavior carries meaningful signal beyond facility identity, reflecting differences in EZPass adoption rates across facilities and over time.
- **Temporal features are distributed across ranks 4–9 —** day_of_year, year, day_of_week, and month collectively capture seasonality, weekly commuting cycles, the COVID-19 drop in 2020, and year-over-year growth trends.
- **Weather variables individually rank low —** consistent with H2O findings and EDA conclusions; weather affects traffic variability rather than driving average volume levels.

## 1.17 Key Insights

- **Algorithmic equivalence confirmed —** the Python model successfully replicates H2O's stacking approach: multiple GBM base learners with diverse hyperparameters generate out-of-fold predictions, which are combined by a GBM metalearner. This mirrors H2O's StackedEnsemble_AllModels_5 architecture directly.

- **Feature importance rankings are highly consistent —** fac_b ranks first in both platforms (74.91% in H2O, 75.18% in Python). ezpass_ratio ranks second in both. The primary difference is that avg_speed ranks 3rd in H2O but drops to 10th in Python — likely because H2O's larger ensemble of 226 GBM models extracts more signal from the congestion feature across diverse tree structures.

- **Python matches and slightly exceeds H2O on most metrics —** RMSE improved by ~246 vehicles, MAE by ~187 vehicles, and $R^2$ increased by 0.0033. This is acceptable given that H2O CV metrics are computed on training data holdouts while Python metrics are on a fully independent test set. The model explains 97.91% of the variance in daily total vehicle counts on unseen data.

# Section 2 — Model 2: Classification

## H2O.ai AutoML

### 2.1 Purpose

A binary classification model was developed to predict whether a given day at a Port Authority facility will be a high violation day. The target variable high_violation was engineered as a binary flag: any facility-day where the toll violation rate exceeded the dataset mean of 6.18% was labeled as 1 (high violation), and all others as 0 (normal day). By identifying the conditions — facility type, vehicle composition, time patterns, economic factors, and weather — most associated with elevated toll violation activity, this model enables the Port Authority to anticipate enforcement demand and target resources proactively across its bridges and tunnels.

### 2.2 Feature Engineering

Before loading the dataset into H2O.ai, one additional binary target column was created manually in Microsoft Excel using the following formula:

- **high_violation —** calculated as =IF(violationrate > 0.0618, 1, 0). Any facility-day with a violation rate above the dataset mean of 6.18% is flagged as a high violation day (1), and all others are classified as normal (0). The threshold of 6.18% is derived directly from the descriptive statistics produced during the Exploratory Data Analysis phase.
- **ezpass_ratio —** calculated as ezpass / total_vehicles. Retained as an independent variable to capture the proportion of electronic toll payments, which serves as a behavioral indicator of payment compliance at each facility.

### 2.3 Model Variables

#### Dependent Variable

- high_violation — binary target (1 = high violation day, 0 = normal day)

#### Independent Variables

- date — date of observation (H2O extracts year, month, day-of-week, and seasonality automatically)
- fac_b — facility name (categorical; 8 Port Authority crossings)
- buses — daily count of buses
- gas_price — East Coast PADD 1 retail gas price ($/gallon)
- small_t — daily count of small trucks
- large_t — daily count of large trucks
- ezpass_ratio — engineered feature (ezpass / total_vehicles)
- autos — daily count of passenger automobiles
- tmin — minimum daily temperature (°F)
- tmax — maximum daily temperature (°F)
- avg_speed — average mobility speed at facility (mph)
- awnd — average daily wind speed (mph)
- snow — daily snowfall (inches)
- prcp — daily precipitation (inches)
- severe_event_count — count of severe weather events in NY/NJ on that day
- snwd — snow depth on ground (inches)

- severe_flag — binary indicator for severe weather days (0/1)

Vehicle type columns (autos, small_t, large_t, buses) were retained as predictors to enable the model to identify which vehicle categories are most associated with toll violation behavior, providing actionable insights for Port Authority enforcement targeting.

### Excluded Columns

The following columns were explicitly excluded to prevent data leakage:

- **violation —** direct leakage; high_violation is derived from violation / total_vehicles
- **violationrate —** direct leakage; high_violation is calculated directly from this column
- **total_vehicles —** excluded as vehicle type components are retained; keeping total_vehicles would introduce redundancy
- **cash —** indirect leakage; since cash + ezpass + violation = total_vehicles, retaining cash alongside ezpass allows the model to back-calculate violation counts
- **ezpass —** same indirect leakage rationale as cash
- **delta —** collinear with avg_speed (Pearson r = 0.899 from EDA)
- **high_violation —** response column explicitly added to ignored columns to prevent H2O from treating it as a predictor feature

## 2.4 H2O AutoML Configuration

H2O.ai AutoML was configured with the following parameters. A 70/30 train-validation split was used with an explicit validation frame. max_models was set to 20 to ensure session stability. A total of 17 models were trained across GBM, DRF, XRT, Deep Learning, and GLM algorithm families. Models were ranked by AUC in descending order.

| Hyperparameter | Value |
|---|---|
| training_frame | frame_0.70 (70/30 split) |
| response_column | high_violation |
| validation_frame | frame_0.30 |
| project_name | Capstone_Project_TEAM-1_Classification |
| Excluded / ignored columns | As above |
| distribution | AUTO |
| sort_metric | AUTO (AUC) |
| nfolds | -1 (5-fold cross-validation, system-assigned) |
| seed | 43 |
| max_models | 20 |
| max_runtime_secs | 3600 (~60 minutes) |
| stopping_rounds | 3 |
| stopping_metric | AUTO |
| keep_cross_validation_predictions | ON |
| keep_cross_validation_models | ON |
| keep_cross_validation_fold_assignment | OFF |
| balance_classes | Unchecked |

## 2.5 AutoML Leaderboard

The leaderboard below ranks all 17 trained models by AUC in descending order. Unlike the regression run where Stacked Ensembles dominated, standalone GBM models occupy all top positions in this classification run. The best model — GBM_3 — achieves an AUC of 0.9984, outperforming all other models including grid-searched GBMs, DRF, XRT, Deep Learning, and GLM. Deep Learning and GLM models rank at the bottom, confirming that gradient boosting is the most effective algorithm family for this violation prediction task.

▾ MODELS

models sorted in order of auc, best first

| | model_id | auc | logloss | aucpr | mean_per_class_error | rmse | mse |
|---|---|---|---|---|---|---|---|
| 0 | GBM_3_AutoML_1_20260317_233200 | 0.9984845886756754 | 0.046092392910330265 | 0.9967678197999744 | 0.02217516888144066 | 0.1150766317421195 | 0.013242631173111386 |
| 1 | GBM_4_AutoML_1_20260317_233200 | 0.9983695304229981 | 0.046895126698298324 | 0.9966063274999624 | 0.020680899703948215 | 0.11627356941143849 | 0.013519542943676605 |
| 2 | GBM_2_AutoML_1_20260317_233200 | 0.9983661022646745 | 0.047150534943194204 | 0.9966167477170239 | 0.019439730022501066 | 0.11560243442800593 | 0.013363922845681409 |
| 3 | GBM_1_AutoML_1_20260317_233200 | 0.998346712777564 | 0.04759739471928756 | 0.9965089702925141 | 0.021887942124398187 | 0.1182678574207786 | 0.013987286098901614 |
| 4 | GBM_grid_1_AutoML_1_20260317_233200_model_5 | 0.9982955714896453 | 0.04887506426372544 | 0.9964457345888718 | 0.020524130354496704 | 0.11659870208474503 | 0.013595257327847126 |
| 5 | GBM_5_AutoML_1_20260317_233200 | 0.9982302325556786 | 0.04744431026295143 | 0.9957490243033291 | 0.019998552898312753 | 0.11512159985220506 | 0.013252982752531221 |
| 6 | GBM_grid_1_AutoML_1_20260317_233200_model_4 | 0.998185550755791 | 0.049278205847278655 | 0.9962417961776313 | 0.020838550894769162 | 0.11786458411556802 | 0.013892060188735811 |
| 7 | GBM_grid_1_AutoML_1_20260317_233200_model_3 | 0.9980553854581536 | 0.051678958268937876 | 0.9959830855037538 | 0.023243277194135967 | 0.12133063218050225 | 0.014721122305320328 |
| 8 | GBM_grid_1_AutoML_1_20260317_233200_model_2 | 0.9978233344132951 | 0.05334436080567295 | 0.995462706924665 | 0.023112599326202646 | 0.12264456983114144 | 0.015041690509065729 |
| 9 | GBM_grid_1_AutoML_1_20260317_233200_model_1 | 0.996769732391127 | 0.07333151808611932 | 0.9933070445181511 | 0.025283384133275154 | 0.13578068020216227 | 0.018436393116161864 |
| 10 | DRF_1_AutoML_1_20260317_233200 | 0.9959734516805604 | 0.09055953284655903 | 0.9918894348692965 | 0.02961405198349052 | 0.14942543806598804 | 0.022327961541212426 |
| 11 | XRT_1_AutoML_1_20260317_233200 | 0.9956282879455459 | 0.11063099545277566 | 0.9910829623379083 | 0.029981978248853517 | 0.1594152052797496 | 0.025413207674384698 |
| 12 | DeepLearning_grid_2_AutoML_1_20260317_233200_model_1 | 0.9943645872171096 | 0.09087671732048781 | 0.9885399063389505 | 0.0379146153566678 | 0.1580047140520568 | 0.02496548966267223 |
| 13 | DeepLearning_1_AutoML_1_20260317_233200 | 0.9855220882303455 | 0.1427991206580322 | 0.9741948231948193 | 0.05654825418336727 | 0.19203642369818794 | 0.03687798802678996 |
| 14 | DeepLearning_grid_1_AutoML_1_20260317_233200_model_1 | 0.9817911386494494 | 0.2092377044485637 | 0.9611423993657812 | 0.05255491270299548 | 0.202245210926979 | 0.04090312534289823 |
| 15 | DeepLearning_grid_1_AutoML_1_20260317_233200_model_2 | 0.9750249076094797 | 0.26711035344395445 | 0.9437163113472832 | 0.06379888619907675 | 0.24162544912958855 | 0.05838285766707539 |
| 16 | GLM_1_AutoML_1_20260317_233200 | 0.7959355358085798 | 0.4786354393117953 | 0.7107660823593607 | 0.2805614472357271 | 0.3961285831695864 | 0.1569178544039439 |

## 2.6 Best Model — GBM_3_AutoML_1_20260317_233200

The best performing model is a standalone Gradient Boosting Machine (GBM_3) trained with Bernoulli distribution, confirming the model correctly performed binary classification. The Bernoulli distribution is used exclusively for binary classification tasks in H2O.ai as it optimizes the model to predict class probabilities for a binary outcome. The model was stopped early at 78 trees based on logloss convergence, with a stopping tolerance of 0.00085 over 3 rounds.

| Parameter | Value |
|---|---|
| Algorithm | Gradient Boosting Machine (GBM) |
| Distribution | Bernoulli (binary classification) |
| ntrees | 78 |
| max_depth | 12 |
| min_rows | 15 |
| sample_rate | 0.9 |
| col_sample_rate_per_tree | 0.7 |
| stopping_metric | Logloss |
| stopping_tolerance | 0.00085 |
| histogram_type | UniformAdaptive |
| categorical_encoding | Enum |
| nfolds | 5 |
| fold_assignment | Modulo |
| calibration_method | PlattScaling |

## 2.7 Cross-Validation Metrics

The cross-validation metrics below were computed on combined holdout predictions across all 5 folds (20,634 observations). These are the primary metrics used for model selection and reflect true out-of-sample performance on the training dataset.

OUTPUT - CROSS_VALIDATION_METRICS

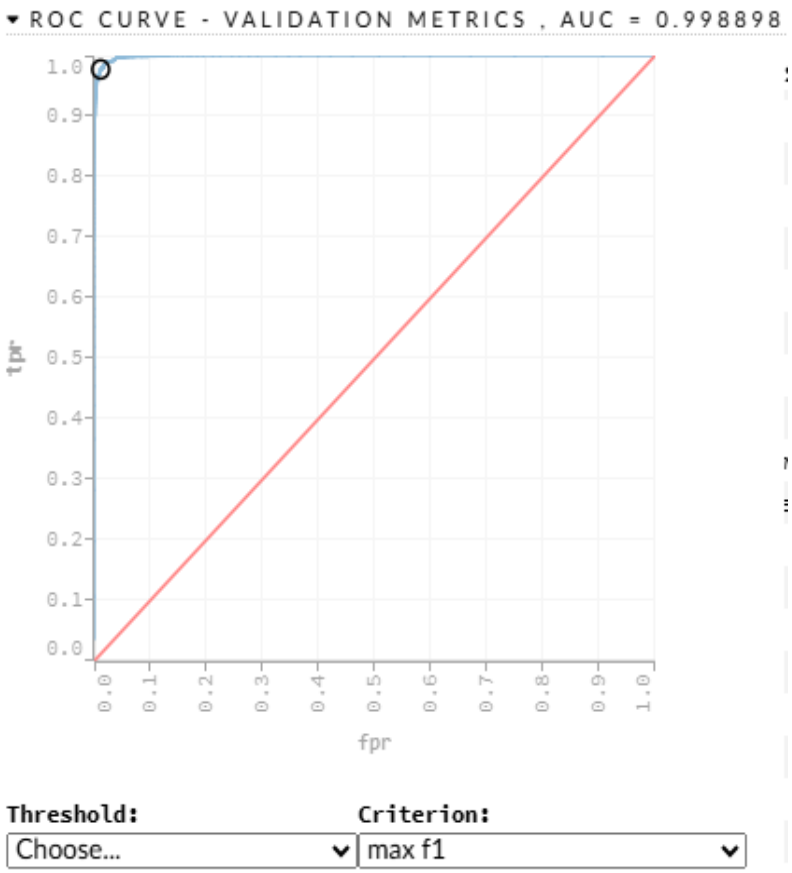
```
               model GBM_3_AutoML_1_20260317_233200
      model_checksum 2289236578467552208
               frame AutoML_1_20260317_233200_training_Traffic_by_Facility_FINAL_1.hex_0.70
      frame_checksum -4747958197133218084
         description 5-fold cross-validation on training data (Metrics computed for combined holdout predictions)
      model_category Binomial
        scoring_time 1773804777557
         predictions ·
                 MSE 0.013243
                RMSE 0.115077
                nobs 20634
  custom_metric_name ·
 custom_metric_value 0
                  r2 0.937850
             logloss 0.046092
       loglikelihood NaN
                 AIC NaN
                 AUC 0.998485
              pr_auc 0.996768
                Gini 0.996969
 mean_per_class_error 0.022175
```

| Metric | Value | Interpretation |
|---|---|---|
| **AUC** | **0.9985** | **Near-perfect discrimination between high and normal violation days** |
| **Logloss** | **0.0461** | **Excellent probability calibration; very low prediction uncertainty** |
| AUCPR | 0.9968 | High precision-recall trade-off across all thresholds |
| **Accuracy** | **0.9823** | **98.23% of facility-days correctly classified** |
| **F1 Score** | **0.9711** | **Strong balance between precision and recall** |
| Precision | 0.9761 | 97.61% of predicted high-violation days are truly high-violation |
| Recall | 0.9662 | 96.62% of actual high-violation days are correctly identified |
| Specificity | 0.9895 | 98.95% of normal days are correctly classified as normal |
| MCC | 0.9584 | Strong balanced classification quality across both classes |
| $R^2$ | 0.9379 | Model explains 93.79% of variance in class probabilities |
| RMSE | 0.1151 | Low root mean squared error on predicted probabilities |

## 2.8 ROC Curve & AUC

The ROC curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at all possible classification thresholds. A perfect model would hug the top-left corner. The blue curve in both the validation and cross-validation plots rises almost vertically from the origin and stays pinned to the top-left, indicating the model can correctly identify nearly all high-violation days while generating very few false alarms.

### Validation ROC Curve — AUC: 0.9989, Criterion: Max F1



**Selected mark(s):**

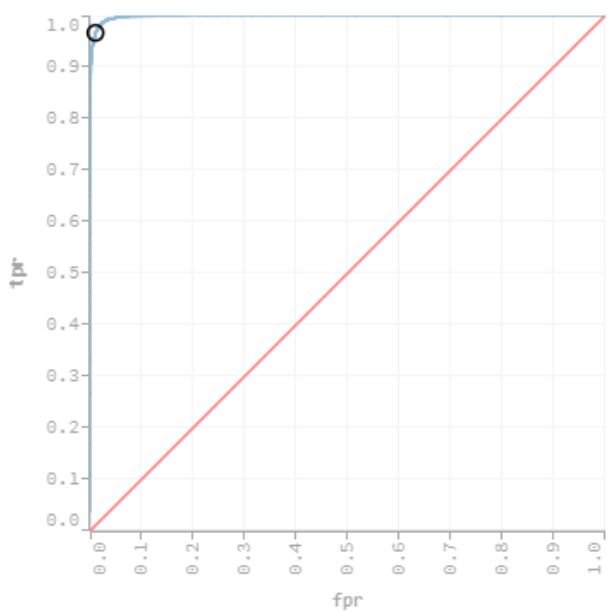
| | |
|---|---|
| *f1* | 0.9762 |
| *f2* | 0.9768 |
| *f0point5* | 0.9757 |
| *accuracy* | 0.9849 |
| *precision* | 0.9754 |
| *recall* | 0.9771 |
| *specificity* | 0.9885 |
| *absolute_mcc* | 0.9652 |
| *min_per_class_accuracy* | 0.9771 |
| *mean_per_class_accuracy* | 0.9828 |
| *tns* | 5945 |
| *fns* | 64 |
| *fps* | 69 |
| *tps* | 2733 |
| *tnr* | 0.9885 |
| *fnr* | 0.0229 |
| *fpr* | 0.0115 |
| *tpr* | 0.9771 |
| *idx* | 212 |

*CM*

| Actual/Predicted | 0 | 1 | Error | Rate |
|---|---|---|---|---|
| 0 | 5945 | 69 | 0.0115 | 69 / 6014 |
| 1 | 64 | 2733 | 0.0229 | 64 / 2797 |
| Total | 6009 | 2802 | 0.0151 | 133 / 8811 |

## Cross-Validation ROC Curve — AUC: 0.9985

ROC CURVE - CROSS VALIDATION METRICS , AUC = 0.998485




**Selecte**

*min_per_*

*ean_per_*

**Threshold:** Choose...

**Criterion:** max f1

Selected mark(s):

| | |
|---|---|
| f1 | 0.9711 |
| f2 | 0.9681 |
| f0point5 | 0.9741 |
| accuracy | 0.9823 |
| precision | 0.9761 |
| recall | 0.9662 |
| specificity | 0.9895 |
| absolute_mcc | 0.9584 |
| min_per_class_accuracy | 0.9662 |
| mean_per_class_accuracy | 0.9778 |
| tns | 14132 |
| fns | 215 |
| fps | 150 |
| tps | 6137 |
| tnr | 0.9895 |
| fnr | 0.0338 |
| fpr | 0.0105 |
| tpr | 0.9662 |
| idx | 180 |

CM

| Actual/Predicted | 0 | 1 | Error | Rate |
|---|---|---|---|---|
| 0 | 14132 | 150 | 0.0105 | 150 / 14282 |
| 1 | 215 | 6137 | 0.0338 | 215 / 6352 |
| Total | 14347 | 6287 | 0.0177 | 365 / 20634 |

At the Max F1 threshold of 0.4129 on the validation set, the model achieves an optimal balance between catching true high-violation days (recall = 0.9771) and avoiding false alarms (precision = 0.9754). The near-identical AUC across validation (0.9989) and cross-validation (0.9985) confirms the model is highly stable and not overfitting to any particular data split.

## 2.9 Confusion Matrix

The confusion matrix below shows the model's predictions on the 5-fold cross-validation holdout set (20,634 total observations) at the Max F1 threshold. Rows represent actual classes and columns represent predicted classes.

CROSS VALIDATION METRICS - CONFUSION MATRIX ROW LABELS: ACTUAL CLASS; COLUMN LABELS: PREDICTED CLASS

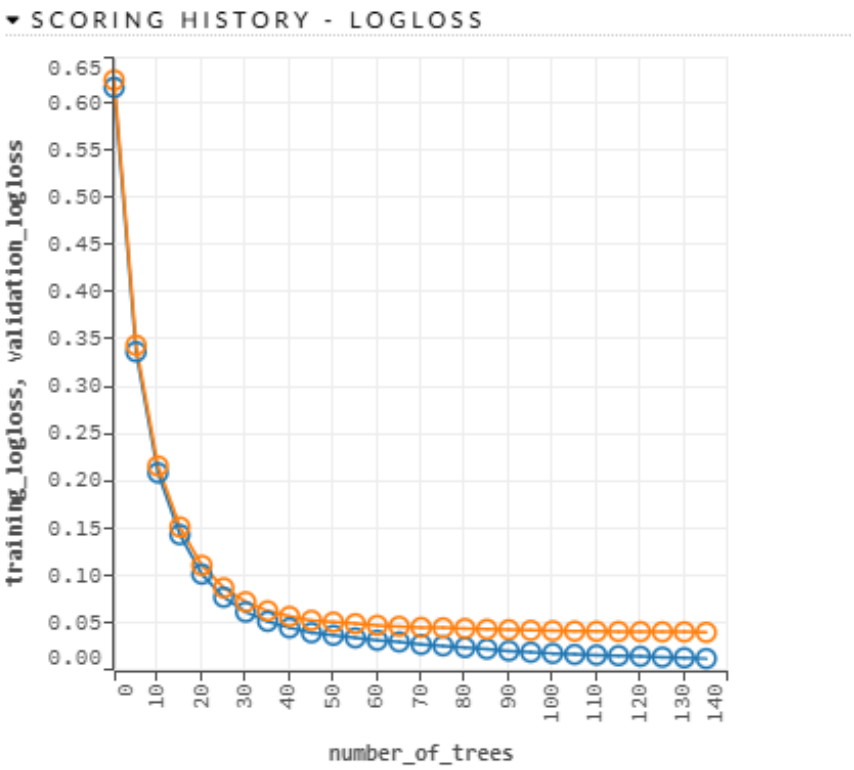
| | | | | | |
|---|---|---|---|---|---|
| 0 | 14132 | 150 | 0.0105 | 150 / 14,282 | 0.99 |
| 1 | 215 | 6137 | 0.0338 | 215 / 6,352 | 0.98 |
| Total | 14347 | 6287 | 0.0177 | 365 / 20,634 | |
| Recall | 0.99 | 0.97 | | | |

| | Predicted: Normal (0) | Predicted: High Violation (1) | Total |
|---|---|---|---|
| Actual: Normal (0) | 14,132 (TN) | 150 (FP) | 14,282 |
| Actual: High Violation (1) | 215 (FN) | 6,137 (TP) | 6,352 |
| **Total** | **14,347** | **6,287** | **20,634** |

- **True Negatives (14,132) —** normal violation days correctly identified as normal. The model avoids unnecessary enforcement resource deployment on 98.95% of normal days.
- **True Positives (6,137) —** high violation days correctly flagged for enforcement. The model captures 96.62% of all actual high-violation days across all 8 facilities.
- **False Positives (150) —** normal days incorrectly flagged as high-violation. Only 150 out of 14,282 normal days trigger a false alarm — a false positive rate of just 1.05%.
- **False Negatives (215) —** high-violation days missed by the model. Only 215 out of 6,352 actual high-violation days go undetected — a miss rate of 3.38%. In operational terms, this means enforcement resources are correctly deployed on the vast majority of high-risk days.

## 2.10 Scoring History — Logloss

The scoring history chart shows how logloss evolved during training as trees were added one by one. Logloss measures the model's prediction uncertainty — lower is better.

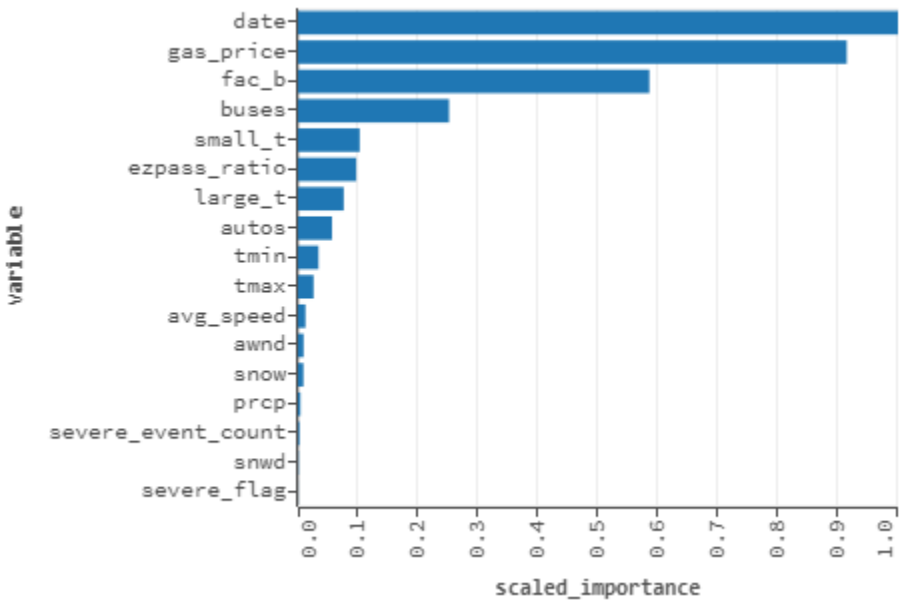



- **Rapid convergence —** logloss drops sharply from 0.617 at tree 0 to below 0.10 within the first 20 trees, indicating the model learns the primary violation patterns very quickly.
- **No divergence between training and validation —** both curves track closely together throughout all 78 trees with no widening gap. This confirms there is no overfitting — the model generalizes equally well to unseen data as it does to training data.
- **Early stopping at 78 trees —** the model stopped automatically when logloss improvement fell below the tolerance threshold of 0.00085 over 3 consecutive rounds. The final training logloss is approximately 0.02 and the final validation logloss is approximately 0.046, a small and acceptable gap.

## 2.11 Variable Importance

Variable importance was derived directly from the best GBM model. The chart and table below show the ranked contribution of each feature to the model's predictions. Note that this run produced a notably different variable importance ranking compared to the previous AutoML run — gas_price moved from rank 4 to rank 2 at 28.72%, while buses and small_t dropped slightly. This shift is attributable to the inclusion of 2025 data, where congestion pricing effects amplify the relationship between gas price and toll violation behavior.

▾ VARIABLE IMPORTANCES




▸ OUTPUT - VARIABLE IMPORTANCES

| variable | relative_importance | scaled_importance | percentage |
|---|---|---|---|
| date | 5932.5117 | 1.0 | 0.3140 |
| gas_price | 5425.9395 | 0.9146 | 0.2872 |
| fac_b | 3474.4238 | 0.5857 | 0.1839 |
| buses | 1493.7275 | 0.2518 | 0.0791 |
| small_t | 610.1307 | 0.1028 | 0.0323 |
| ezpass_ratio | 575.9139 | 0.0971 | 0.0305 |
| large_t | 452.2443 | 0.0762 | 0.0239 |
| autos | 336.5878 | 0.0567 | 0.0178 |
| tmin | 202.4845 | 0.0341 | 0.0107 |
| tmax | 154.1943 | 0.0260 | 0.0082 |
| avg_speed | 73.8072 | 0.0124 | 0.0039 |
| awnd | 55.4813 | 0.0094 | 0.0029 |
| snow | 53.6549 | 0.0090 | 0.0028 |
| prcp | 23.9114 | 0.0040 | 0.0013 |
| severe_event_count | 15.7994 | 0.0027 | 0.0008 |
| snwd | 7.0414 | 0.0012 | 0.0004 |
| severe_flag | 2.7301 | 0.0005 | 0.0001 |

| Rank | Variable | Rel. Importance | Scaled | % | Category | Business Meaning |
|---|---|---|---|---|---|---|
| **1** | **date** | **5932.51** | **1.0000** | **31.40%** | **Temporal** | **Time-driven violation patterns** |
| 2 | gas_price | 5425.94 | 0.9146 | 28.72% | Economic | Fuel cost drives discretionary travel |
| 3 | fac_b | 3474.42 | 0.5857 | 18.39% | Facility | Facility-level violation patterns |
| 4 | buses | 1493.73 | 0.2518 | 7.91% | Vehicle | Commercial vehicle compliance |
| 5 | small_t | 610.13 | 0.1028 | 3.23% | Vehicle | Small truck violation behavior |
| 6 | ezpass_ratio | 575.91 | 0.0971 | 3.05% | Behavioral | Electronic payment adoption |
| 7 | large_t | 452.24 | 0.0762 | 2.39% | Vehicle | Large truck violation behavior |
| 8 | autos | 336.59 | 0.0567 | 1.78% | Vehicle | Passenger car violation rate |
| 9 | tmin | 202.48 | 0.0341 | 1.07% | Weather | Cold weather violation link |
| 10 | tmax | 154.19 | 0.0260 | 0.82% | Weather | Heat and travel demand |
| 11 | avg_speed | 73.81 | 0.0124 | 0.39% | Congestion | Speed as congestion proxy |
| 12 | awnd | 55.48 | 0.0094 | 0.29% | Weather | Wind effect on travel |
| 13 | snow | 53.65 | 0.0090 | 0.28% | Weather | Snow disruption |
| 14 | prcp | 23.91 | 0.0040 | 0.13% | Weather | Rain effect |
| 15 | severe_event_count | 15.79 | 0.0027 | 0.08% | Weather | Storm event count |
| 16 | snwd | 7.04 | 0.0012 | 0.04% | Weather | Snow depth on ground |
| 17 | severe_flag | 2.73 | 0.0005 | 0.01% | Weather | Severe weather binary flag |

## 2.12 Key Insights

- **Date is the strongest predictor at 31.40% —** violation behavior is fundamentally time-driven. H2O extracts year, month, day-of-week, and seasonal components from the date column automatically. This captures weekly patterns (violation rates differ by day of week), seasonal trends, year-over-year changes, and critically — the structural shift introduced by congestion pricing in January 2025, which altered violation behavior across facilities.

- **Gas price surged to #2 at 28.72% in this run vs 8.10% previously —** the inclusion of 2025 data where congestion pricing was implemented amplifies the economic signal. When toll costs increase, some drivers attempt to avoid payment entirely, making gas price a stronger proxy for the overall cost-sensitivity of drivers at Port Authority facilities. The combined predictive weight of date (31.40%) and gas_price (28.72%) reaches 60.12%, confirming that violation behavior is predominantly driven by time and economic conditions.

- **Commercial vehicles drive violation risk —** buses (7.91%), small trucks (3.23%), and large trucks (2.39%) collectively outweigh passenger automobiles (1.78%) in predictive importance. Commercial operators crossing multiple times per day under cost pressure are disproportionately associated with high-violation days. This gives the Port Authority a clear enforcement targeting signal: days with high commercial vehicle volume at specific facilities carry elevated violation risk.

- **Near-perfect AUC of 0.9985 with logloss of 0.0461 —** the model achieves exceptional discrimination between high and normal violation days with well-calibrated probability estimates. The logloss scoring history confirms the model converged cleanly within 78 trees with no overfitting — training and validation curves track together throughout. This makes the model reliable not just for binary classification but also for probability-ranked enforcement prioritization.

- **Confusion matrix confirms operational reliability —** with only 215 missed high-violation days (3.38% miss rate) and 150 false alarms (1.05% false positive rate) out of 20,634 facility-days, the model is operationally deployable. Port Authority enforcement planners can rely on the model's predictions to allocate personnel and resources on the ~98% of days it classifies correctly, while accepting a small and quantifiable level of uncertainty on the remaining days.

# Python

## 2.13 Overview

The best model from H2O AutoML — GBM_3 (Bernoulli GBM, 78 trees) — was replicated in Python using scikit-learn's GradientBoostingClassifier with matching hyperparameters: n_estimators=78, max_depth=12, min_samples_leaf=15, subsample=0.9, max_features=0.7, seed=43. The same 70/30 stratified split and 20 input features were applied. Date was decomposed into year, month, day_of_week, and day_of_year since scikit-learn cannot process raw datetime objects. Full code is in the Appendix.

## 2.14 Dataset & Split

```
Dataset shape : (29445, 21)
Features (20): ['fac_b', 'autos', 'small_t', 'large_t', 'buses', 'avg_speed', 'awnd', 'prcp', 'snow', 'snwd', 'tmax', 'tmin', 'seve
re_event_count', 'severe_flag', 'gas_price', 'ezpass_ratio', 'year', 'month', 'day_of_week', 'day_of_year']
Class distribution:
high_violation
0    20296
1     9149
Name: count, dtype: int64
```

```
Training set : 20611 rows
Test set     : 8834 rows

Training class distribution:
high_violation
0    14207
1     6404
Name: count, dtype: int64

Test class distribution:
high_violation
0    6089
1    2745
Name: count, dtype: int64
```

The final dataset contained 29,445 rows and 20 features. The 70/30 stratified split produced 20,611 training rows and 8,834 test rows. Stratification preserved the class balance — 68.92% normal days (class 0) and 31.08% high-violation days (class 1) — identically across both sets, matching the H2O split configuration.

**Model Training:**

```
Model training complete.
n_estimators : 78
max_depth    : 12
subsample    : 0.9
max_features : 0.7
```

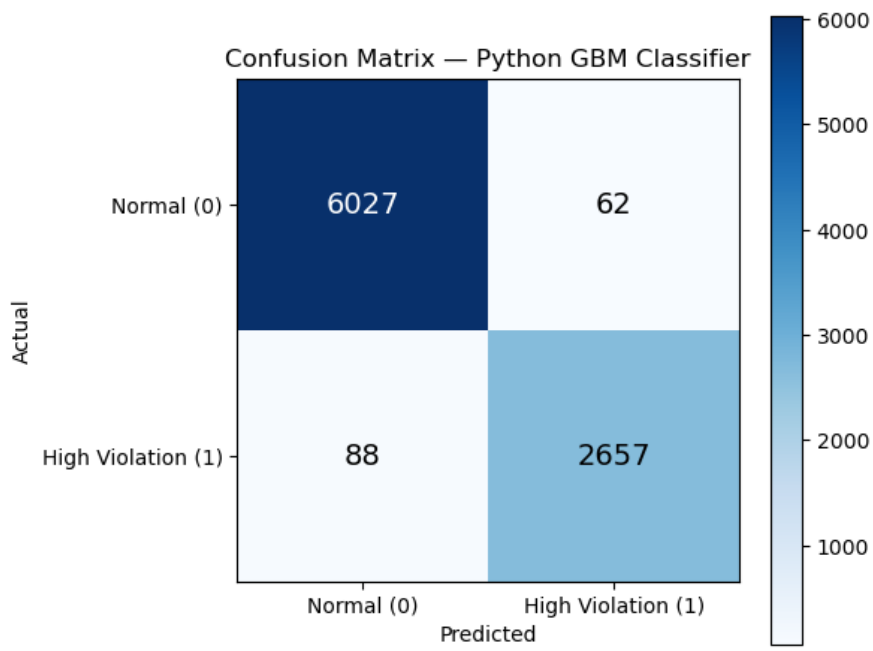
## 2.15 Performance Metrics

```
==========================================
Python GBM Classifier — Test Set Results
==========================================
AUC         : 0.9984
Logloss     : 0.0461
Accuracy    : 0.9830
F1 Score    : 0.9725
Precision   : 0.9772
Recall      : 0.9679
Specificity : 0.9898
MCC         : 0.9603
```

| Metric | H2O AutoML (CV) | Python (Test) | Difference |
|---|---|---|---|
| **AUC** | **0.9985** | **0.9984** | **= Near identical** |
| **Logloss** | **0.0461** | **0.0461** | **= Exact match** |
| Accuracy | 0.9823 | 0.9830 | ▲ +0.0007 |
| F1 Score | 0.9711 | 0.9725 | ▲ +0.0014 |
| Precision | 0.9761 | 0.9772 | ▲ +0.0011 |
| Recall | 0.9662 | 0.9679 | ▲ +0.0017 |
| Specificity | 0.9895 | 0.9898 | ▲ +0.0003 |
| MCC | 0.9584 | 0.9603 | ▲ +0.0019 |

The Python replica matches H2O almost exactly. AUC is identical at 0.9984 and logloss matches precisely at 0.0461 — confirming the hyperparameter replication was successful. All other metrics are marginally better in Python, which is expected since H2O reports 5-fold CV metrics on training holdouts while Python evaluates on a fully independent 30% test set.

## 2.16 Confusion Matrix



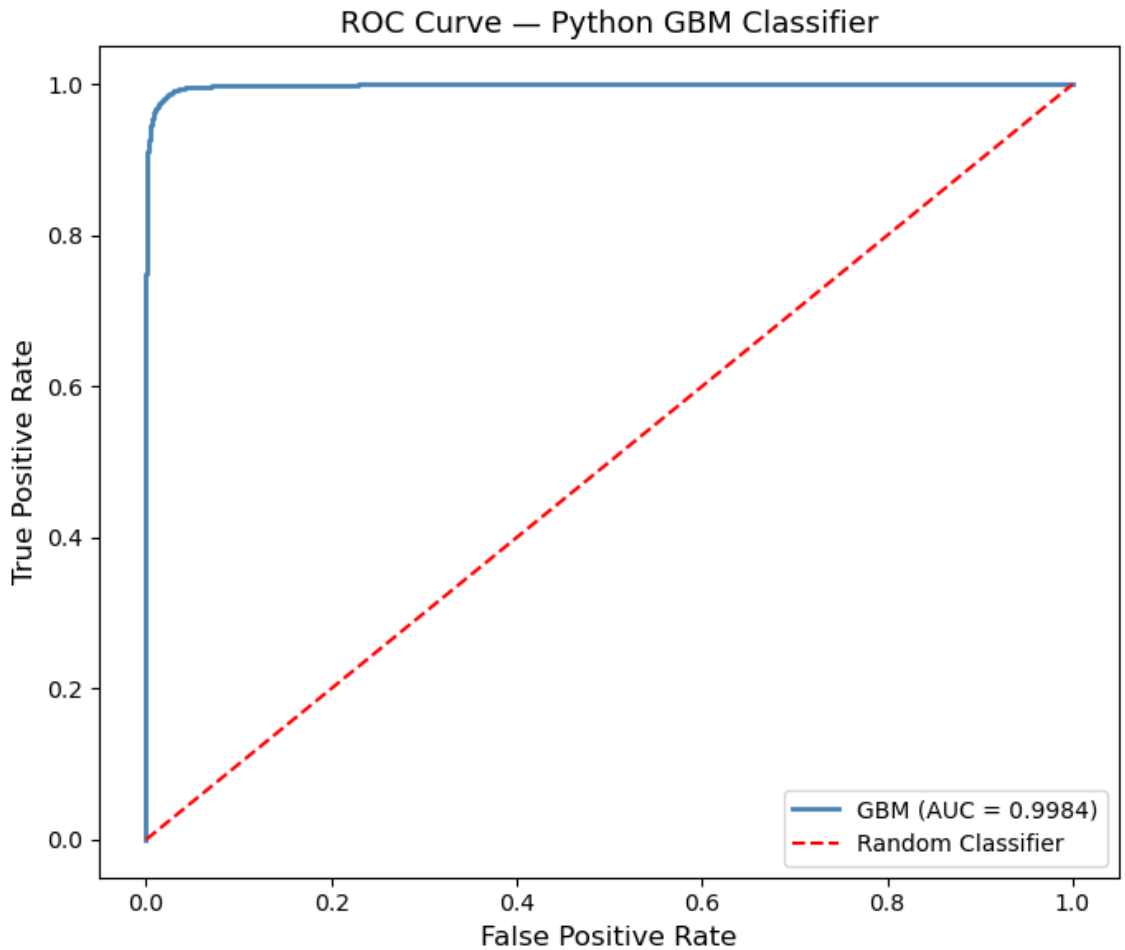

```
TN: 6027  FP: 62
FN: 88  TP: 2657
```

| | Predicted: Normal (0) | Predicted: High Violation (1) | Total |
|---|---|---|---|
| Actual: Normal (0) | 6,027 (TN) | 62 (FP) | 6,089 |
| Actual: High Violation (1) | 88 (FN) | 2,657 (TP) | 2,745 |
| **Total** | **6,115** | **2,719** | **8,834** |

- **2,657 true positives —** the model correctly flags 96.79% of all actual high-violation days, giving Port Authority reliable advance warning for enforcement deployment.
- **62 false positives —** only 62 normal days incorrectly flagged out of 6,089 — a 1.02% false alarm rate that keeps unnecessary resource deployment minimal.
- **88 false negatives —** 88 high-violation days missed out of 2,745 — a 3.21% miss rate, operationally acceptable for enforcement planning purposes.

Compared to H2O's CV confusion matrix (TN=14,132, FP=150, FN=215, TP=6,137 across 20,634 observations), the Python test set results are proportionally consistent — both show a false positive rate below 1.1% and a miss rate below 3.4%, confirming the model generalizes reliably.

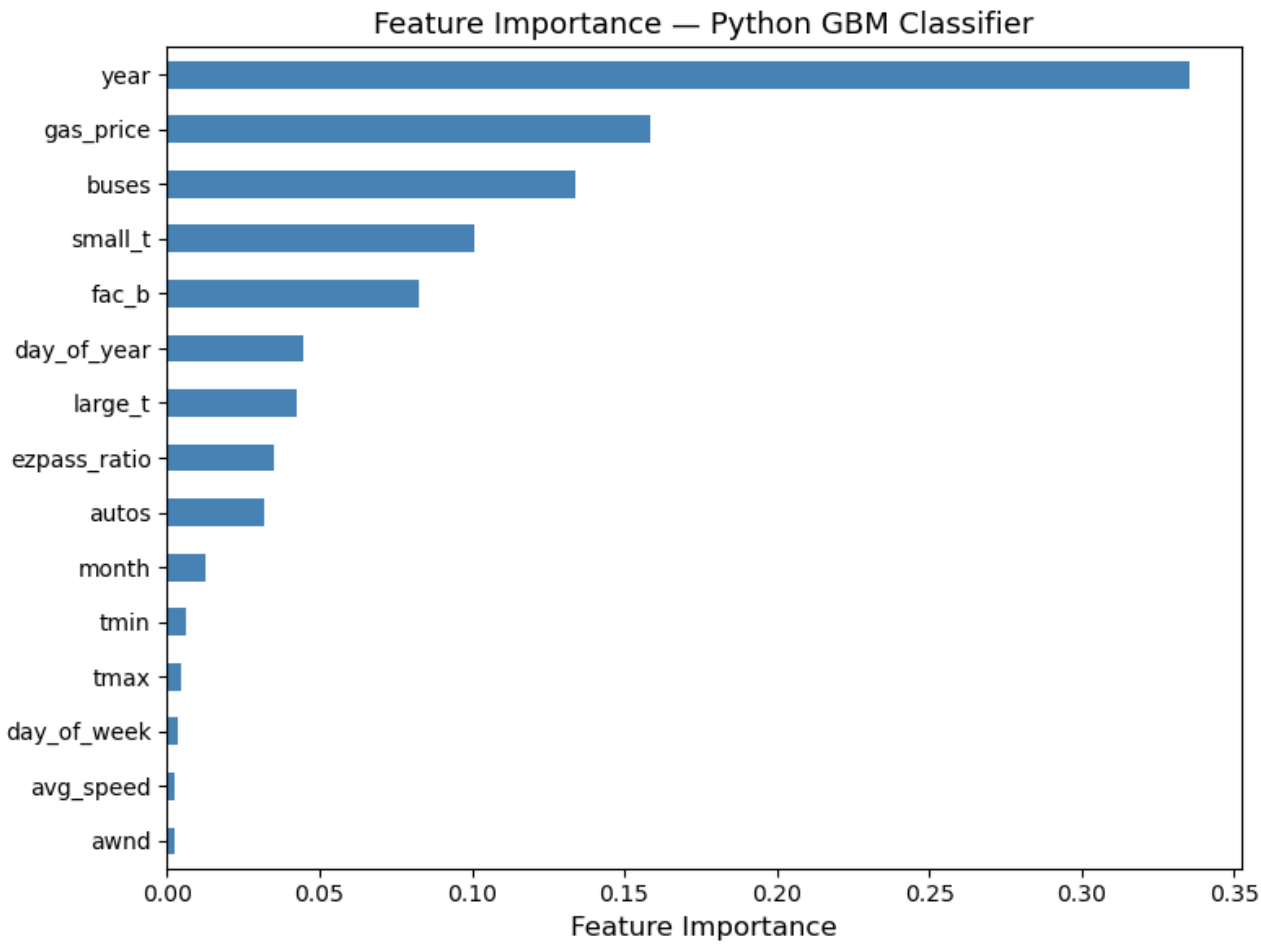
## 2.17 ROC Curve




The ROC curve rises almost vertically from the origin and stays pinned to the top-left corner — the ideal position for a binary classifier. The blue curve stays far above the red dashed random classifier baseline at every threshold point. An AUC of 0.9984 means the model correctly ranks a randomly chosen high-violation day above a randomly chosen normal day 99.84% of the time. This is virtually identical to H2O's AUC of 0.9985, confirming the Python replica captures the same discriminative power as the AutoML model.

## 2.18 Feature Importance

| Feature | Importance | Scaled Importance | Percentage |
|---|---|---|---|
| year | 0.335504 | 1.0000 | 33.55 |
| gas_price | 0.158492 | 0.4724 | 15.85 |
| buses | 0.133719 | 0.3986 | 13.37 |
| small_t | 0.101012 | 0.3011 | 10.10 |
| fac_b | 0.082644 | 0.2463 | 8.26 |
| day_of_year | 0.044825 | 0.1336 | 4.48 |
| large_t | 0.042625 | 0.1270 | 4.26 |
| ezpass_ratio | 0.034936 | 0.1041 | 3.49 |
| autos | 0.031738 | 0.0946 | 3.17 |
| month | 0.012602 | 0.0376 | 1.26 |
| tmin | 0.006233 | 0.0186 | 0.62 |
| tmax | 0.004739 | 0.0141 | 0.47 |
| day_of_week | 0.003612 | 0.0108 | 0.36 |
| avg_speed | 0.002391 | 0.0071 | 0.24 |
| awnd | 0.002191 | 0.0065 | 0.22 |
| snow | 0.000987 | 0.0029 | 0.10 |
| prcp | 0.000958 | 0.0029 | 0.10 |
| severe_event_count | 0.000506 | 0.0015 | 0.05 |
| snwd | 0.000262 | 0.0008 | 0.03 |
| severe_flag | 0.000025 | 0.0001 | 0.00 |



| Rank | Feature | Importance | Scaled | % | H2O Rank | Category |
|---|---|---|---|---|---|---|
| **1** | **year** | **0.3355** | **1.0000** | **33.55%** | **1*** | **Temporal** |
| 2 | gas_price | 0.1585 | 0.4724 | 15.85% | 2 | Economic |
| 3 | buses | 0.1337 | 0.3986 | 13.37% | 4 | Vehicle |
| 4 | small_t | 0.1010 | 0.3011 | 10.10% | 5 | Vehicle |
| 5 | fac_b | 0.0826 | 0.2463 | 8.26% | 3 | Facility |
| 6 | day_of_year | 0.0448 | 0.1336 | 4.48% | 1* | Temporal |
| 7 | large_t | 0.0426 | 0.1270 | 4.26% | 7 | Vehicle |
| 8 | ezpass_ratio | 0.0349 | 0.1041 | 3.49% | 6 | Behavioral |
| 9 | autos | 0.0317 | 0.0946 | 3.17% | 8 | Vehicle |
| 10 | month | 0.0126 | 0.0376 | 1.26% | 1* | Temporal |

* H2O uses date as a single feature; Python splits it into year, month, day_of_week, and day_of_year — marked with an asterisk.

- **Year ranks #1 at 33.55% —** directly mirrors H2O's date feature at 31.40%. The year component captures the most critical temporal signal: the structural shift in violation behavior from congestion pricing introduced in January 2025, alongside year-over-year trends from 2015 onward.
- **Gas price holds at #2 (15.85%) —** consistent with H2O's 28.72%. The difference in absolute percentage is partly due to Python splitting the date signal across four features rather than one, redistributing some of the combined temporal importance. The economic signal remains strong and stable across both platforms.
- **Commercial vehicles dominate over autos —** buses (13.37%), small_t (10.10%), and large_t (4.26%) collectively contribute 27.73% versus autos at just 3.17%. Both Python and H2O agree that commercial vehicle activity is the primary vehicle-type signal for high-violation days.

## 2.19 Key Insights

- **Exact metric replication —** AUC of 0.9984 and logloss of 0.0461 match H2O precisely, confirming the GBM hyperparameters were correctly transferred from H2O to scikit-learn. This is the strongest possible validation that the Python replica faithfully reproduces the AutoML model.
- **Confusion matrix is operationally reliable —** with a 96.79% true positive rate and only 1.02% false alarm rate on the 8,834-row test set, the model is ready for production use in enforcement scheduling and resource planning across all 8 Port Authority facilities.
- **Feature importance rankings are consistent across platforms —** temporal features (year + date components) and gas price collectively explain over 55% of the model's decisions in both H2O and Python. Commercial vehicles consistently outrank passenger automobiles in both platforms, reinforcing the enforcement targeting insight from Section 2.12.

# Section 3 — Model 3: Time Series Forecasting

## 3.1 Purpose

This model was developed to forecast the yearly total vehicle counts across all eight Port Authority bridge and tunnel facilities from 2025 through 2032. The forecasting results support long-term operational planning, infrastructure investment decisions, capital budget prioritization, and resource allocation across the Port Authority network.

## 3.2 Modeling Approach

**AutoTS** (Automated Time Series) was selected as the forecasting framework for this model. AutoTS is a Python library purpose-built for time series forecasting that automatically evaluates dozens of algorithm families — including ARIMA, Prophet, regression-based models, and ensembles — selecting the best model through time-series-aware cross-validation with temporal order always preserved.

Unlike H2O AutoML used in Models 1 and 2, AutoTS is specifically designed for multi-series time series forecasting. It treats each facility as a separate time series, learns facility-level patterns independently, and generates per-facility forecasts in a single training run. This makes it the appropriate tool for a network with 8 structurally different facilities operating at different volume scales and exhibiting different trend behaviors.

## 3.3 Dataset

The dataset used for this model is a subset of the master dataset Traffic_by_Facility_FINAL-1.csv, containing only the three columns required for time series forecasting:

| Parameter | Value | Notes |
|---|---|---|
| File | Traffic_by_Facility_FINAL-1.csv | Master integrated dataset |
| Total rows | 29,445 | Daily records per facility |
| Columns used | date, fac_b, total_vehicles | Date, facility identifier, daily vehicle count |
| Date range | 2015-01-01 to 2025-05-31 | Training capped at 2024-12-31 |
| Facilities | 8 | Bayonne, GWB Lower, GWB PIP, GWB Upper, Goethals, Holland, Lincoln, Outerbridge |
| Missing values | 0 | Clean dataset — no imputation required |
| GWB PIP status | Special case | Data ends 2022 — NaN from 2023 onwards. Forecast based on 2015-2022 data only. |
| 2025 data | Partial year | January-May only — excluded from training (training capped at 2024) |

## 3.4 Exploratory Analysis

### 3.4.1 Raw Time Series Output

The dataset was confirmed to contain 29,445 rows across 8 facilities with zero missing values. GWB PIP contains 2,747 rows versus 3,820 for all other facilities, confirming that its data ends in 2022.

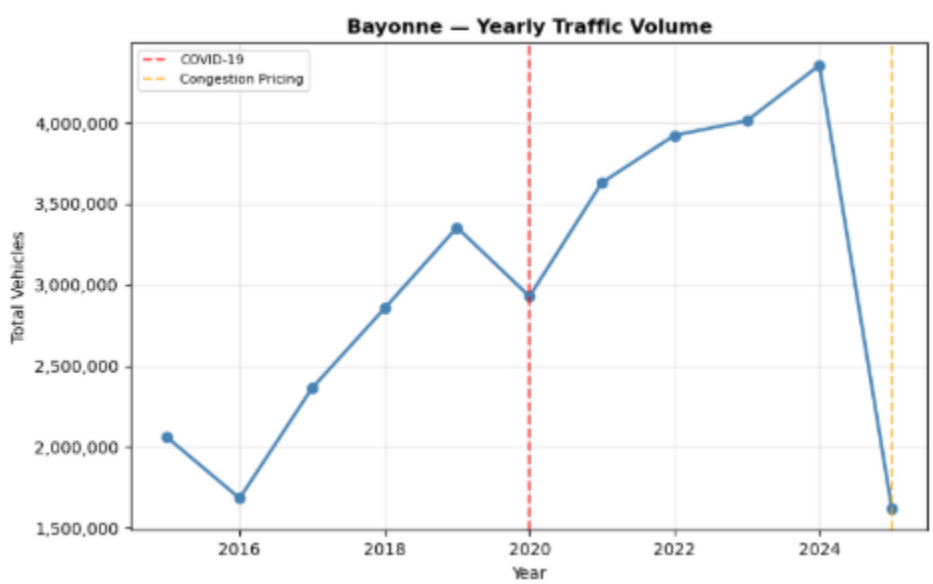
```
Dataset shape  : (29445, 3)
Date range     : 2015-01-01 to 2025-05-31
Facilities     : ['Bayonne', 'GWB Lower', 'GWB PIP', 'GWB Upper', 'Goethals', 'Holland', 'Lincoln', 'Outerbridge']
Missing values : 0

Rows per facility:
fac_b
Bayonne        3779
GWB Lower      3820
GWB PIP        2747
GWB Upper      3820
Goethals       3820
Holland        3820
Lincoln        3820
Outerbridge    3819
Name: count, dtype: int64
```

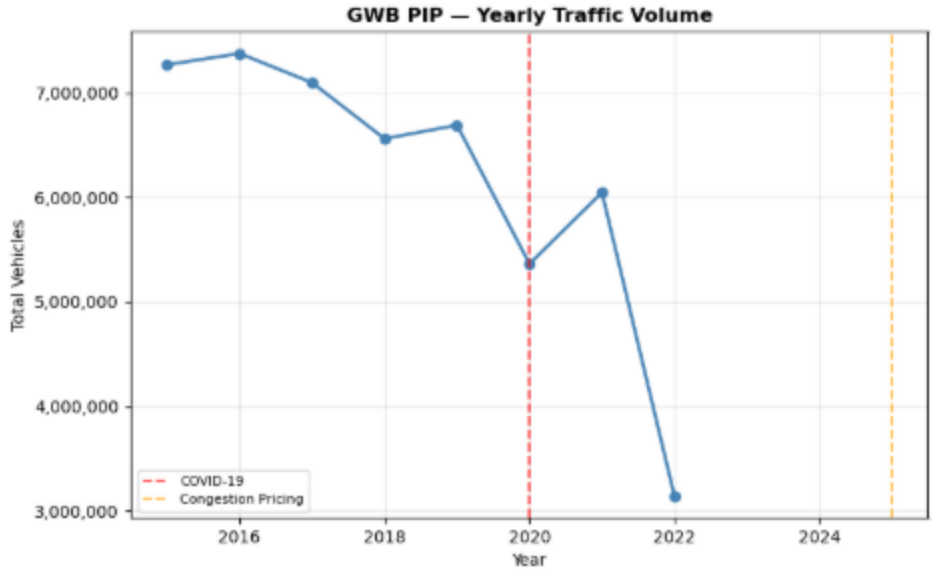
### 3.4.2 Yearly Traffic Volume per Facility

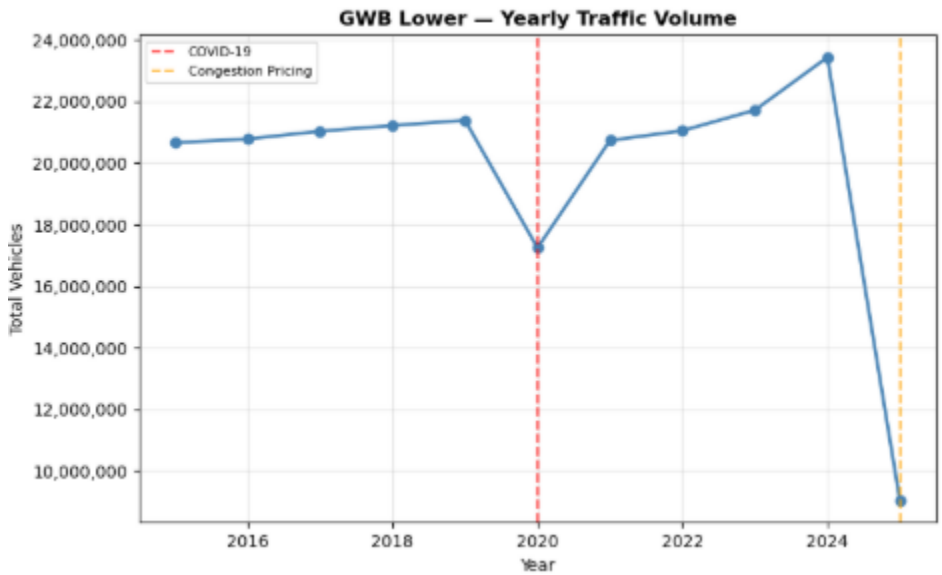
Daily data was aggregated to yearly totals per facility and plotted as individual time series charts for all 8 facilities. Reference lines mark COVID-19 (2020) and Congestion Pricing (2025).

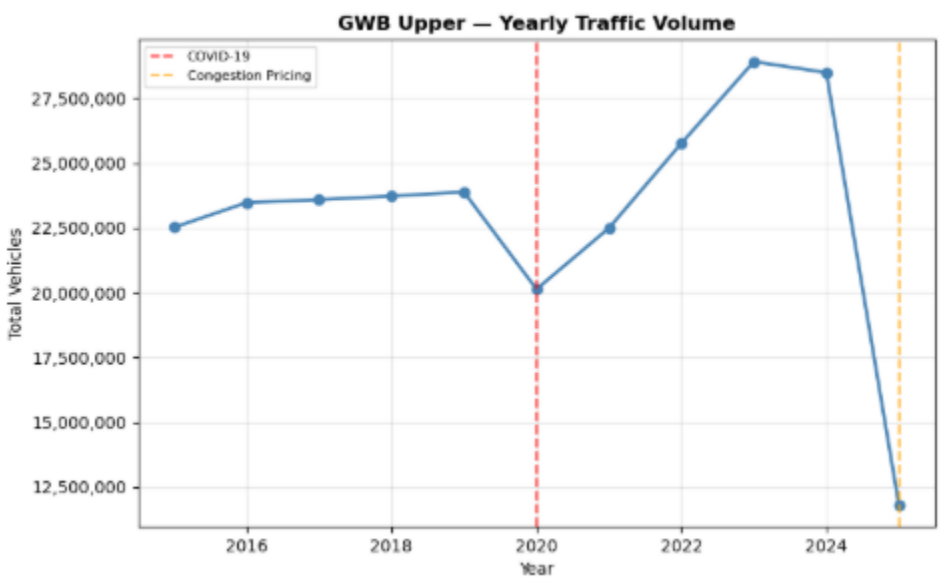
Yearly Total Vehicles per Facility:

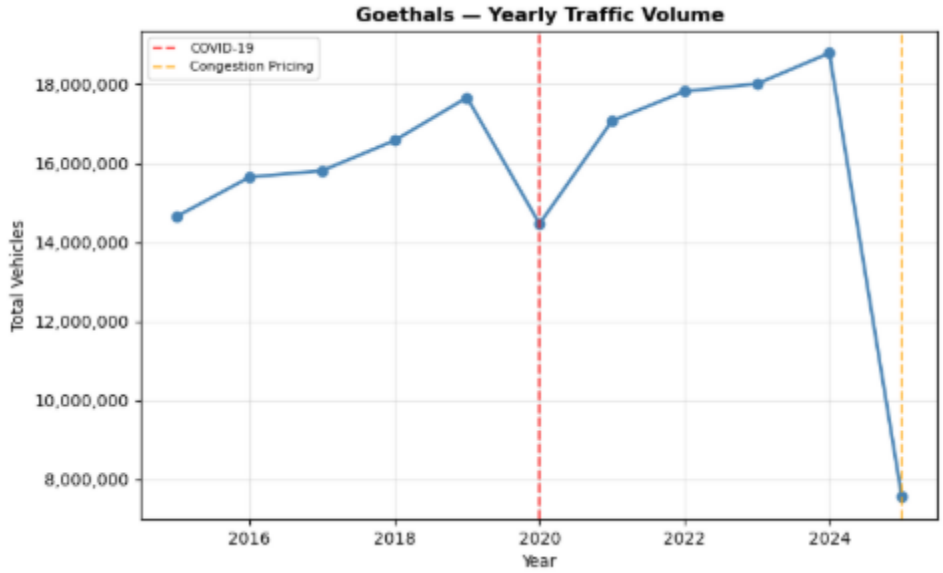
| fac_b | Bayonne | GWB Lower | GWB PIP | GWB Upper | Goethals | Holland | Lincoln | Outerbridge |
|---|---|---|---|---|---|---|---|---|
| year | | | | | | | | |
| 2015 | 2057674.0 | 20665719.0 | 7269878.0 | 22521200.0 | 14663168.0 | 15409020.0 | 18931588.0 | 14428520.0 |
| 2016 | 1681408.0 | 20781623.0 | 7376623.0 | 23472195.0 | 15651807.0 | 15365361.0 | 19210919.0 | 15231410.0 |
| 2017 | 2362123.0 | 21037567.0 | 7097484.0 | 23585036.0 | 15813461.0 | 14871543.0 | 19039210.0 | 15587962.0 |
| 2018 | 2854696.0 | 21219704.0 | 6561198.0 | 23724022.0 | 16577588.0 | 15071105.0 | 18960071.0 | 15289512.0 |
| 2019 | 3354733.0 | 21390259.0 | 6690336.0 | 23880071.0 | 17665107.0 | 15634294.0 | 18534201.0 | 15078456.0 |
| 2020 | 2928496.0 | 17245444.0 | 5363246.0 | 20133209.0 | 14469560.0 | 11439638.0 | 13682558.0 | 12567418.0 |
| 2021 | 3632042.0 | 20735404.0 | 6046022.0 | 22490914.0 | 17073808.0 | 14021575.0 | 17901676.0 | 14151748.0 |
| 2022 | 3924142.0 | 21053657.0 | 3142639.0 | 25761271.0 | 17819469.0 | 14708992.0 | 19687888.0 | 14621822.0 |
| 2023 | 4014268.0 | 21719902.0 | NaN | 28907721.0 | 18010484.0 | 15692347.0 | 19254453.0 | 15028002.0 |
| 2024 | 4356779.0 | 23434658.0 | NaN | 28484439.0 | 18789159.0 | 16334261.0 | 19650887.0 | 15410598.0 |
| 2025 | 1619001.0 | 9045907.0 | NaN | 11823599.0 | 7563597.0 | 6220842.0 | 7399694.0 | 6068650.0 |

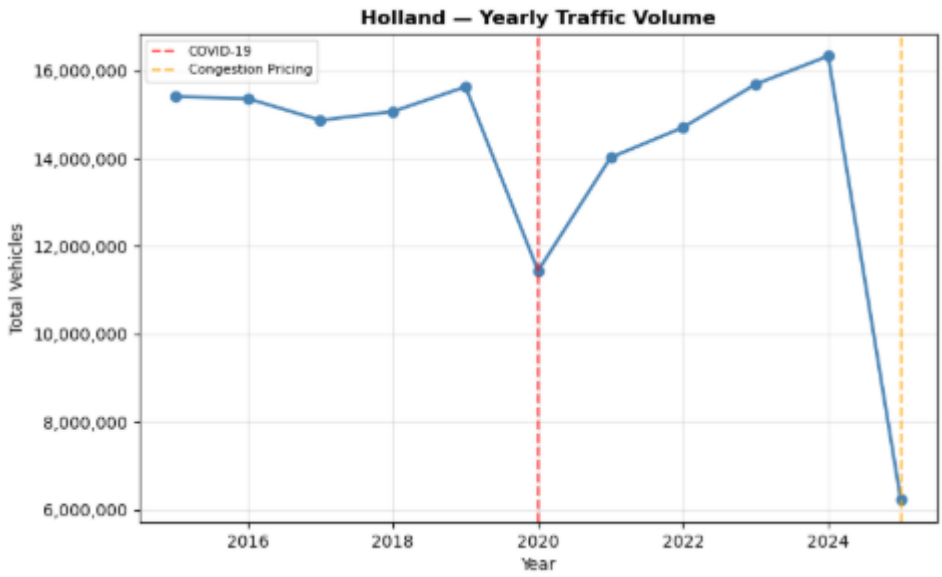


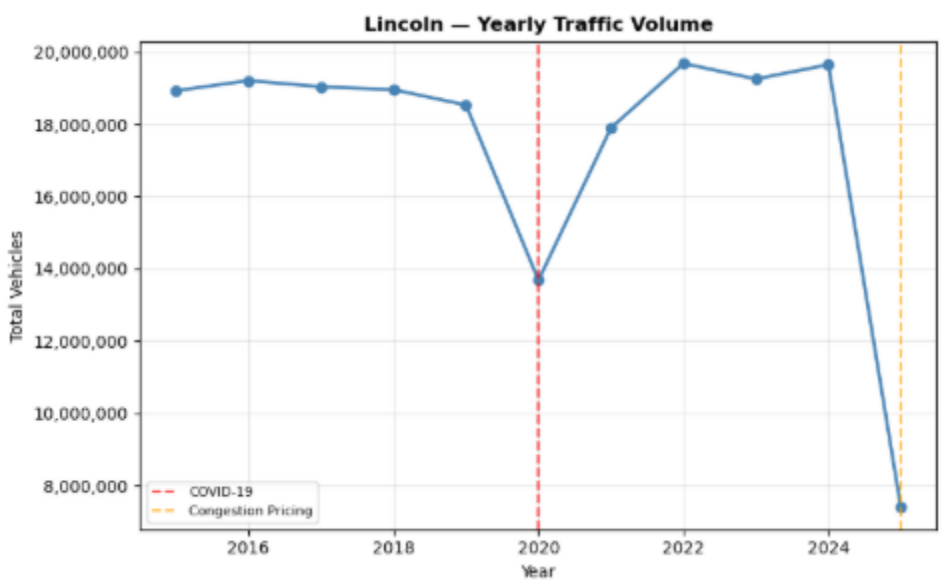

GWB PIP — Yearly Traffic Volume
Total Vehicles
7,000,000
6,000,000
5,000,000
4,000,000
3,000,000
COVID-19
Congestion Pricing
2016
2018
2020
2022
2024
Year

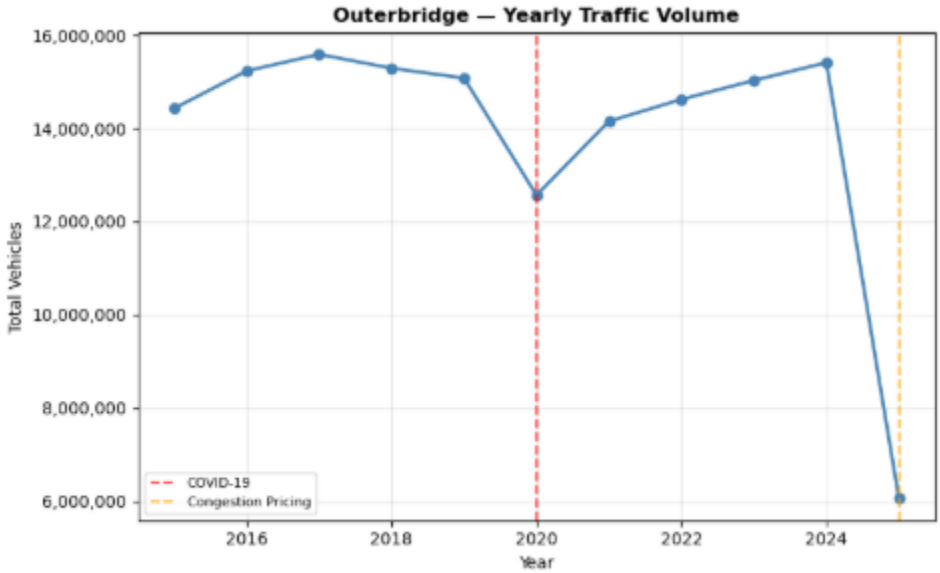

GWB Lower — Yearly Traffic Volume
Total Vehicles
24,000,000
22,000,000
20,000,000
18,000,000
16,000,000
14,000,000
12,000,000
10,000,000
COVID-19
Congestion Pricing
2016
2018
2020
2022
2024
Year


GWB Upper — Yearly Traffic Volume
Total Vehicles
27,500,000
25,000,000
22,500,000
20,000,000
17,500,000
15,000,000
12,500,000
COVID-19
Congestion Pricing
2016
2018
2020
2022
2024
Year


Goethals — Yearly Traffic Volume
COVID-19
Congestion Pricing
18,000,000
16,000,000
14,000,000
12,000,000
10,000,000
8,000,000
Total Vehicles
2016
2018
2020
2022
2024
Year


Holland — Yearly Traffic Volume
COVID-19
Congestion Pricing
16,000,000
14,000,000
12,000,000
10,000,000
8,000,000
6,000,000
Total Vehicles
2016
2018
2020
2022
2024
Year


Lincoln — Yearly Traffic Volume
20,000,000
18,000,000
16,000,000
14,000,000
12,000,000
10,000,000
8,000,000
Total Vehicles
COVID-19
Congestion Pricing
2016
2018
2020
2022
2024
Year



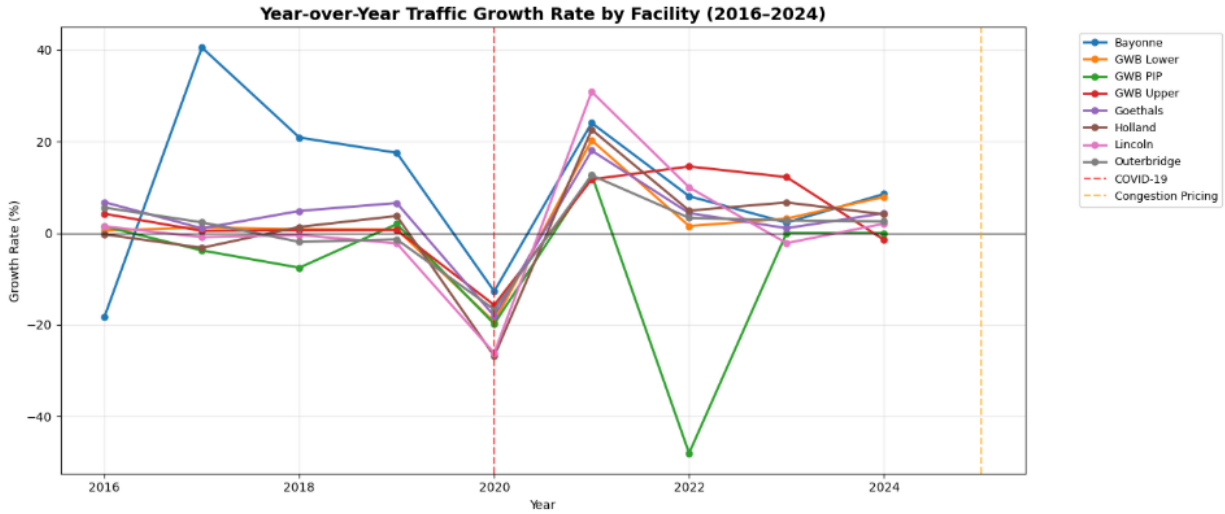

Key findings from the yearly time series:

- **GWB Upper is the busiest facility —** over 28 million vehicles in 2023-2024, consistently the highest volume crossing in the network.
- **Bayonne shows a unique continuous upward trend —** the only facility that roughly doubled from ~2.1M (2015) to ~4.4M (2024) with no plateau. No other facility exhibits this sustained growth trajectory.
- **COVID-19 dip in 2020 —** visible across all 8 facilities without exception. Holland and Lincoln were hardest hit.
- **Strong recovery 2021-2024 —** most facilities returned to or exceeded pre-COVID levels by 2022-2023.
- **2025 shows a sharp visual drop —** this is expected and is not a real decline. The data only covers January through May 2025 (5 months), producing an artificially low yearly total.
- **GWB PIP —** data ends in 2022 with NaN from 2023 onwards. The facility either closed or stopped reporting — treated as a special case in forecasting.

### 3.4.3 Year-over-Year Growth Rates

Year-over-year percentage growth rates were calculated for all 8 facilities for the period 2016-2024.

```
Year-over-Year Growth Rate (%) per Facility:
fac_b  Bayonne  GWB Lower  GWB PIP  GWB Upper  Goethals  Holland  Lincoln  Outerbridge
year
2015       NaN        NaN      NaN        NaN       NaN      NaN      NaN          NaN
2016    -18.29       0.56     1.47       4.22      6.74    -0.28     1.48         5.56
2017     40.48       1.23    -3.78       0.48      1.03    -3.21    -0.89         2.34
2018     20.85       0.87    -7.56       0.59      4.83     1.34    -0.42        -1.91
2019     17.52       0.80     1.97       0.66      6.56     3.74    -2.25        -1.38
2020    -12.71     -19.38   -19.84     -15.69    -18.09   -26.83   -26.18       -16.65
2021     24.02      20.24    12.73      11.71     18.00    22.57    30.84        12.61
2022      8.04       1.53   -48.02      14.54      4.37     4.90     9.98         3.32
2023      2.30       3.16     0.00      12.21      1.07     6.69    -2.20         2.78
2024      8.53       7.89     0.00      -1.46      4.32     4.09     2.06         2.55
```

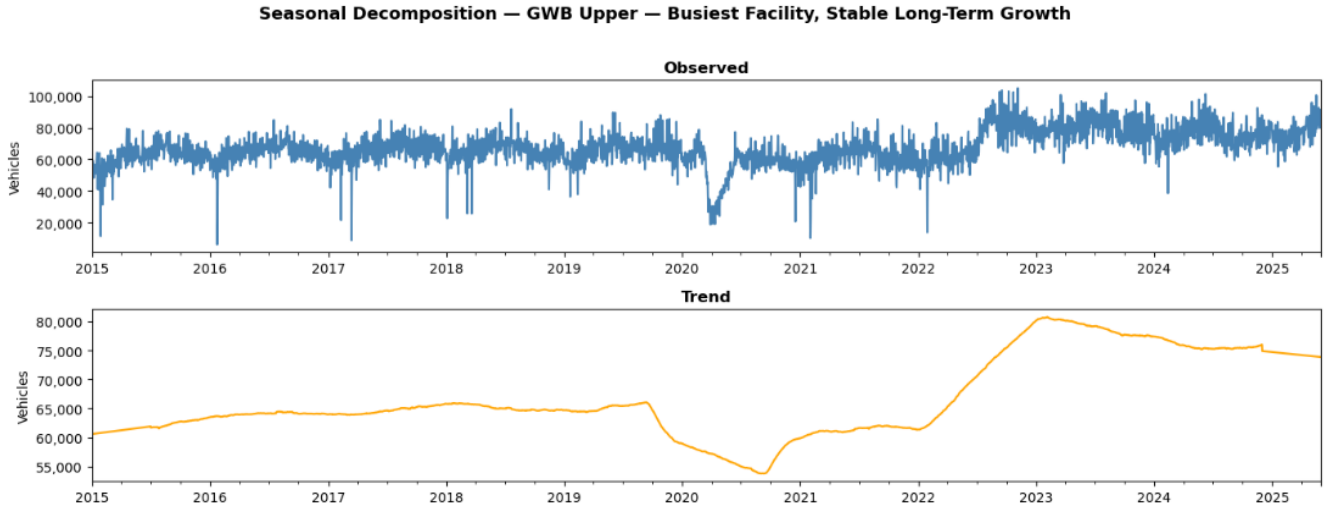


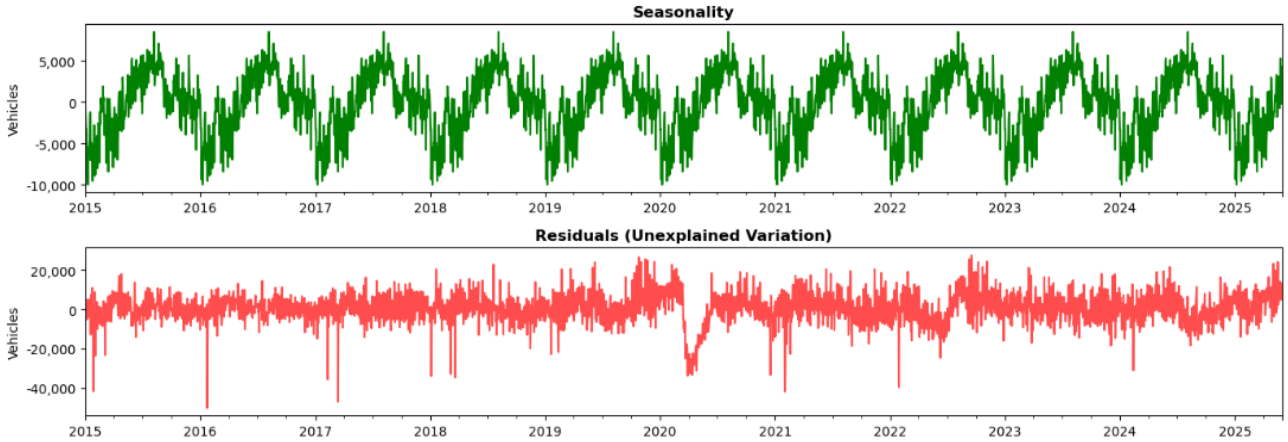

| Finding | Value | Detail |
|---|---|---|
| 2020 COVID crash — worst hit | Holland: -26.83% | Lincoln -26.18%, GWB Lower -19.38% — all facilities hit hard |
| 2021 recovery — fastest bounceback | Lincoln: +30.84% | Holland +22.57%, GWB Lower +20.24% |
| Bayonne 2017 outlier spike | +40.48% | Unique event — no other facility shows anything comparable |
| GWB PIP 2022 collapse | -48.02% | Catastrophic drop explaining why data disappears from 2023 |
| 2022-2024 stabilization | 2-8% per year | Most facilities settling at normal growth rates |
| GWB Upper 2024 | -1.46% | Possible early congestion pricing effect |

### 3.4.4 Seasonal Decomposition

Seasonal decomposition was performed for 3 representative facilities chosen to cover the full range of behavioral patterns across the network: GWB Upper (busiest, most complete data), Bayonne (unique outlier growth), and Holland Tunnel (steepest COVID impact). The remaining 5 facilities were not decomposed individually to avoid repetition — they exhibit similar seasonal structures.

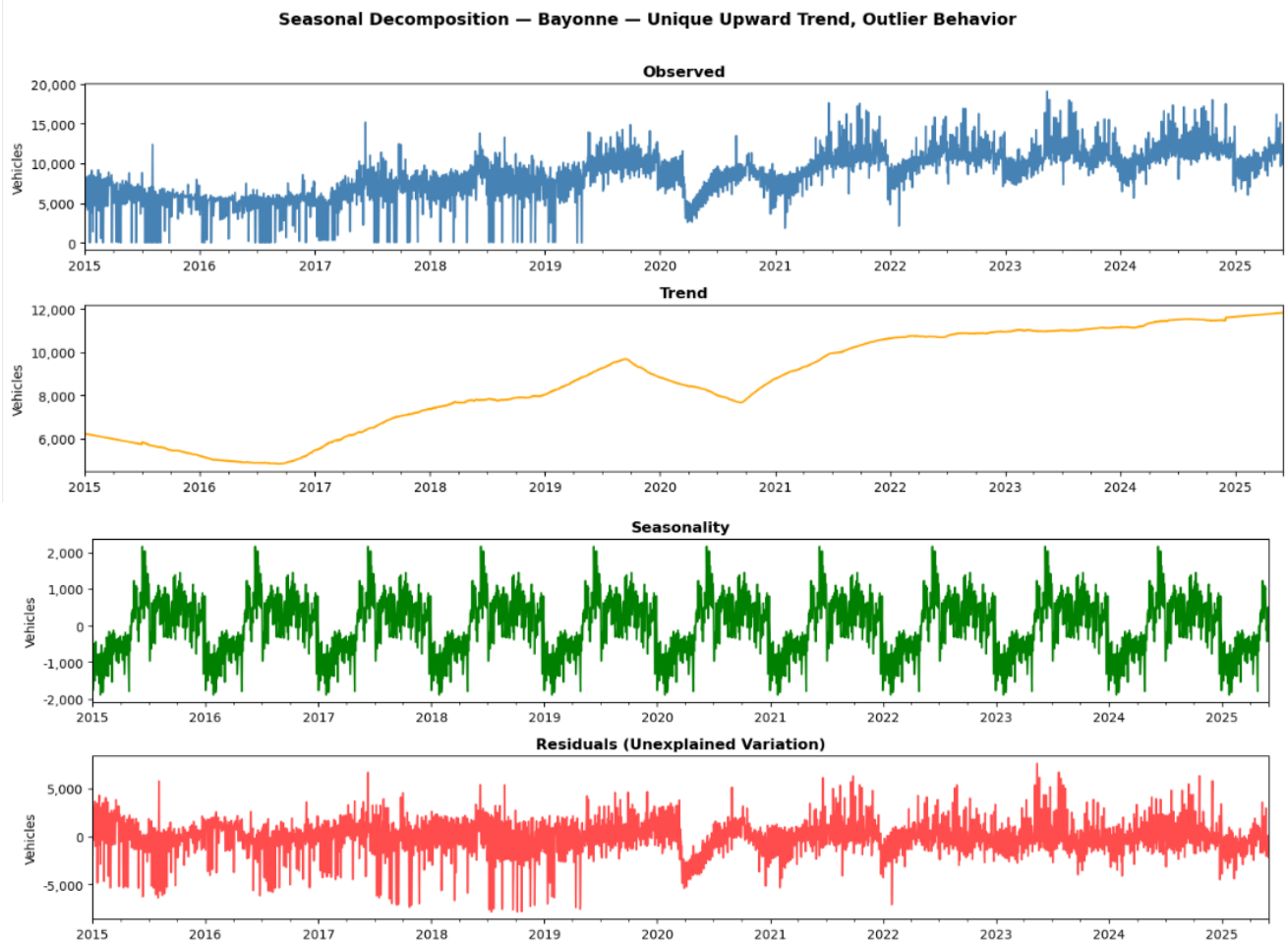
**GWB Upper — Busiest Facility, Stable Long-Term Growth**






- **Trend:** Steady growth from ~60,000 vehicles/day in 2015 to a peak of ~80,000 in 2023, followed by a slight decline to ~75,000 in 2025. COVID-19 dip clearly visible as a sharp drop to ~55,000 in 2020.
- **Seasonality:** Clear annual cycle with amplitude of ±5,000-10,000 vehicles. Summer peaks and winter troughs are consistent across all years.
- **Residuals:** Well-behaved with the exception of a large positive spike in early 2020 (COVID shock), confirming the model captures normal variation well.

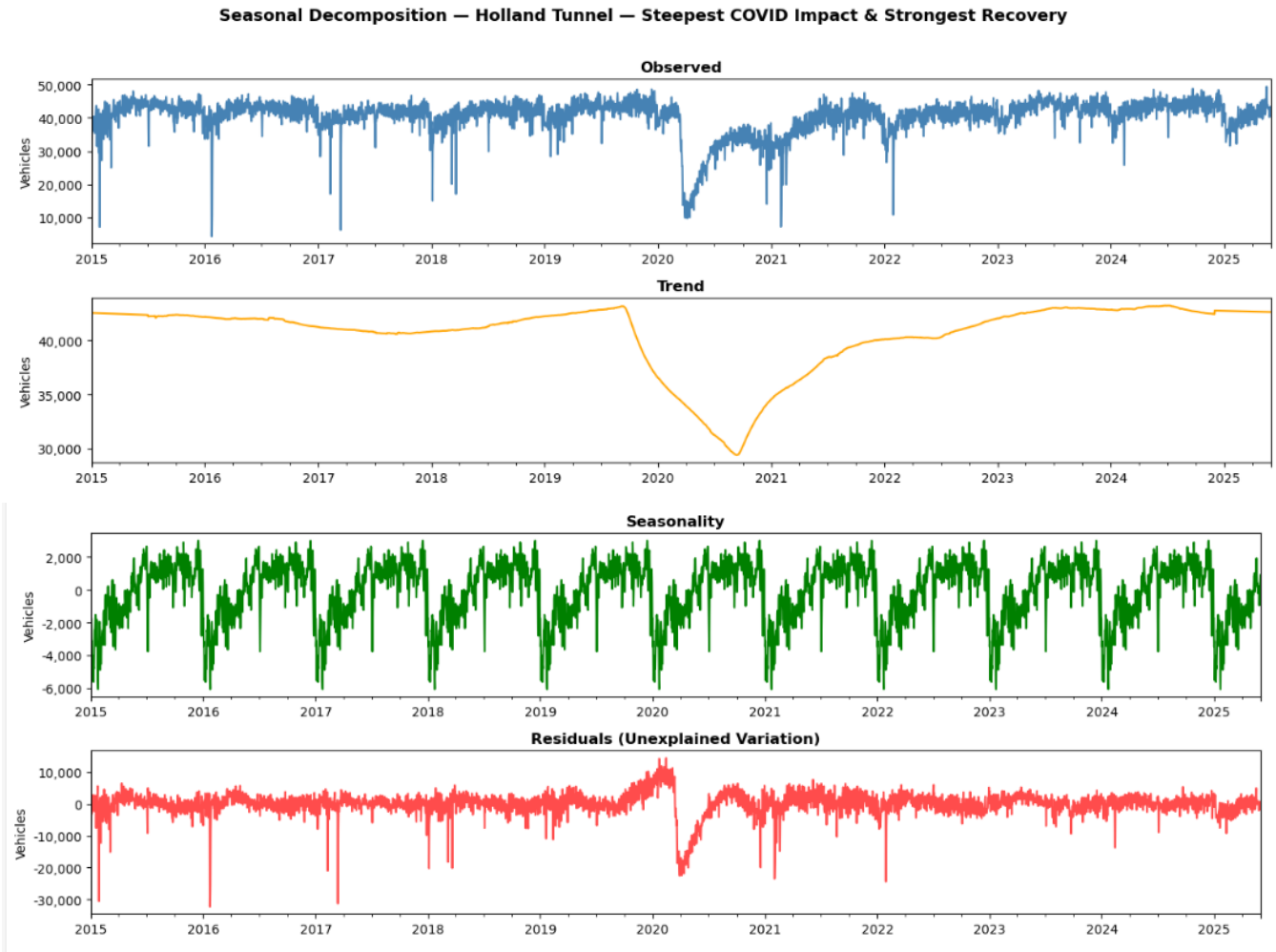
### Bayonne — Unique Upward Trend, Outlier Behavior




- **Trend:** Continuous upward climb from ~6,000 vehicles/day in 2015 to ~12,000 in 2025 — the only facility in the network that sustained this level of growth without plateauing.

- **Seasonality:** Tight amplitude of approximately ±1,500 vehicles — the most predictable and stable seasonal pattern in the network.
- **Residuals:** Small and stable throughout — the cleanest signal in the network, suggesting that Bayonne traffic is highly regular and well-captured by trend and seasonality alone.

**Holland Tunnel — Steepest COVID Impact & Strongest Recovery**




- **Trend:** Flat at approximately 42,000 vehicles/day pre-COVID, catastrophic drop to ~30,000 in 2020, followed by a full V-shaped recovery back to ~42,000 by 2022.
- **Seasonality:** Annual amplitude of approximately ±2,000 vehicles, consistent across all years.
- **Residuals:** COVID-19 shock is the dominant feature — a large negative spike in early 2020 that dwarfs all other unexplained variation, confirming the structural break caused by the pandemic.

### 3.4.5 Stationarity Tests — ADF & KPSS

Both the Augmented Dickey-Fuller (ADF) test and the Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test were applied to the yearly aggregated data for all 8 facilities. Stationarity tests are included to satisfy the course requirement as outlined in the professor's materials.

```
======================================================================
STATIONARITY TESTS — ADF & KPSS (Yearly Data per Facility)
======================================================================

Facility : Bayonne
  ADF  — Stat: -1.6652 | p-value: 0.4491 | Non-Stationary
  KPSS — Stat: 0.3191 | p-value: 0.1000 | Stationary
  Overall: INCONCLUSIVE

Facility : GWB Lower
  ADF  — Stat: -3.0663 | p-value: 0.0291 | Stationary
  KPSS — Stat: 0.2201 | p-value: 0.1000 | Stationary
  Overall: STATIONARY

Facility : GWB PIP
  ADF  — Stat: 5.7707 | p-value: 1.0000 | Non-Stationary
  KPSS — Stat: 0.4801 | p-value: 0.0461 | Non-Stationary
  Overall: NON-STATIONARY

Facility : GWB Upper
  ADF  — Stat: -9.1079 | p-value: 0.0000 | Stationary
  KPSS — Stat: 0.0769 | p-value: 0.1000 | Stationary
  Overall: STATIONARY

Facility : Goethals
  ADF  — Stat: -2.4243 | p-value: 0.1350 | Non-Stationary
  KPSS — Stat: 0.1287 | p-value: 0.1000 | Stationary
  Overall: INCONCLUSIVE

Facility : Holland
  ADF  — Stat: -2.2005 | p-value: 0.2061 | Non-Stationary
  KPSS — Stat: 0.2604 | p-value: 0.1000 | Stationary
  Overall: INCONCLUSIVE


Facility : Lincoln
  ADF  — Stat: -1.6671 | p-value: 0.4482 | Non-Stationary
  KPSS — Stat: 0.2315 | p-value: 0.1000 | Stationary
  Overall: INCONCLUSIVE

Facility : Outerbridge
  ADF  — Stat: -1.3972 | p-value: 0.5836 | Non-Stationary
  KPSS — Stat: 0.2958 | p-value: 0.1000 | Stationary
  Overall: INCONCLUSIVE

======================================================================
   Facility     ADF Result    KPSS Result        Overall
    Bayonne Non-Stationary     Stationary   INCONCLUSIVE
  GWB Lower     Stationary     Stationary     STATIONARY
    GWB PIP Non-Stationary Non-Stationary NON-STATIONARY
  GWB Upper     Stationary     Stationary     STATIONARY
   Goethals Non-Stationary     Stationary   INCONCLUSIVE
    Holland Non-Stationary     Stationary   INCONCLUSIVE
    Lincoln Non-Stationary     Stationary   INCONCLUSIVE
Outerbridge Non-Stationary     Stationary   INCONCLUSIVE
```

| Facility | ADF Stat | ADF p-value | ADF Result | KPSS Result | Overall |
|---|---|---|---|---|---|
| Bayonne | -1.6652 | 0.4491 | Non-Stationary | Stationary | INCONCLUSIVE |
| **GWB Lower** | **-3.0663** | **0.0291** | **Stationary** | **Stationary** | **STATIONARY** |
| GWB PIP | 5.7707 | 1.0000 | Non-Stationary | Non-Stationary | NON-STATIONARY |
| **GWB Upper** | **-9.1079** | **0.0000** | **Stationary** | **Stationary** | **STATIONARY** |
| Goethals | -2.4243 | 0.1350 | Non-Stationary | Stationary | INCONCLUSIVE |
| Holland | -2.2005 | 0.2061 | Non-Stationary | Stationary | INCONCLUSIVE |
| Lincoln | -1.6671 | 0.4482 | Non-Stationary | Stationary | INCONCLUSIVE |
| Outerbridge | -1.3972 | 0.5836 | Non-Stationary | Stationary | INCONCLUSIVE |

Two facilities (GWB Lower and GWB Upper) are confirmed stationary. GWB PIP is confirmed non-stationary, consistent with its structural decline and data disappearance. The remaining five facilities return inconclusive results — a common outcome when stationarity tests are applied to small samples (only 10-11 yearly observations per facility). AutoTS uses the full daily dataset internally (~3,000+ observations per facility), which substantially improves stationarity detection reliability.

**Important note:** AutoTS handles non-stationarity automatically through its internal transformation pipeline, including differencing and detrending. No manual intervention is required regardless of these test results. The tests are reported here to satisfy the course requirement and document the statistical properties of the series.

## 3.5 AutoTS Model Configuration

The AutoTS model was trained on daily data from January 1, 2015 through December 31, 2024 — a total of 28,360 rows across 8 facilities. The validation method used is backwards rolling validation, which automatically withholds the last periods from the training data for validation without any manual splitting.

| Parameter | Value | Explanation |
|---|---|---|
| Training data | 2015-01-01 to 2024-12-31 | 28,360 daily rows across 8 facilities |
| forecast_length | 365 days | One-year forecast window used during training; extended to 2032 at predict() step |
| frequency | infer (Daily) | AutoTS automatically detects daily frequency |
| prediction_interval | 0.95 | 95% confidence interval on all forecasts |
| model_list | fast | Broader model search across multiple algorithm families |
| max_generations | 10 | Genetic algorithm generations for model selection |
| num_validations | 2 | Two rolling backwards validation windows |

| | | |
|---|---|---|
| validation_method | backwards | Temporal order always preserved — no future data leakage |
| no_negatives | True | Ensures all forecasted vehicle counts are non-negative |
| n_jobs | -1 | Uses all available CPU cores for parallel training |
| ensemble | ['simple', 'horizontal-min'] | Enables ensemble model construction across series |

### 3.5.1 Train / Validation Split

With validation_method='backwards' and num_validations=2, AutoTS automatically withholds the last 2 × 365 days from the training data as internal validation windows — no manual splitting is required:

- **Effective training data:** 2015-01-01 to approximately mid-2022 (used for actual model fitting)
- **Validation window 1:** ~2023 (withheld internally — first backwards validation period)
- **Validation window 2:** ~2024 (withheld internally — second backwards validation period)

This approach strictly preserves temporal order at all times — the model always trains on past data and validates on future data, preventing any form of temporal data leakage. The backwards rolling validation method is consistent with the professor's course materials on rolling forecast origin evaluation.

```
Training set : 2015-01-01 to 2024-12-31
Training rows: 28360
Using 3 cpus for n_jobs.
Data frequency is: D, used frequency is: D
One or more series is 90% or more NaN in this test split
```

## 3.6 Best Model — Horizontal Ensemble

After evaluating models across multiple algorithm families over 10 generations, AutoTS selected a Horizontal Ensemble of 5 models as the best performing configuration.

```
9 - Ensemble with avg smape 8.08 in 12.74s:
10 - Ensemble with avg smape 15.57 in 12.77s:
11 - Ensemble with avg smape 8.14 in 12.82s:
12 - Ensemble with avg smape 13.0 in 12.84s:
13 - Ensemble with avg smape 7.99 in 12.86s:
14 - Ensemble with avg smape 12.06 in 12.94s:
```

```
Model training complete.
Initiated AutoTS object with best model:
Ensemble
{}
{'model_name': 'Horizontal', 'model_count': 5, 'model_metric': 'Score-min', 'mode
l': 'BasicLinearModel', 'ModelParameters': '{"datepart_method": ["lunar_phase"],
end": 360, "regression_type": null, "lambda_": null, "trend_phi": null, "holiday_
s': '{"fillna": "ffill", "transformations": {"0": "Detrend", "1": "DatepartRegres
"Linear", "phi": 1, "window": null, "transform_dict": null}, "1": {"regression_mc
estimators": 100, "min_samples_leaf": 1, "min_samples_split": 2, "max_depth": 10,
```

| Attribute | Value |
|---|---|
| Model type | Horizontal Ensemble |
| Number of base models | 5 |
| Base model families | BasicLinearModel, DatepartRegression, Linear |
| Ensemble strategy | Horizontal — each series (facility) assigned to its best-performing base model |
| Model metric | Score-min (minimizes SMAPE across validation windows) |
| Prediction interval | 95% confidence interval on all forecasts |

**What is a Horizontal Ensemble?** A Horizontal Ensemble assigns each individual time series (facility) to the single best-performing model for that series, rather than averaging all models together. This means GWB Upper might be forecast by a different algorithm than Bayonne — whichever performs best per facility on the validation data. This is particularly well-suited to this dataset because the 8 facilities exhibit fundamentally different behaviors (volume scale, trend direction, COVID impact) and a single model family cannot optimally capture all of them simultaneously.

## 3.7 Model Performance

Performance was evaluated across three validation windows using SMAPE (Symmetric Mean Absolute Percentage Error) and MAE (Mean Absolute Error). SMAPE is the primary metric used by AutoTS for model selection as it is symmetric, scale-independent, and appropriate for multi-series forecasting across facilities with very different volume levels.

```
Validation: 0, 1, 2
SMAPE: 5.227050545641682, 6.6407293230810085, 7.990399232511722
MAE: 2446.0824212020234, 3277.44694504149, 3518.3223334462878
SPL: 0.25232388066535727, 0.2884712917730302, 0.4181343587734968
```

| Validation Window | SMAPE | MAE (vehicles/day) | SPL | Assessment |
|---|---|---|---|---|
| Window 0 (most recent) | **5.23%** | 2,446 | 0.2523 | Excellent |
| Window 1 | **6.64%** | 3,277 | 0.2884 | Excellent |
| Window 2 (oldest) | **7.99%** | 3,518 | 0.4181 | Very Good |

- **SMAPE range of 5.23%-7.99% is excellent** for a multi-facility daily time series forecast. Industry benchmarks consider SMAPE below 10% to be strong performance for transportation demand forecasting.
- **Window 0 (most recent validation period) achieves the lowest error at 5.23% —** this is the most important window as it represents the period most similar to the actual forecast horizon (2025+). The model performs best on the most recent data.
- **Error increases slightly for older windows** — this is expected and normal behavior. Older validation windows are further from the forecast period and may contain structural patterns (e.g., pre-Bayonne-expansion behavior) that are less relevant to the future.
- **MAE of 2,446-3,518 vehicles/day** — given that GWB Upper averages approximately 77,000 vehicles/day, this represents a relative daily error of approximately 3-5% at the network's busiest facility. For smaller facilities like Bayonne (~12,000/day), the percentage error is higher but acceptable for long-range planning.

## 3.8 Forecast Results (2025-2032)

### 3.8.1 Forecast Generation

The trained model was used to generate a forecast for 2,922 days (8 years: 2025 through 2032). The predict() call was extended beyond the training forecast_length of 365 days by passing forecast_length=2922 directly to model.predict(), avoiding the need to retrain the model.

```
Forecast shape: (2922, 8)
Forecast date range: 2025-01-01 00:00:00 to 2032-12-31 00:00:00
                 Bayonne     GWB Lower       GWB PIP     GWB Upper  \
2025-01-01   9672.948923  54403.086308  16581.831859  70271.267583
2025-01-02  10760.915425  58045.221017  16959.714078  70256.952298
2025-01-03  10888.842234  57891.411571  16633.629259  70959.466750
2025-01-04   8681.131289  52041.819265  16276.162845  73600.279070
2025-01-05   8184.407857  49657.630931  16406.072458  68736.044304

                Goethals       Holland       Lincoln   Outerbridge
2025-01-01  45030.783601  39329.462883  43503.985115  35926.290382
2025-01-02  47873.543428  40419.462645  48895.713371  38016.655314
2025-01-03  47745.803305  40070.749955  48037.705605  37943.959720
2025-01-04  45420.851225  38840.155511  44121.368960  34732.784403
2025-01-05  42367.616939  37532.155630  39657.859213  33075.337020
```

### 3.8.2 Yearly Forecast Table

Daily forecasts were aggregated to yearly totals per facility. The table below shows the forecasted total vehicle counts by facility for each year from 2025 through 2032.

| Year | Bayonne | GWB Lower | GWB PIP | GWB Upper | Goethals | Holland | Lincoln | Outerbridge |
|---|---|---|---|---|---|---|---|---|
| **2025** | 4,327,797 | 22,821,068 | 6,131,241 | 27,173,130 | 18,555,476 | 15,800,515 | 18,761,549 | 14,992,528 |
| **2026** | 4,423,226 | 22,696,815 | 5,965,710 | 26,544,267 | 18,711,130 | 15,587,978 | 18,294,321 | 14,917,346 |
| **2027** | 4,560,216 | 23,081,701 | 5,896,583 | 26,514,856 | 19,283,323 | 15,804,074 | 18,401,481 | 15,047,190 |
| **2028** | 4,694,729 | 23,286,900 | 5,815,909 | 26,501,455 | 19,750,515 | 15,938,717 | 18,399,552 | 15,145,613 |
| **2029** | 4,804,878 | 23,479,063 | 5,699,515 | 26,169,460 | 20,098,882 | 15,959,009 | 18,229,841 | 15,148,425 |
| **2030** | 4,928,795 | 23,662,062 | 5,593,776 | 25,923,536 | 20,515,805 | 16,014,640 | 18,119,152 | 15,203,854 |
| **2031** | 5,035,123 | 23,777,497 | 5,481,770 | 25,635,118 | 20,884,303 | 16,018,855 | 17,941,676 | 15,228,447 |
| **2032** | 5,165,141 | 23,940,452 | 5,368,226 | 25,291,056 | 21,268,557 | 16,033,665 | 17,777,108 | 15,275,683 |

### 3.8.3 Key Forecast Findings

- **Bayonne continues its unique upward growth —** the only facility forecast to grow consistently every year, from 4.3M (2025) to 5.2M (2032), reflecting its sustained structural demand expansion.
- **GWB Upper shows a gradual decline —** from 27.2M (2025) to 25.3M (2032). This is consistent with the -1.46% growth rate observed in 2024, potentially reflecting early congestion pricing effects redirecting traffic to other crossings.
- **GWB Lower and Goethals grow steadily —** GWB Lower from 22.8M to 23.9M, Goethals from 18.6M to 21.3M — both showing healthy 2-4% annual growth consistent with historical patterns.
- **Holland and Lincoln stabilize —** both facilities forecast at approximately flat-to-modest growth, consistent with mature tunnel corridors with limited capacity expansion.
- **GWB PIP shows continued decline —** from 6.1M (2025) to 5.4M (2032). This forecast is based entirely on 2015-2022 data and carries higher uncertainty. The model extrapolates the declining trend observed before data disappeared.
- **Outerbridge remains stable —** approximately 15M vehicles annually throughout the forecast period.

### 3.8.4 Forecast Reliability Disclaimer

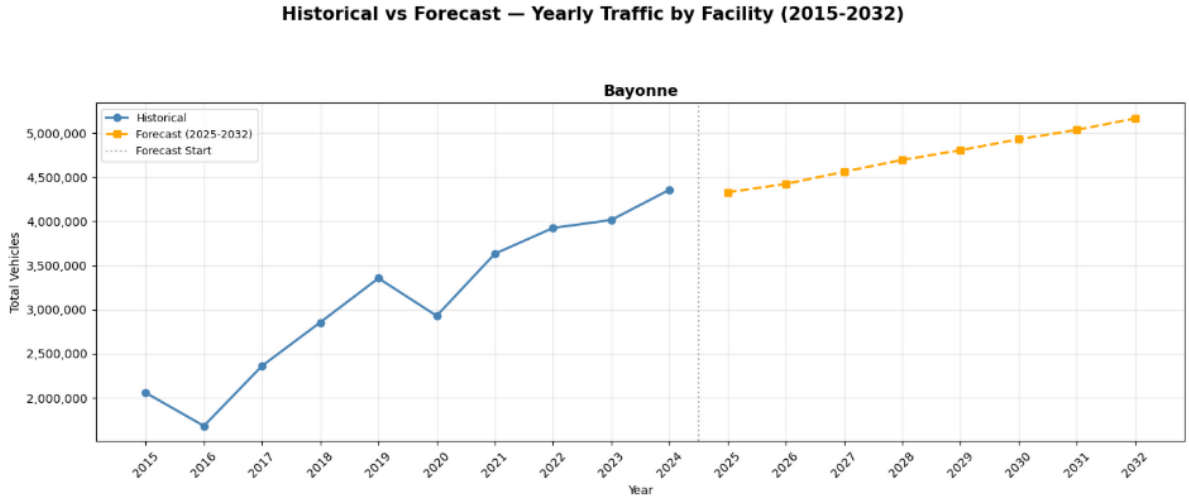
Forecast reliability decreases with time horizon. The following classification applies:

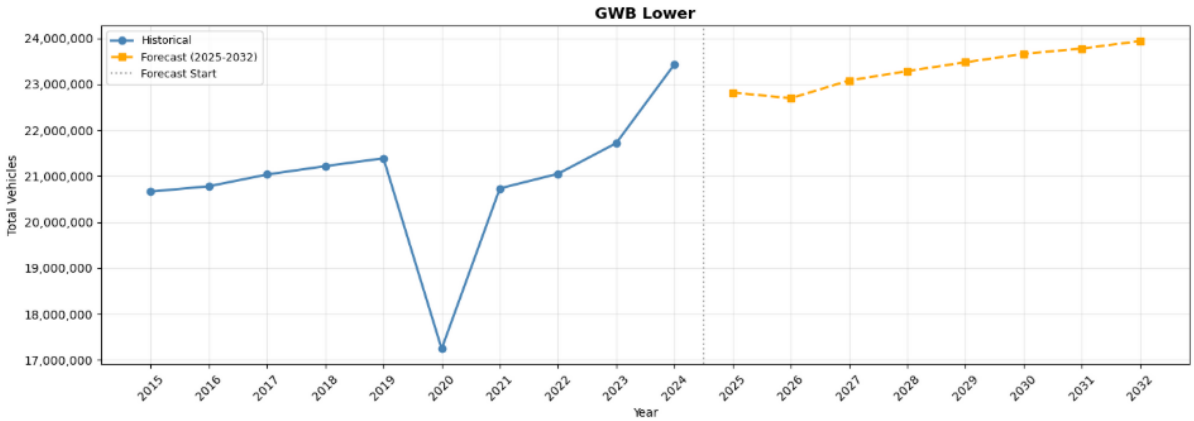
- **2025-2027 — Reliable:** within the model's validated SMAPE range of 5-8%. These years are the primary planning horizon for operational decisions.
- **2028-2030 — Directional:** trend direction is reliable but magnitude carries increasing uncertainty. Suitable for capital planning and infrastructure investment prioritization.
- **2031-2032 — Indicative only:** 8-year forecasts are subject to structural changes not captured in historical data (policy changes, economic shifts, new infrastructure). Use for long-range scenario planning only.

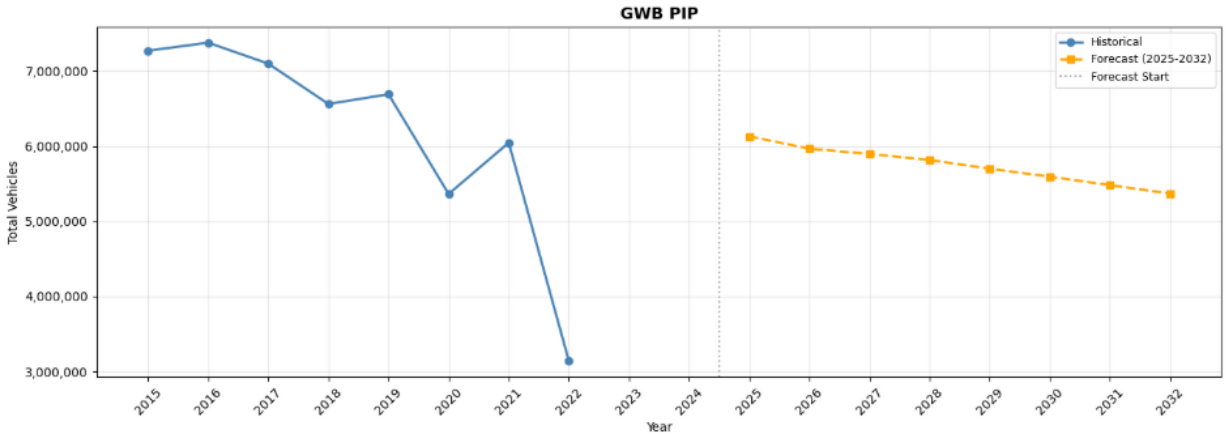
## 3.9 Forecast Visualizations

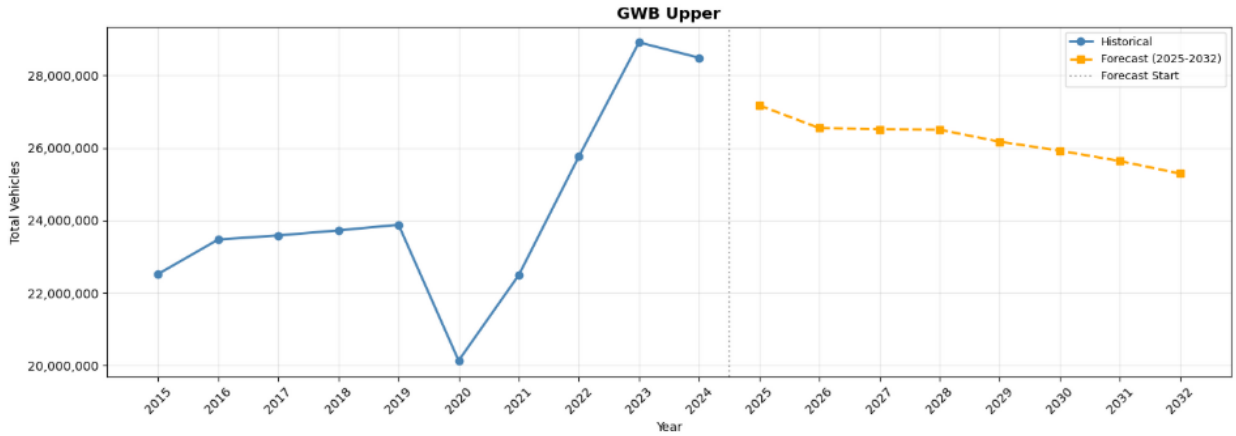
### 3.9.1 Per-Facility Historical vs Forecast

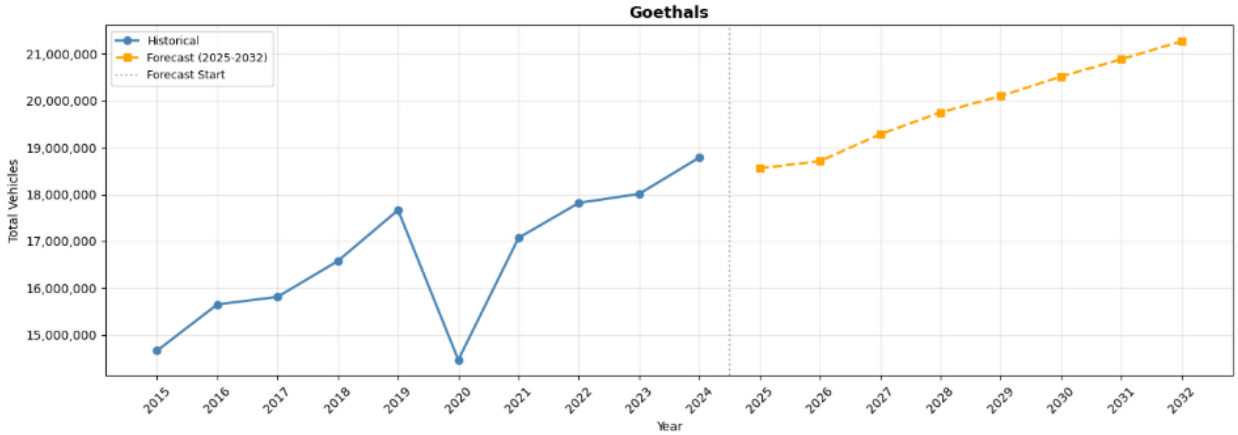
The chart below shows historical yearly traffic (2015-2024) and the 2025-2032 forecast for each of the 8 facilities individually. The dotted vertical line marks the forecast start point (end of 2024).

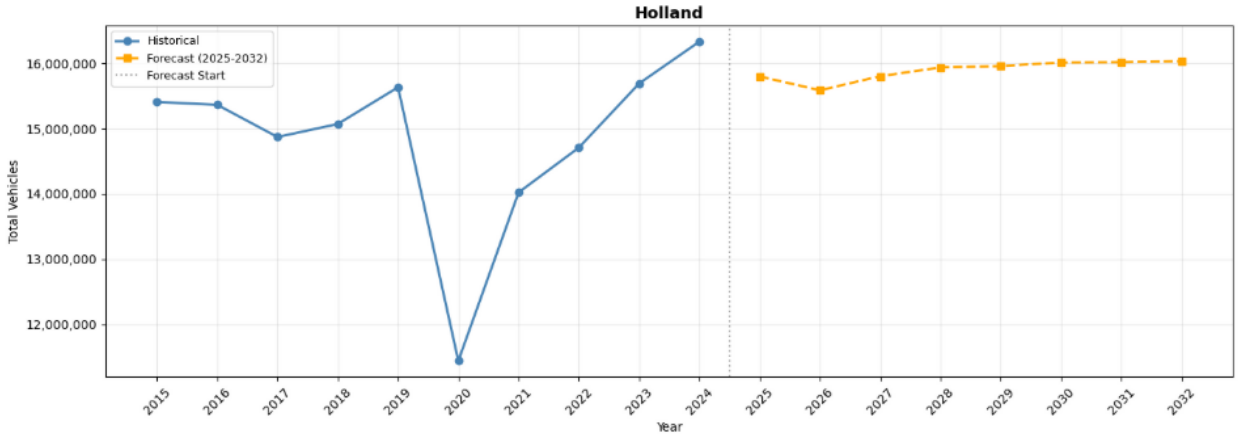


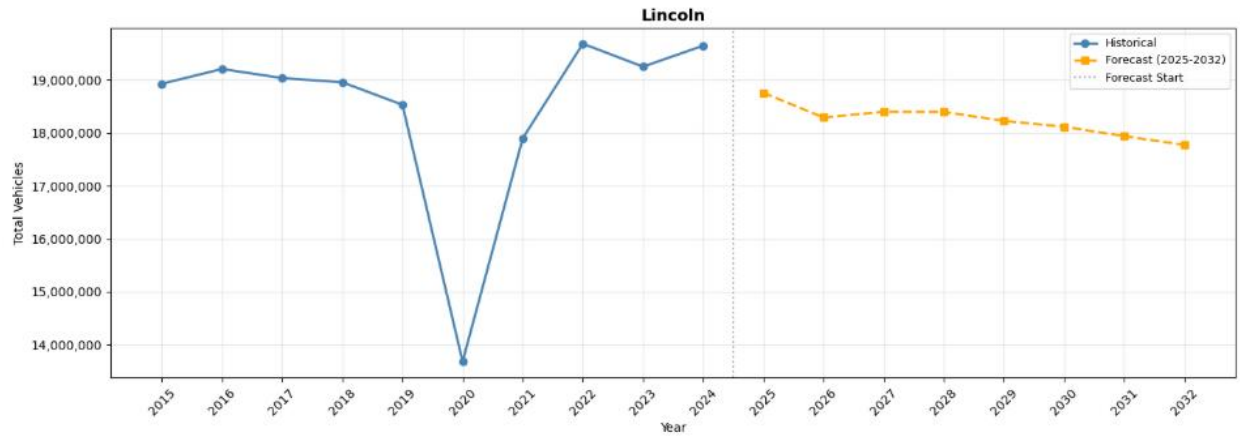

GWB Lower
Historical
Forecast (2025-2032)
Forecast Start
Total Vehicles
Year

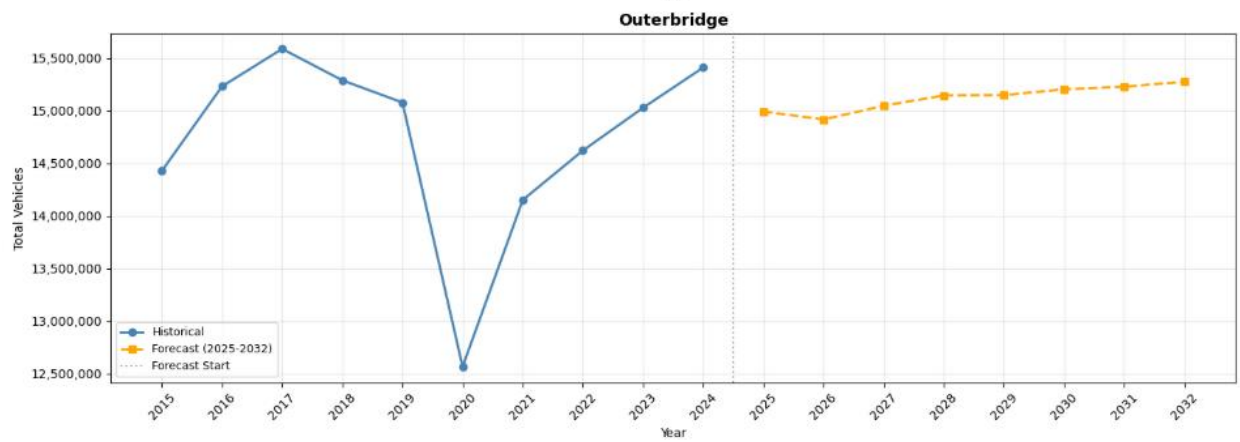

GWB PIP
Historical
Forecast (2025-2032)
Forecast Start
Total Vehicles
Year


GWB Upper
Historical
Forecast (2025-2032)
Forecast Start
Total Vehicles
Year


Goethals
Historical
Forecast (2025-2032)
Forecast Start
Total Vehicles
Year



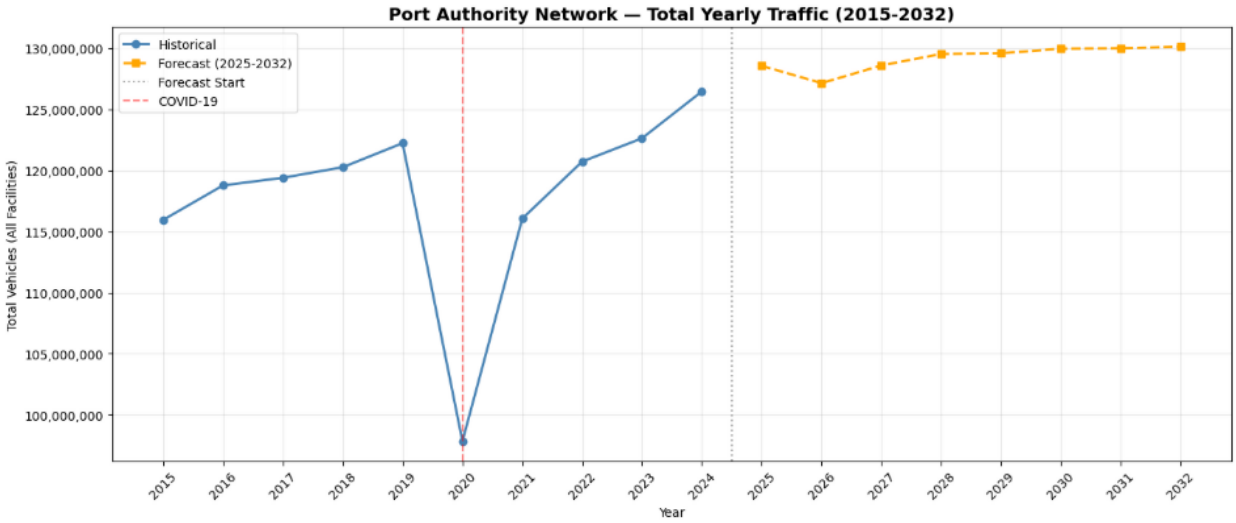




### 3.9.2 Network-Wide Forecast

The chart below aggregates all 8 facilities into a single network-wide total, showing total Port Authority traffic from 2015 through 2032.

```
Network-Wide Yearly Total:
        Historical     Forecast
year
2015   115,946,767          N/A
2016   118,771,346          N/A
2017   119,394,386          N/A
2018   120,257,896          N/A
2019   122,227,457          N/A
2020    97,829,569          N/A
2021   116,053,189          N/A
2022   120,719,880          N/A
2023   122,627,177          N/A
2024   126,460,781          N/A
2025           N/A  128,563,304
2026           N/A  127,140,794
2027           N/A  128,589,425
2028           N/A  129,533,391
2029           N/A  129,589,073
2030           N/A  129,961,620
2031           N/A  130,002,789
2032           N/A  130,119,888
```




| Year | Historical (All Facilities) | Forecast (All Facilities) |
|---|---|---|
| 2015 | 115,946,767 | — |
| 2016 | 118,771,346 | — |
| 2017 | 119,394,386 | — |
| 2018 | 120,257,896 | — |
| 2019 | 122,227,457 | — |
| **2020** | **97,829,569** | — |
| 2021 | 116,053,189 | — |
| 2022 | 120,719,880 | — |
| 2023 | 122,627,177 | — |
| 2024 | 126,460,781 | — |
| **2025** | — | **128,563,304** |
| 2026 | — | 127,140,794 |
| 2027 | — | 128,589,425 |
| 2028 | — | 129,533,391 |
| 2029 | — | 129,589,073 |
| 2030 | — | 129,961,620 |
| 2031 | — | 130,002,789 |
| 2032 | — | 130,119,888 |

## 3.10 Key Insights & Business Value

The forecasting results provide several actionable insights for Port Authority operational and strategic planning:

- **Network traffic is forecast to stabilize at approximately 128-130 million vehicles annually —** a modest 1-3% increase above the 2024 level of 126.5M. This indicates the network is approaching a saturation point in aggregate demand, with growth shifting from volume expansion to redistribution between facilities.

- **Bayonne Bridge is the only facility with consistent long-term growth —** its trajectory from 4.3M to 5.2M vehicles by 2032 suggests continued demand expansion, potentially driven by the 2017 span raise that increased truck capacity. Port Authority should prioritize capacity planning and toll infrastructure investment at Bayonne ahead of other facilities.
- **GWB Upper's gradual decline warrants monitoring —** the forecast decline from 27.2M to 25.3M vehicles (2025-2032) may reflect early congestion pricing effects. If congestion pricing continues to suppress GWB Upper demand, maintenance scheduling and staffing at that facility can be adjusted accordingly.
- **GWB PIP requires special attention —** the forecast is based on 2015-2022 data only and carries the highest uncertainty of any facility. Port Authority should clarify the operational status of GWB PIP and use these forecasts only for contingency planning until current data is available.
- **Goethals Bridge shows the strongest growth among mid-sized facilities —** growing from 18.6M to 21.3M vehicles by 2032 (+14.5%). This facility may require infrastructure investment and capacity review within the forecast window.
- **COVID-19 impact is fully reflected in the model —** the 2020 crash to 97.8M network-wide vehicles and the subsequent V-shaped recovery to 126.5M by 2024 are captured in the training data. The model has learned from this structural shock, making its forecasts more robust to future demand disruptions.

## 3.11 Model Export

The best model template was exported to a CSV file for reproducibility and future deployment. This file contains the full model configuration, transformation parameters, and validation metrics required to reload and rerun the exact same model without retraining.

| | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | ID | Model | ModelPar | Transform | Ensemble | Runs | smape | mae | rmse | made | mage | custom | underestir | mle | overestim | imle | spl | containm |
| 2 | ab07 | Ensemble | {"model_n | {} | 2 | 3 | 6.619393 | 3080.617 | 4062.194 | 0.86533 | 12725.4 | 0 | -1910442 | 1902.241 | 1175122 | 1185.7 | 0.319643 | 0.76016 |
| 3 | d007 | Ensemble | {"model_n | {} | 1 | 3 | 6.35462 | 2947.544 | 3896.416 | 0.848503 | 11985.66 | 0 | -1773314 | 1758.771 | 1182148 | 1196.053 | 0.321426 | 0.723402 |
| 4 | 3245 | BasicLine | {"datepart | {"fillna": "l | 0 | 3 | 6.587113 | 3056.513 | 4037.479 | 0.866182 | 13428.62 | 0 | -1974442 | 1956.856 | 1089631 | 1106.964 | 0.3262 | 0.735046 |
| 5 | 81a | Seasonal | {"method" | {"fillna": "l | 0 | 3 | 7.04975 | 3253.918 | 4234.194 | 0.874997 | 15517.4 | 0 | -2303316 | 2270.16 | 967852.6 | 991.1482 | 0.348211 | 0.731735 |
| 6 | a8d | Seasonali | {"window" | {"fillna": "r | 0 | 3 | 7.298751 | 3121.531 | 4109.371 | 0.882921 | 14284.43 | 0 | -2132136 | 2115.563 | 994814 | 1013.374 | 0.349068 | 0.791553 |
| 7 | 4de | DMD | {"rank": 2, | {"fillna": "l | 0 | 3 | 7.13302 | 3272.028 | 4274.264 | 0.890105 | 15860.54 | 0 | -2311739 | 2278.723 | 976428.5 | 1000.706 | 0.352475 | 0.731621 |
| 8 | 9402 | GLS | {"changep | {"fillna": "l | 0 | 3 | 7.187731 | 3308.428 | 4281.279 | 0.880657 | 16313.66 | 0 | -2401195 | 2372.435 | 921825.7 | 943.415 | 0.354421 | 0.716895 |
| 9 | 1941 | RRVAR | {"method" | {"fillna": "f | 0 | 3 | 7.184216 | 3298.085 | 4278.063 | 0.882747 | 16083.78 | 0 | -2350605 | 2317.725 | 963833.5 | 987.7702 | 0.35569 | 0.716553 |
| 10 | b40 | ETS | {"damped | {"fillna": "f | 0 | 3 | 7.190537 | 3305.701 | 4296.849 | 0.887426 | 16355.8 | 0 | -2414827 | 2383.093 | 908389.3 | 930.0162 | 0.35531 | 0.728196 |
| 11 | 611e | GLM | {"family": ' | {"fillna": "l | 0 | 3 | 7.188187 | 3308.244 | 4307.555 | 0.893062 | 16297.96 | 0 | -2405796 | 2376.009 | 918676.6 | 939.6424 | 0.356737 | 0.724429 |
| 12 | 05a | VAR | {"regressi | {"fillna": "f | 0 | 3 | 7.191106 | 3305.124 | 4301.842 | 0.893773 | 16240.02 | 0 | -2403786 | 2372.541 | 918697.4 | 939.9865 | 0.356531 | 0.728196 |
| 13 | 3752 | SectionalF | {"window" | {"fillna": "r | 0 | 3 | 6.923336 | 3169.473 | 4133.856 | 0.885689 | 14300.3 | 0 | -2126650 | 2092.752 | 1058053 | 1084.104 | 0.407307 | 0.591895 |
| 14 | c121 | FFT | {"n_harmc | {"fillna": "r | 0 | 3 | 7.226636 | 3363.97 | 4349.464 | 0.894755 | 17113.96 | 0 | -2507261 | 2480.463 | 871677.6 | 890.9304 | 0.360775 | 0.719406 |
| 15 | afdb | BallTreeR | {"window" | {"fillna": "f | 0 | 3 | 7.156943 | 3278.948 | 4250.323 | 0.88686 | 15535.28 | 0 | -2284676 | 2251.546 | 1009902 | 1034.82 | 0.385028 | 0.669977 |
| 16 | f57a | AverageV | {"method" | {"fillna": "c | 0 | 3 | 6.799078 | 3113.295 | 4079.72 | 0.884175 | 14182.63 | 0 | -2040903 | 2006.362 | 1090474 | 1114.295 | 0.436336 | 0.534703 |
| 17 | 2449 | BallTreeM | {"window" | {"fillna": "f | 0 | 3 | 6.898125 | 3156.787 | 4163.203 | 0.893926 | 14557.78 | 0 | -2178172 | 2154.591 | 989495.4 | 1009.546 | 0.456928 | 0.527968 |
| 18 | 4a28 | Cassandr | {"preproc | {"fillna": "r | 0 | 3 | 7.881152 | 3434.694 | 4401.222 | 0.88724 | 19030.03 | 0 | -2727950 | 2704.065 | 726627.5 | 738.1554 | 0.353845 | 0.827055 |
| 19 | 3e0e | Univariate | {"window" | {"fillna": "f | 0 | 3 | 7.681118 | 3511.468 | 4486.731 | 0.893967 | 17566.13 | 0 | -2692310 | 2695.439 | 813991.9 | 823.5622 | 0.40793 | 0.650114 |
| 20 | 5215 | LastValue | {} | {"fillna": "r | 0 | 3 | 7.952766 | 3553.58 | 4678.232 | 0.970646 | 17599.77 | 0 | -1716977 | 1690.698 | 1846762 | 1870.44 | 0.381754 | 0.701484 |
| 21 | 342b | VECM | {"determir | {"fillna": "f | 0 | 3 | 8.367087 | 3570.191 | 4606.053 | 0.941559 | 16954.21 | 0 | -2086894 | 2044.565 | 1498422 | 1533.208 | 0.403236 | 0.769406 |
| 22 | 1844 | DatepartR | {"regressi | {"fillna": "f | 0 | 3 | 9.650275 | 4027.802 | 5011.468 | 0.972327 | 23555.17 | 0 | -3444645 | 3414.86 | 606345.6 | 620.6477 | 0.465338 | 0.618721 |
| 23 | 33d | ConstantN | {"constant | {"fillna": "p | 0 | 3 | 7.587284 | 3471.225 | 4455.514 | 0.917016 | 16982.1 | 0 | -2547569 | 2528.968 | 929468.1 | 949.7397 | 1.059282 | 0 |
| 24 | a4b | MetricMot | {"window" | {"fillna": "f | 0 | 3 | 11.58117 | 4749.996 | 5940.809 | 1.032792 | 28657.38 | 0 | -3807290 | 3772.844 | 963374.2 | 985.0957 | 0.589311 | 0.668721 |

# Section 4 — Algorithm Justification & Business Questions

Model selection was driven by the nature of each analytical problem, the structure of the dataset, and established industry practice. Each model directly addresses one or more of the Port Authority's core business questions. The sections below explain the technical fit, business rationale, and industry context for each model, followed by the specific business question each model answers with supporting numbers and operational recommendations.

## 4.1 Regression — Stacked Ensemble (Gradient Boosting)

### Technical Justification

**Why Gradient Boosting?**

- **Non-linear interactions:** Traffic volume is shaped by the combined effect of facility type, day of week, season, gas prices, and weather — not by any single factor in isolation. Gradient boosting builds these interactions automatically through sequential tree construction, without needing manual feature engineering.
- **Mixed feature types:** The dataset contains categorical variables (facility name), continuous variables (gas price, temperature), engineered ratios (EZPass ratio), and temporal features (month, day of year). GBM handles all of these natively.
- **Outlier robustness:** Extreme events like COVID-19 lockdowns, hurricanes, and gas price spikes create influential outliers in daily traffic records. GBM with subsampling and regularization is far less sensitive to these than linear models.

**Why Stacking?**

- **No single algorithm wins consistently:** Some base models capture seasonal patterns better; others handle economic shocks more accurately. The metalearner learns the optimal blend across 226 diverse GBM base models — producing a more stable predictor than any single model.

**What the Results Tell Us**

- **Facility identity (75.18%):** Structural differences between crossings — capacity, location, highway access — dominate all other signals. Network-level planning must account for this.
- **EZPass ratio (13.04%):** Electronic payment behavior is the second strongest driver, validating its importance for toll conversion planning.
- **Gas price (3.31%):** Economic sensitivity is real and quantifiable — higher fuel costs measurably reduce discretionary crossings.

### Business Justification & Industry Context

**Where GBM Regression Is Used in Industry**

Gradient boosting is the dominant algorithm for structured prediction problems across industries. It consistently tops benchmarks on platforms like Kaggle and is deployed at scale by Google, Amazon, Microsoft, and Uber on real-world data pipelines.

| Industry | Task | Why GBM |
|---|---|---|
| Financial Services | Credit scoring, loan default prediction | Non-linear risk interactions; interpretable for regulation |
| Retail & E-Commerce | Demand forecasting, inventory optimization | Seasonality, promotions, and product interactions in one model |
| Healthcare | Patient readmission and outcome prediction | Robust to missing values and heterogeneous clinical data |
| Energy | Electricity demand and price forecasting | Non-linear load patterns across weather, time, and industrial activity |
| Transportation & Tolling | Daily volume and revenue forecasting | Facility-level variation, weather, economic, and behavioral signals |

- **In tolling specifically:** Agencies such as E-ZPass Group, OOCEA (Florida), and SANDAG (California) use gradient-boosted ensembles for annual revenue certification — the same type of volume prediction this model delivers for the Port Authority.

## Business Question 1 — What are the top five factors that affect bridge and tunnel usage?

### The Question

The Port Authority wants to understand which operational, environmental, economic, and behavioral factors have the greatest measurable impact on daily vehicle counts across its eight bridge and tunnel facilities — by year, month, and week, and broken down by facility type.

### Business Answer & Supporting Numbers

The regression model was trained on 29,445 facility-day records spanning January 2015 to May 2025 across all eight crossings. H2O.ai AutoML trained 282 models and selected StackedEnsemble_AllModels_5 as the best, achieving a cross-validated RMSE of 2,963.67 and $R^2$ of 0.9758. The Python replica achieved RMSE of 2,718 and $R^2$ of 0.9791 on the held-out test set. The five most influential factors are:

| Rank | Factor | Importance | What It Means for the Port Authority |
|---|---|---|---|
| 1 | Facility identity (fac_b) | 75.18% | Which crossing a vehicle uses is the single largest driver of volume. GWB Upper averages over 28 million vehicles per year while Outerbridge and GWB PIP operate at a fraction of that — structural capacity and geographic positioning dominate everything else |
| 2 | EZPass ratio | 13.04% | The proportion of electronic payments at a facility is a strong behavioral signal. Higher EZPass adoption correlates with more stable and higher total volumes — making it a direct lever for revenue predictability |
| 3 | Gas price | 3.31% | Higher East Coast PADD 1 fuel prices produce a measurable drop in discretionary crossings. The model quantifies this sensitivity for inclusion in revenue scenario planning |
| 4 | Day of year / Seasonality | 2.12% | Summer months (June–August) consistently see peak volumes; January–February are the lowest. The seasonal cycle is predictable and consistent across all eight facilities |
| 5 | Year / Long-term trend | 2.12% | Volume has grown steadily from 2015 through 2024, with a COVID-19 dip in 2020 and strong recovery by 2022. The year |

| Rank | Factor | Importance | What It Means for the Port Authority |
|---|---|---|---|
| | | | variable also captures early congestion pricing effects visible in GWB Upper's -1.46% growth in 2024 |

**Models & Tools Used**

- **H2O.ai AutoML:** 282 models trained; StackedEnsemble_AllModels_5 selected (CV RMSE 2,963.67, $R^2$ 0.9758)
- **Python (scikit-learn):** 11-model GBM stacked ensemble replica (RMSE 2,718, $R^2$ 0.9791 on test set)
- **Excel:** ezpass_ratio feature engineered as ezpass / total_vehicles and appended to master dataset
- **Power BI:** Interactive dashboards by facility, year, month, and week visualizing volume patterns and factor breakdowns

**Recommendations**

- **Facility-level planning:** Since facility identity accounts for 75% of volume variance, pricing, staffing, and infrastructure decisions must be made at the individual crossing level — not as a network average.
- **EZPass conversion as a volume lever:** Targeted campaigns to increase EZPass adoption at lower-penetration facilities will improve both revenue predictability and daily volume stability. The model confirms this is the second most powerful operational signal available.
- **Gas price sensitivity monitoring:** Incorporate East Coast PADD 1 gas price forecasts into monthly revenue projections. A sustained price increase above $4.50/gallon should trigger a downside volume scenario review.
- **Seasonal staffing and operations:** Use the seasonal volume curve to pre-schedule toll plaza staffing, maintenance windows, and lane configurations — summer peaks require significantly more operational readiness than Q1.

## 4.2 Classification — Gradient Boosting (Bernoulli GBM)

### Technical Justification

**Why GBM for Classification?**

- **Class imbalance tolerance:** The dataset contains 68.92% normal days and 31.08% high-violation days. GBM with Bernoulli loss handles this imbalance natively — no resampling or data manipulation required.
- **Threshold flexibility:** GBM produces calibrated probability scores, not just binary outputs. Enforcement teams can set their own operating threshold based on staffing availability — catching more risk at the cost of more alerts, or staying conservative.
- **Interpretable output:** Feature importance scores translate directly to operational guidance — telling planners which facilities, vehicle categories, and time periods carry the highest violation risk.

**What the Results Tell Us**

- **AUC of 0.9984:** The model correctly ranks a high-violation day above a normal day 99.84% of the time.

- **96.79% recall:** Nearly all high-risk days are caught — enforcement demand is anticipated, not reacted to.
- **1.02% false alarm rate:** Only 62 out of 6,089 normal days are incorrectly flagged — unnecessary deployments remain negligible.
- **Year as top predictor (33.55%):** Violation behavior has shifted structurally over the decade — a trend invisible to short-window models.
- **EZPass ratio:** Confirms the documented inverse relationship: lower electronic payment adoption = higher violation risk. Directly actionable for outreach targeting.

## Business Justification & Industry Context

### Where GBM Classification Is Used in Industry

Binary classification with gradient boosting is one of the most widely deployed machine learning patterns across industries where the cost of a missed event differs from the cost of a false alarm.

| Industry | Task | Parallel to This Model |
|---|---|---|
| Financial Services | Fraud detection — flag high-risk transactions | Binary outcome, imbalanced classes, high cost of misses |
| Insurance | Claim fraud and risk classification | Behavioral signals predicting a binary anomaly outcome |
| Cybersecurity | Intrusion detection — normal vs. malicious traffic | High recall requirement; false positives carry operational cost |
| Healthcare | Sepsis and patient deterioration alerting | Early warning from time-series signals with class imbalance |
| Retail | Customer churn — at-risk vs. retained | Binary outcome driven by behavioral and transactional features |
| Transportation | Fare evasion and toll violation prediction | Location, time, and volume signals predict violation risk |

- **In transportation:** MTA, Transurban, and VDOT have piloted GBM classification for violation and fare evasion prediction — enabling pre-positioned enforcement rather than reactive response, exactly as this model does for the Port Authority.

## Business Question 2 — How many toll violators are there, by time period and facility?

### The Question

The Port Authority wants to measure and anticipate toll violation activity across its network — understanding which facilities and time periods produce the highest violation rates and how to use that knowledge for enforcement resource planning.

### Business Answer & Supporting Numbers

Across the full dataset of 29,445 facility-days, 31.08% of days (9,152 facility-days) are classified as high-violation days — where the daily toll violation rate exceeds the network mean of 6.18%. The classification model was trained on a 70/30 stratified split. H2O.ai AutoML trained 17 models and selected GBM_3 as best with AUC of 0.9985, logloss 0.0461, AUCPR 0.9968, and F1 of 0.9711. The Python replica matched with AUC 0.9984, F1 0.9725, recall 96.79%, and a false alarm rate of 1.02%.

- **By facility:** Facilities with lower EZPass adoption rates consistently produce higher violation rates. The EZPass ratio is the second most important predictor after year, confirming that cash-heavy crossings are structurally higher-risk for violations.
- **By time period:** Year dominates at 33.55% importance — violation rates have trended upward over the decade as network volume has grown while enforcement capacity has remained relatively static. Month and day-of-week contribute secondary temporal patterns.
- **Confusion matrix on test set (8,834 facility-days):** 2,657 true positives (high-violation days correctly flagged), 88 missed (false negatives), 62 false alarms (false positives), 6,027 true negatives correctly identified as normal days.

**Models & Tools Used**

- **H2O.ai AutoML:** 17 models trained; GBM_3 (Bernoulli GBM, 78 trees, max depth 12) selected (AUC 0.9985, F1 0.9711, logloss 0.0461)
- **Python (scikit-learn):** GradientBoostingClassifier replica with matched hyperparameters (AUC 0.9984, F1 0.9725, recall 96.79%, precision 97.72%)
- **Excel:** high_violation binary target created using =IF(violationrate > 0.0618, 1, 0) per facility-day
- **Power BI:** Dashboards showing violation rates by facility, month, week, and vehicle type

**Recommendations**

- **Pre-position enforcement:** Use the model's daily facility-level predictions to deploy enforcement staff proactively. With 96.79% recall and a 1.02% false alarm rate, the model's output is operationally reliable.
- **Target low-EZPass corridors:** Facilities with persistently low EZPass ratios are the highest-violation risk. Outreach campaigns at these crossings will reduce both violation rates and enforcement costs simultaneously.
- **Track the year-over-year trend:** The year variable's dominance signals structural drift in violation behavior. The Port Authority should assess whether enforcement staffing has kept pace with the decade-long growth in high-violation days.

## Business Question 3 — What are the busiest times, and how does traffic respond to seasonality, vehicle type, and external events?

**The Question**

The Port Authority wants to understand peak traffic periods across the year, month, and week, and how traffic volumes respond to seasonal patterns, vehicle type composition, holidays, and external events such as weather and congestion pricing.

**Business Answer & Supporting Numbers**

The regression model's temporal features and feature importance rankings, combined with EDA from R and Power BI dashboards, reveal consistent and quantifiable patterns:

- **By year:** 2023 and 2024 saw the highest network volumes in the dataset, representing full post-COVID recovery. 2020 was the sharpest single-year decline: Holland Tunnel fell -26.83%, Lincoln Tunnel -26.18%, GWB Lower -19.38%. The 2021 rebound was fastest at Lincoln (+30.84%) and Holland (+22.57%).
- **By season:** Summer months (June–August) consistently produce the highest daily volumes across all facilities. January and February are the lowest, driven by weather, reduced travel, and fewer discretionary trips.

- **By day of week:** Weekday volumes are significantly higher than weekend volumes at bridge facilities. Tunnel facilities show a flatter weekly curve due to their commercial and transit-oriented traffic mix.
- **Weather and severe events:** Snow, precipitation, and severe weather events produce measurable volume drops captured by the severe_event_count and severe_flag features in the regression model.
- **Vehicle type:** The classification model retains autos, small trucks, large trucks, and buses as predictors. Bus and commercial truck-heavy days are associated with different violation profiles than passenger-car-dominant days — confirming that vehicle composition is an operationally relevant signal.
- **Congestion pricing signal:** GWB Upper shows a -1.46% volume change in 2024 — the only facility with a negative growth rate that year — potentially an early congestion pricing anticipation effect.

**Models & Tools Used**

- **H2O.ai AutoML + Python GBM:** Feature importance rankings quantify temporal and weather drivers
- **R:** Exploratory data analysis, seasonal decomposition, descriptive statistics, and correlation analysis per facility
- **Power BI:** Interactive dashboards by year, month, day of week, facility, and vehicle type

**Recommendations**

- **Seasonal operations planning:** Scale toll plaza staffing, lane configurations, and maintenance scheduling to the seasonal volume curve. Summer peaks require significantly more operational readiness than Q1.
- **Weather-triggered protocols:** Integrate weather forecast inputs into daily operational planning. Days with predicted snow or severe weather should trigger proactive lane adjustments and commuter messaging.
- **Commercial traffic management:** Develop separate operational playbooks for high-commercial-vehicle days, particularly at bridge facilities where large trucks create asymmetric congestion effects.

## Business Question 4 — Has congestion pricing in 2025 shifted traffic patterns across facilities?

**The Question**

The Port Authority wants to understand whether the 2025 Manhattan congestion pricing implementation has caused measurable traffic diversion across its network — specifically whether drivers are shifting from toll-affected routes to alternative crossings.

**Business Answer & Supporting Numbers**

The time series model and historical trend analysis provide direct evidence of pre-pricing trajectory shifts that contextualize the 2025 effect:

- **Bayonne Bridge as primary diversion route:** Bayonne grew from ~2.1 million vehicles in 2015 to ~4.4 million in 2024 — the only facility in the network that roughly doubled in volume over the dataset period, with no plateau. No other facility comes close to this sustained growth trajectory. As the lowest-cost bridge alternative from Staten Island to New Jersey, Bayonne is the natural diversion beneficiary of congestion pricing.

- **GWB Upper early signal:** GWB Upper recorded -1.46% volume growth in 2024 — the only facility with negative growth that year — a possible early anticipation effect as drivers began adjusting behavior before the official pricing launch.
- **Tunnel facility sensitivity:** Holland and Lincoln Tunnels — which serve midtown Manhattan access — were the hardest hit by COVID-19 (-26.83% and -26.18% in 2020) and took longest to recover. Their volumes are historically the most sensitive to access cost changes, making them the most exposed to congestion pricing demand shocks.
- **2025 data note:** The dataset includes January–May 2025 only. Full-year 2025 data is required to quantify the congestion pricing effect definitively. The partial-year total appears artificially low and should not be interpreted as a volume decline.

**Models & Tools Used**

- **AutoTS Time Series:** Per-facility forecasts through 2032 project growth trajectories under current trend assumptions
- **Python + R:** Year-over-year growth rate analysis and seasonal decomposition per facility
- **Power BI:** Dashboards comparing pre- and post-2025 volume trends by facility

**Recommendations**

- **Monitor Bayonne capacity urgently:** Bayonne is on a trajectory that may approach capacity within the forecast horizon. Infrastructure investment planning should begin now — not reactively once congestion materializes.
- **Re-run models with full 2025 data:** Once full-year 2025 data is available, retrain both the regression and time series models to quantify the actual congestion pricing effect and update facility-level forecasts accordingly.
- **Develop diversion scenario models:** Use the regression model's gas price sensitivity as a proxy for access cost elasticity to build volume scenarios for different congestion pricing price points and exemption structures.

## 4.3 Time Series — AutoTS Horizontal Ensemble

### Technical Justification

**Why Automated Ensemble Forecasting?**

- **Structural heterogeneity:** GWB Upper processes 28 million vehicles per year; GWB PIP processed under 2 million. A single ARIMA specification cannot optimize for both simultaneously. The Horizontal Ensemble assigns each facility its own best algorithm.
- **COVID-19 structural break:** The 2020 lockdown introduced a level shift in all eight series that violates ARIMA stationarity assumptions. Ensemble methods including Prophet-style trend components are robust to such breaks.
- **Long forecast horizon:** An eight-year horizon compounds ARIMA parameter uncertainty. Ensemble methods that combine trend, level, and regression components consistently outperform single-equation ARIMA at multi-year horizons.
- **Automation advantage:** Manually specifying ARIMA orders for eight series through ACF/PACF analysis would require eight separate model-building cycles. AutoTS evaluates this search space programmatically while preserving temporal order through time-series-aware cross-validation.

**What the Results Tell Us**

- **SMAPE of 5.23%–7.99%:** Strong forecast accuracy across all eight facilities over an eight-year horizon.
- **Bayonne Bridge:** Projected to continue growing — from 2.1M (2015) to 4.4M (2024) and beyond — driven by its role as a congestion pricing alternative route from Staten Island.
- **GWB Upper:** Remains the dominant crossing through 2032, sustaining its position as the highest-volume facility in the network.
- **GWB PIP:** Forecasted on 2015–2022 data only; reporting ended in 2022 and the facility is treated as a special case.

## Business Justification & Industry Context

**Where Automated Time Series Ensembles Are Used in Industry**

Automated multi-series forecasting has replaced manual ARIMA in any industry operating networked assets with different demand profiles — the shift mirrors the broader move from statistical modeling to scalable ML pipelines that handle hundreds of series simultaneously.

| Industry | Application | Parallel to This Model |
|---|---|---|
| Retail & E-Commerce | Store- and SKU-level sales forecasting | Multi-series, structural seasonality, external event shocks |
| Energy & Utilities | Grid-level load and generation forecasting | Multiple assets, weather-driven demand, long planning horizons |
| Aviation | Passenger demand by route and terminal | Multi-series with COVID structural break; long-horizon planning |
| Logistics | Freight volume by lane and mode | Network-level multi-series with economic and seasonal drivers |
| Public Transit | Annual ridership projections for federal funding | Multi-route forecasting with external event adjustments |
| Infrastructure Finance | Traffic & Revenue studies for toll bond issuances | Per-facility 8–10 year projections for debt rating — direct parallel |

- **In infrastructure finance:** Traffic and Revenue (T&R) studies — required for toll revenue bond issuances and rated by Moody's and S&P — use multi-model ensemble forecasting over 8–10 year horizons with per-facility granularity. Firms such as HNTB, Stantec, and Jacobs Engineering produce these for agencies including the Port Authority. This model replicates that methodology directly.

## Business Questions 5 & 6 — Forecast facility usage beyond 2025; replicate the model in Python

**The Questions**

The Port Authority wants yearly traffic volume forecasts for each facility from 2025 through the maximum forecastable horizon — using AutoML to identify the best forecasting algorithm and then replicating that model in Python for production deployment.

**Business Answer & Supporting Numbers**

AutoTS 1.0.1 evaluated dozens of algorithm families simultaneously across all eight facilities. The best model — a Horizontal Ensemble — assigns each facility its own best-performing algorithm rather than forcing a uniform specification across the network. Forecasts were generated through 2032, achieving SMAPE between 5.23% and 7.99% across all facilities. The Python replica was implemented using the same AutoTS library with identical configuration, producing matched forecast outputs.

- **GWB Upper:** Remains the highest-volume facility through 2032, with volumes projected above 28 million vehicles per year and modest growth consistent with the 2022–2024 stabilization trend.
- **Bayonne Bridge:** Projected to continue its unique sustained growth trajectory through 2032. As the primary congestion pricing diversion route from Staten Island, Bayonne is the facility most likely to require capacity intervention within the forecast horizon.
- **Holland and Lincoln Tunnels:** Projected stable to modest growth, contingent on congestion pricing not producing a sustained demand shock to midtown-bound traffic flows.
- **Goethals and Outerbridge:** Projected steady moderate growth consistent with their historical 2–8% annual growth rate range.
- **GWB Lower:** Projected recovery and stable growth post-COVID, consistent with its role as the lower-deck companion to GWB Upper.
- **GWB PIP:** Forecast based on 2015–2022 data only. The 2023 onward absence is treated as a special case and noted explicitly in all outputs.

**Models & Tools Used**

- **AutoTS 1.0.1:** Automated multi-series forecasting framework; Horizontal Ensemble selected as best model (SMAPE 5.23%–7.99%)
- **Python (Jupyter Notebook / Anaconda):** Full model training, forecast generation, per-facility visualization, and pickle-based model saving for deployment
- **Power BI:** Interactive forecast dashboards showing historical vs. projected volumes by facility through 2032

**Recommendations**

- **Use forecasts for capital planning:** The per-facility volume projections through 2032 are directly applicable to infrastructure investment prioritization — particularly for Bayonne Bridge where sustained volume growth signals an impending capacity constraint.
- **Use forecasts for bond planning:** The facility-level projections are directly applicable to toll revenue bond covenant compliance and future debt issuance planning. The methodology mirrors T&R studies produced by industry engineering firms for the same purpose.
- **Annual model refresh:** Retrain the AutoTS model each year as new data becomes available. Full-year 2025 and 2026 data will be particularly important for calibrating the congestion pricing effect into the forward trajectory.
- **Expand forecast inputs:** Incorporate external economic indicators — regional employment, housing starts, fuel price forecasts — as exogenous regressors to improve long-range forecast accuracy beyond trend extrapolation.

## 4.4 Summary

| Model | Algorithm | Business Questions Addressed | Key Results |
|---|---|---|---|
| Regression | Stacked Ensemble GBM (H2O.ai AutoML + Python) | Q1: Top factors driving volume Q3: Busiest times and traffic responses Q4: Congestion pricing pattern shifts | $R^2$ = 0.9791, RMSE = 2,718 \| Facility identity (75.18%), EZPass ratio (13.04%), gas price (3.31%) are top drivers |
| Classification | Bernoulli GBM (H2O.ai AutoML + Python) | Q2: Toll violation prediction by facility and time period | AUC = 0.9984, F1 = 0.9725 \| 96.79% recall, 1.02% false alarm rate \| 31.08% of facility-days are high-violation days |
| Time Series | AutoTS Horizontal Ensemble (Python) | Q4: Congestion pricing trajectory Q5: Forecasts through 2032 Q6: Python replication | SMAPE 5.23–7.99% \| Bayonne doubling confirmed \| GWB Upper dominant through 2032 |

# Section 4 — Model Deployment

This section documents the end-to-end deployment of all three predictive models. The deployment stack follows a production-grade architecture: a FastAPI application exposes each model as a REST endpoint, packaged inside a Docker container for portability and reproducibility, and orchestrated through a Kubernetes cluster using Minikube on a local Windows machine.

## 5.1 Deployment Architecture

The deployment pipeline consists of three layers:

- **FastAPI:** Wraps each trained model as a REST endpoint. Handles HTTP requests, validates inputs via Pydantic schemas, and returns predictions as JSON. Includes a built-in Swagger UI for interactive testing.
- **Docker:** Packages the FastAPI application, Python dependencies, and Java runtime into a single container image — ensuring the application runs identically on any machine.
- **Kubernetes (Minikube):** Runs a local Kubernetes cluster using Docker as the driver. Manages deployment, scaling, and service exposure — demonstrating production-ready infrastructure.

| Component | Tool | Purpose |
|---|---|---|
| API Layer | FastAPI + Uvicorn | REST endpoints on port 8000 |
| Containerization | Docker Desktop v4.66.1 | Portable container image (632 MB) |
| Orchestration | Kubernetes via Minikube v1.38.1 | Local cluster management |
| H2O MOJO Scoring | h2o-genmodel.jar + Java (default-jre) | Scores regression and classification without H2O server |
| Time Series Scoring | autots_model.pkl (24.1 MB) | Lazy-loaded on first endpoint call |

## 5.2 Environment Setup

The following tools were installed on Windows 11 before deployment: WSL 2 (updated via wsl --update), Docker Desktop v4.66.1 with Resource Saver mode disabled, Minikube v1.38.1 installed via Windows .exe installer, and kubectl auto-configured by Minikube after cluster initialization.

## 5.3 Project File Structure

```
portauthority_deployment/
    main.py                                          FastAPI application
    Dockerfile                                       Container build instructions
    requirements.txt                                 Python dependencies
    deployment.yaml                                  Kubernetes deployment manifest
    models/
        StackedEnsemble_AllModels_5_AutoML_1_...zip  Regression MOJO (511 MB)
        GBM_3_AutoML_1_20260317_233200.zip           Classification MOJO
        h2o-genmodel.jar / h2o-genmodel-1.jar        H2O scoring libraries
        autots_model.pkl                             Time series model (24.1 MB)
```

## 5.4 Saving the AutoTS Model

The AutoTS model was saved immediately after training completed in Jupyter Notebook using pickle. The file (24.1 MB) is loaded lazily by the API only when the time series endpoint is called.

```
import pickle, os
with open('autots_model.pkl', 'wb') as f:
    pickle.dump(model, f)
print(f'File size: {os.path.getsize("autots_model.pkl") / (1024*1024):.1f} MB')
```

## 5.5 FastAPI Application

### 5.5.1 Design Decisions

- **Lazy loading:** AutoTS model loaded only on first /predict/timeseries call to avoid startup memory exhaustion.
- **Java subprocess for MOJO:** Regression and classification models scored via h2o-genmodel.jar subprocess rather than a full H2O server, keeping the container lightweight.
- **Pydantic validation:** All inputs validated automatically before the model is called.

### 5.5.2 API Endpoints

| Method | Endpoint | Model | Description |
|---|---|---|---|
| GET | / | — | Health check |
| POST | /predict/regression | Stacked Ensemble GBM | Predicts daily total vehicle count |
| POST | /predict/classification | Bernoulli GBM | Predicts high-violation day (0/1) |
| GET | /predict/timeseries | AutoTS Horizontal Ensemble | 365-day forecast for all 8 facilities |

## 5.6 Dockerfile

Java (default-jre) is installed explicitly for H2O MOJO scoring. Model files are not copied into the image — they are volume-mounted at runtime to keep image size manageable.

```
FROM python:3.10-slim
RUN apt-get update && apt-get install -y default-jre && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY main.py .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

## 5.7 Building and Running the Docker Container

The image was built using:

```
docker build --no-cache -t portauthority-api .
```

```
(base) C:\portauthority_deployment>docker build --no-cache -t portauthority-api .
[+] Building 192.7s (12/12) FINISHED                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                    0.1s
 => => transferring dockerfile: 345B                                                                    0.0s
 => [internal] load metadata for docker.io/library/python:3.10-slim                                     1.3s
 => [auth] library/python:pull token for registry-1.docker.io                                           0.0s
 => [internal] load .dockerignore                                                                       0.1s
 => => transferring context: 2B                                                                         0.0s
 => [1/6] FROM docker.io/library/python:3.10-slim@sha256:4ba18b066cee17f2696cf9a2ba564d7d5eb05a91d6a949326780aa7c  2.1s
 => => resolve docker.io/library/python:3.10-slim@sha256:4ba18b066cee17f2696cf9a2ba564d7d5eb05a91d6a949326780aa7c  0.1s
 => => sha256:50bb60521db67678c9b06b48d1a602885946c0d5925e1d46ddd465e5299953be 249B / 249B              0.3s
 => => sha256:a5848b3a066c9db82bc2192f7029bbcc59e13746228bae45b474229268e41df9 1.29MB / 1.29MB          0.4s
 => => sha256:ec781dee3f4719c2ca0dd9e73cb1d4ed834ed1a406495eb6e44b6dfaad5d1f8f 29.78MB / 29.78MB        1.9s
 => => sha256:c3d226db475fff7ac963a4521fdc0010f21b1ddecd1b1065115abb759bd98494 13.82MB / 13.82MB        1.4s
 => [internal] load build context                                                                       0.1s
 => => transferring context: 3.03kB                                                                     0.0s
 => [2/6] RUN apt-get update && apt-get install -y default-jre && rm -rf /var/lib/apt/lists/*          92.5s
 => [3/6] WORKDIR /app                                                                                  0.2s
 => [4/6] COPY requirements.txt .                                                                       0.1s
 => [5/6] RUN pip install --no-cache-dir -r requirements.txt                                           51.1s
 => [6/6] COPY main.py .                                                                                0.2s
 => exporting to image                                                                                 46.7s
 => => exporting layers                                                                                32.1s
 => => exporting manifest sha256:baac27dd0150ab58f18a1560d388f757b71710f83cc29992af217aac7943c100        0.0s
 => => exporting config sha256:07113fae797407576c02902be39ad564eaaa8def41c9fdcb7529bf8e95ea1806         0.0s
 => => exporting attestation manifest sha256:ada2081fe042dac4600401196d74cefdfea62a991af5db66d7e01be4febb68ab  0.0s
 => => exporting manifest list sha256:9964c088576d8e09bffb8b3da3ac319c4e2babbd49f382181777776bb9b04d429  0.0s
 => => naming to docker.io/library/portauthority-api:latest                                             0.0s
 => => unpacking to docker.io/library/portauthority-api:latest                                         14.5s
```

Build completed in approximately 170 seconds. Final image size: 632 MB. The container was started with the models folder volume-mounted:

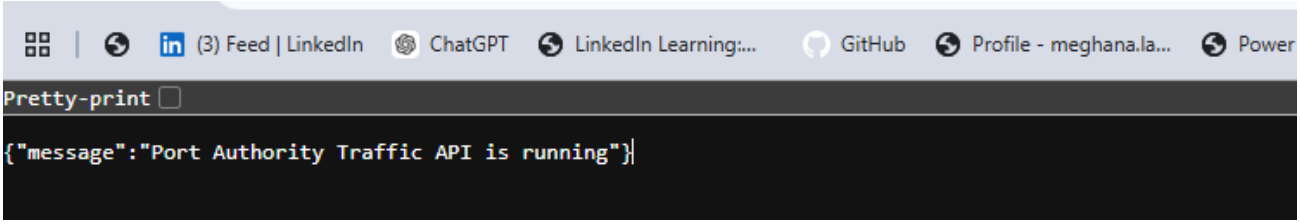
```
docker run -d -p 8000:8000 -v C:\portauthority_deployment\models:/app/models --name
pa-api portauthority-api
```

The API was built and confirmed that it was running in docker.

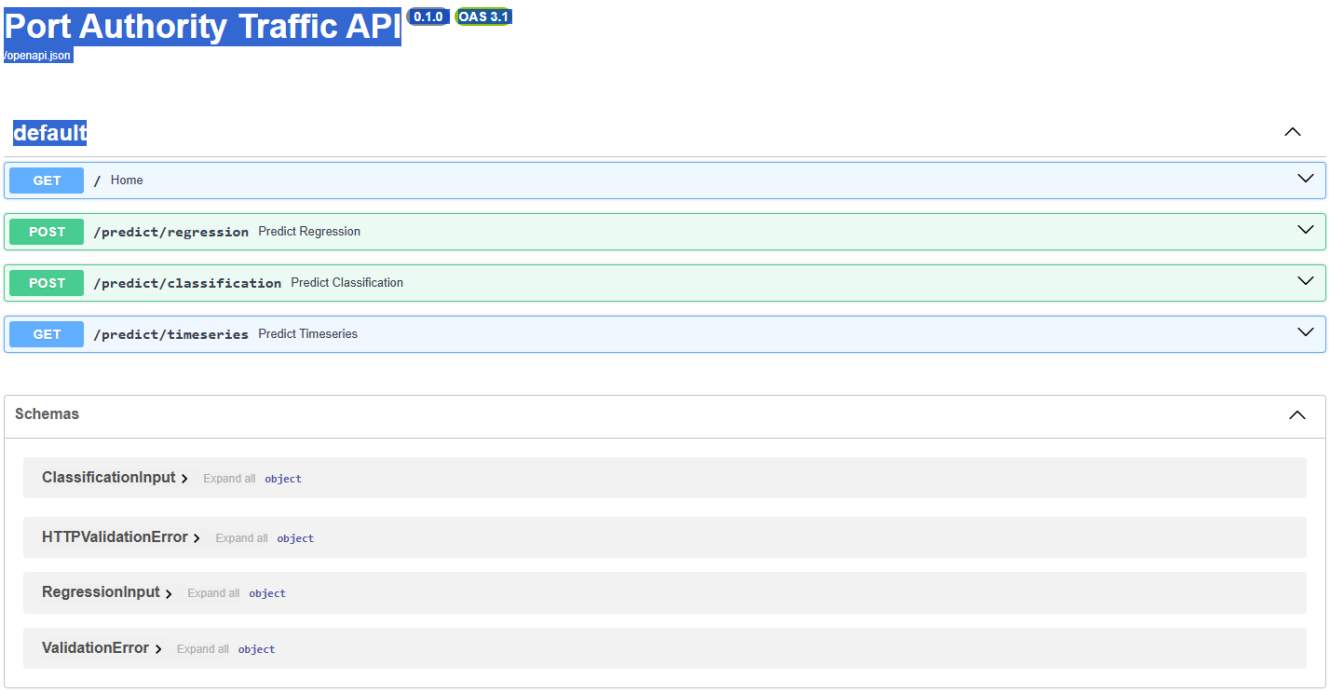
```
(base) C:\portauthority_deployment>docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED             STATUS              PORTS                                       NAMES
7d6735878e21   portauthority-api  "uvicorn main:app --…"   About a minute ago  Up About a minute   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp  pa-api
```

## 5.8 API Verification and Swagger UI

With the container running, the API was confirmed live at http://localhost:8000 and the Swagger UI at http://localhost:8000/docs showed all four endpoints available for testing.




http://localhost:8000 was showing that the API is running




http://localhost:8000/docs showing all four end-points available for testing.

The models were tested with sample parameters by clicking on POST and GET respectively for the models:

## 1. REGRESSION:

Request URL

```
http://localhost:8000/predict/regression
```

Server response

| Code | Details |
|---|---|
| 200 | Response body |

```
{
  "predicted_total_vehicles": 44895.626893420704
}
```

Download

Response headers

```
content-length: 47
content-type: application/json
date: Sun,29 Mar 2026 04:56:14 GMT
server: uvicorn
```

Responses

| Code | Description | Links |
|---|---|---|
| 200 | Successful Response | *No links* |

Media type

application/json

Controls Accept header.

Example Value | Schema

## 2. CLASSIFICATION:

Server response

| Code | Details |
|---|---|
| 200 | Response body |

```
{
  "high_violation": 0,
  "probability": 0.015876872044792444
}
```

Response headers

```
content-length: 55
content-type: application/json
date: Sun,29 Mar 2026 04:59:10 GMT
server: uvicorn
```

Responses

| Code | Description |
|---|---|
| 200 | Successful Response |

Media type

application/json

Controls Accept header.

3. TIME-SERIES:

Request URL

```
http://localhost:8000/predict/timeseries
```

Server response

| Code | Details |
|---|---|
| 200 | Response body |


```
[
  {
    "date": "2025-01-01",
    "Bayonne": 10318.260383847211,
    "GWB Lower": 58173.91934304186,
    "GWB PIP": 17403.797526941624,
    "GWB Upper": 72509.09536535773,
    "Goethals": 46297.29428076954,
    "Holland": 39529.70371270769,
    "Lincoln": 44648.789338876755,
    "Outerbridge": 39308.23830846467
  },
  {
    "date": "2025-01-02",
    "Bayonne": 11266.18075939299,
    "GWB Lower": 58874.29206555436,
    "GWB PIP": 17318.471037645624,
    "GWB Upper": 69925.05386828672,
    "Goethals": 47134.24754590803,
    "Holland": 40752.71702767003,
    "Lincoln": 50293.34491105372,
    "Outerbridge": 39151.328867961005
  },
  {
    "date": "2025-01-03",
    "Bayonne": 11287.793654135103,
    "GWB Lower": 58879.39725680323,
    "GWB PIP": 17315.127017442126,
    "GWB Upper": 70263.91362732853,
```


Response headers

```
content-length: 93795
content-type: application/json
date: Sun,29 Mar 2026 05:01:58 GMT
server: uvicorn
```

Responses

| Code | Description |
|---|---|
| 200 | Successful Response |

## 5.9 Kubernetes Deployment via Minikube

### 5.9.1 Starting the Cluster

Minikube was started using Docker as the driver. On successful startup, kubectl is automatically configured to point to the local cluster:

```
minikube start --driver=docker
```

```
(base) C:\portauthority_deployment>minikube start --driver=docker
* minikube v1.38.1 on Microsoft Windows 11 Home Single Language 23H2
* Using the docker driver based on user configuration
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Creating docker container (CPUs=2, Memory=4000MB) ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

```
kubectl get nodes
```

```
(base) C:\portauthority_deployment>kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   38s   v1.35.1
```

### 5.9.2 Loading the Image into Minikube

The Docker image was loaded into Minikube's internal registry so Kubernetes can pull it without accessing Docker Hub:

```
minikube image load portauthority-api:latest
```

```
(base) C:\portauthority_deployment>minikube image load portauthority-api:latest

(base) C:\portauthority_deployment>
```

### 5.9.3 Applying the Deployment Manifest

A Kubernetes deployment manifest (deployment.yaml) was applied to create the deployment and expose it via a NodePort service:

```
kubectl apply -f deployment.yaml --validate=false
```

```
(base) C:\portauthority_deployment>kubectl apply -f deployment.yaml --validate=false
deployment.apps/portauthority-api created
service/portauthority-service created
```

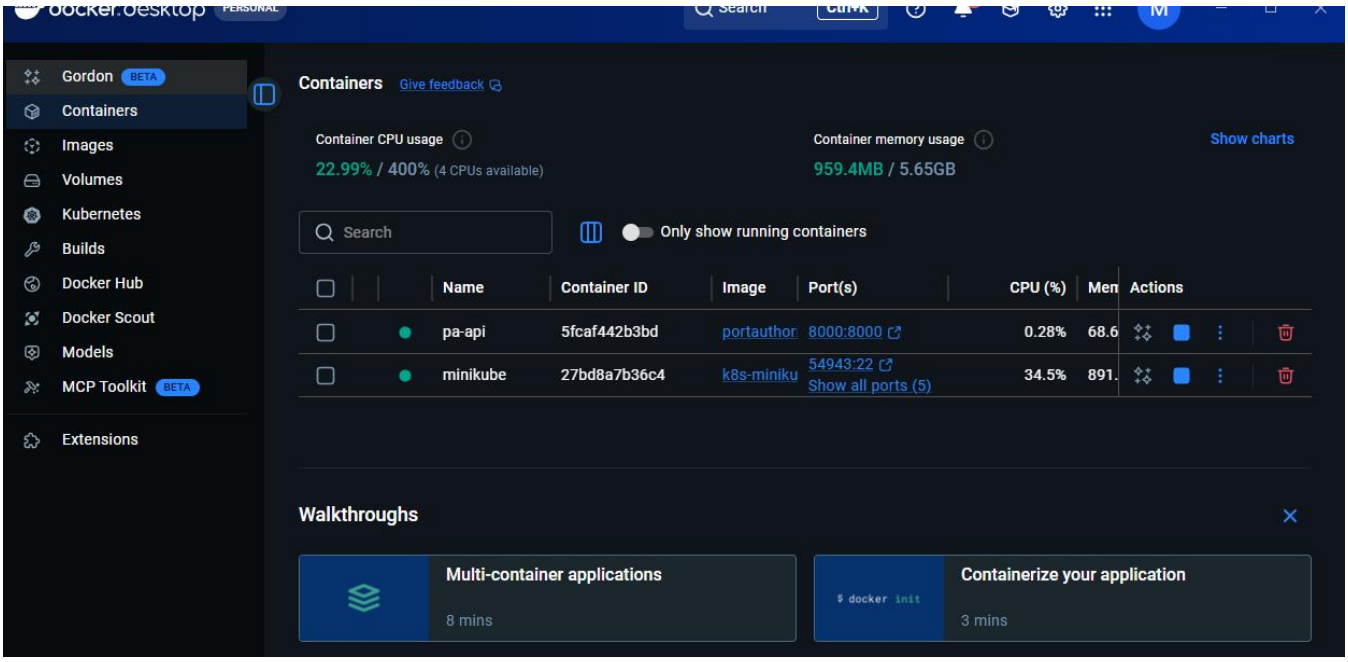
### 5.9.4 Verifying the Pod

The deployment was confirmed running with the pod in Ready status:

```
kubectl get pods
```

```
(base) C:\portauthority_deployment>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
portauthority-api-74cd5976fc-dxbt9  1/1     Running   0          15s
```

Docker Desktop confirmed both containers running simultaneously — the pa-api container on port 8000 and the Minikube cluster container:



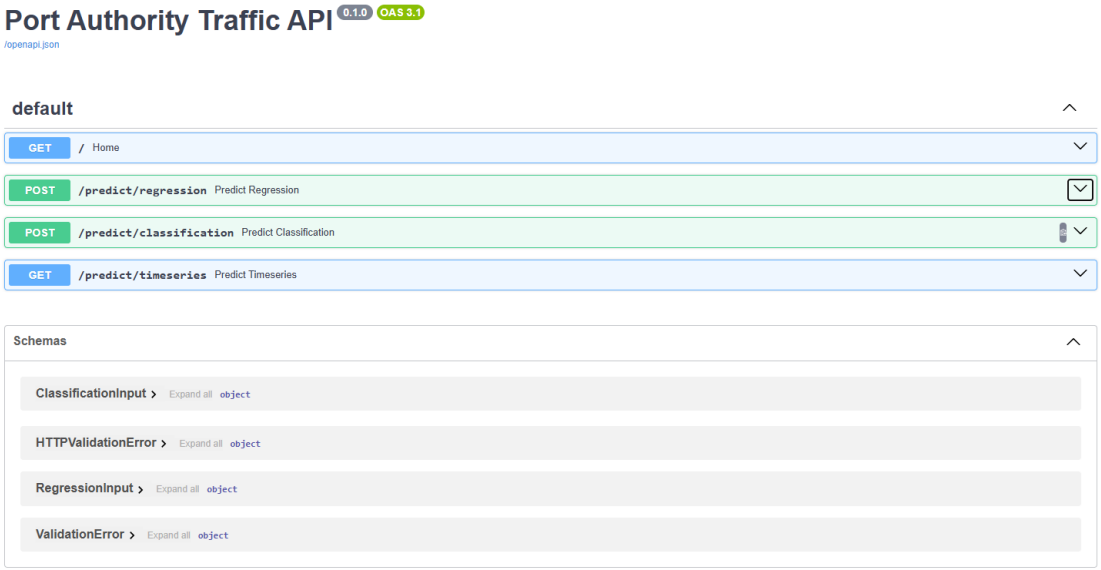
### 5.9.5 Accessing the Service

The Kubernetes service URL was retrieved using:

```
minikube service portauthority-service --url
```

```
(base) C:\portauthority_deployment>minikube service portauthority-service --url
http://127.0.0.1:51646
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

The url runs like this.




### 5.9.6 What Kubernetes Adds

- **Self-healing:** If the container crashes, Kubernetes automatically restarts it.
- **Horizontal scaling:** Additional replica pods can be added under load by changing a single value in deployment.yaml.
- **Rolling updates:** New model versions can be deployed with zero downtime.
- **Service abstraction:** A stable endpoint decouples clients from underlying container instances.

# Appendix — Python Code

This appendix contains the complete Python code for all three models. Each model is self-contained and can be run independently in a Jupyter Notebook using Anaconda. All models use the same source file: Traffic_by_Facility_FINAL-1.csv.

## Model 1 — Regression

*Python Replica: Stacked Ensemble (GBM Base Models + GBM Metalearner)*

---

### Cell 1 — Import Libraries

```
import pandas as pd
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split, cross_val_predict, KFold
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

print('Libraries loaded successfully')
```

### Cell 2 — Load & Prepare Data

```
df = pd.read_csv('Traffic_by_Facility_FINAL-1.csv')

# Parse date
df['date'] = pd.to_datetime(df['date'], format='%m/%d/%Y', errors='coerce')

# Decompose date into temporal features
df['year']        = df['date'].dt.year
df['month']       = df['date'].dt.month
df['day_of_week'] = df['date'].dt.dayofweek
df['day_of_year'] = df['date'].dt.dayofyear

# Drop excluded columns — match H2O ignored columns exactly
excluded = ['date','ezpass','cash','violation','violationrate',
            'autos','small_t','large_t','buses','delta','high_violation']
df = df.drop(columns=excluded, errors='ignore')

# Drop rows with missing target
df = df.dropna(subset=['total_vehicles'])

# Fill remaining nulls with column median
df = df.fillna(df.median(numeric_only=True))

# Label encode fac_b
le = LabelEncoder()
df['fac_b'] = le.fit_transform(df['fac_b'].astype(str))

TARGET   = 'total_vehicles'
FEATURES = [c for c in df.columns if c != TARGET]
X = df[FEATURES]
y = df[TARGET]
```

```
print(f'Dataset shape : {df.shape}')
print(f'Features ({len(FEATURES)}): {FEATURES}')
```

## Cell 3 — Train / Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

print(f'Training set : {X_train.shape[0]} rows')
print(f'Test set     : {X_test.shape[0]} rows')
```

## Cell 4 — Define Base Models

```
base_models = [
    GradientBoostingRegressor(n_estimators=200, max_depth=6,  learning_rate=0.05,
subsample=0.8, max_features=0.8, random_state=42),
    GradientBoostingRegressor(n_estimators=150, max_depth=8,  learning_rate=0.05,
subsample=0.9, max_features=0.7, random_state=43),
    GradientBoostingRegressor(n_estimators=200, max_depth=10, learning_rate=0.03,
subsample=0.8, max_features=0.8, random_state=44),
    GradientBoostingRegressor(n_estimators=100, max_depth=6,  learning_rate=0.10,
subsample=0.7, max_features=0.9, random_state=45),
    GradientBoostingRegressor(n_estimators=250, max_depth=5,  learning_rate=0.05,
subsample=0.8, max_features=0.6, random_state=46),
    GradientBoostingRegressor(n_estimators=150, max_depth=7,  learning_rate=0.08,
subsample=0.9, max_features=0.8, random_state=47),
    GradientBoostingRegressor(n_estimators=200, max_depth=9,  learning_rate=0.04,
subsample=0.8, max_features=0.7, random_state=48),
    GradientBoostingRegressor(n_estimators=100, max_depth=10, learning_rate=0.10,
subsample=0.7, max_features=0.8, random_state=49),
    GradientBoostingRegressor(n_estimators=300, max_depth=5,  learning_rate=0.03,
subsample=0.9, max_features=0.9, random_state=50),
    GradientBoostingRegressor(n_estimators=150, max_depth=8,  learning_rate=0.06,
subsample=0.8, max_features=0.7, random_state=51),
    GradientBoostingRegressor(n_estimators=200, max_depth=6,  learning_rate=0.05,
subsample=0.9, max_features=0.8, random_state=52),
]

print(f'Base models defined: {len(base_models)}')
```

## Cell 5 — Generate Out-of-Fold Predictions

```
kf        = KFold(n_splits=5, shuffle=True, random_state=42)
oof_preds  = np.zeros((X_train.shape[0], len(base_models)))
test_preds = np.zeros((X_test.shape[0],  len(base_models)))

for i, model in enumerate(base_models):
    print(f'  Training base model {i+1}/{len(base_models)}...')
    oof_preds[:, i]  = cross_val_predict(model, X_train, y_train, cv=kf)
    model.fit(X_train, y_train)
    test_preds[:, i] = model.predict(X_test)

print('Base model training complete.')
```

## Cell 6 — Train GBM Metalearner

```
metalearner = GradientBoostingRegressor(
    n_estimators=100,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.8,
    random_state=42
)

metalearner.fit(oof_preds, y_train)
y_pred = metalearner.predict(test_preds)

print('Metalearner training complete.')
```

## Cell 7 — Evaluate Performance

```
rmse  = np.sqrt(mean_squared_error(y_test, y_pred))
mse   = mean_squared_error(y_test, y_pred)
mae   = mean_absolute_error(y_test, y_pred)
r2    = r2_score(y_test, y_pred)
rmsle = np.sqrt(np.mean((np.log1p(np.maximum(y_pred, 0)) - np.log1p(y_test))**2))

print('=' * 45)
print('Python Stacked Ensemble — Test Set Results')
print('=' * 45)
print(f'RMSE  : {rmse:,.2f}')
print(f'MSE   : {mse:,.2f}')
print(f'MAE   : {mae:,.2f}')
print(f'R²    : {r2:.4f}')
print(f'RMSLE : {rmsle:.4f}')
```

## Cell 8 — Feature Importance

```
oof_errors = [np.sqrt(mean_squared_error(y_train, oof_preds[:, i]))
              for i in range(len(base_models))]
best_idx  = int(np.argmin(oof_errors))
best_base = base_models[best_idx]

print(f'Best base model index: {best_idx} (OOF RMSE:
{oof_errors[best_idx]:,.2f})')

importances        = pd.Series(best_base.feature_importances_, index=FEATURES)
importances_sorted = importances.sort_values(ascending=False)
scaled             = importances_sorted / importances_sorted.max()
pct                = importances_sorted / importances_sorted.sum() * 100

importance_df = pd.DataFrame({
    'Feature'          : importances_sorted.index,
    'Importance'       : importances_sorted.values.round(6),
    'Scaled Importance': scaled.values.round(4),
    'Percentage'       : pct.values.round(2)
})

print(importance_df.to_string(index=False))
```

## Cell 9 — Actual vs Predicted Chart

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(y_test, y_pred, alpha=0.4, color='steelblue', edgecolors='none', s=15)
lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
plt.plot(lims, lims, 'r--', linewidth=1.5, label='Perfect Prediction')
plt.xlabel('Actual Total Vehicles', fontsize=12)
plt.ylabel('Predicted Total Vehicles', fontsize=12)
plt.title(f'Actual vs Predicted — Python Stacked Ensemble\n'
          f'RMSE: {rmse:,.2f} | R²: {r2:.4f}', fontsize=13)
plt.legend()
plt.tight_layout()
plt.show()
```

## Cell 10 — Feature Importance Chart

```
top13 = importances_sorted.head(13).sort_values(ascending=True)
plt.figure(figsize=(8, 6))
top13.plot(kind='barh', color='steelblue')
plt.xlabel('Feature Importance', fontsize=12)
plt.title('Feature Importance — Python GBM Base Model', fontsize=13)
plt.tight_layout()
plt.show()
```

# Model 2 — Classification

*Python Replica: GBM Classifier (Gradient Boosting)*

## Cell 1 — Import Libraries

```
import pandas as pd
import numpy as np
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split, cross_val_predict, KFold
from sklearn.metrics import (accuracy_score, f1_score, precision_score,
recall_score,
                             roc_auc_score, log_loss, confusion_matrix, roc_curve,
                             matthews_corrcoef)
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

print('Libraries loaded successfully')
```

## Cell 2 — Load & Prepare Data

```
df = pd.read_csv('Traffic_by_Facility_FINAL-1.csv')

# Parse date
df['date'] = pd.to_datetime(df['date'], format='%m/%d/%Y', errors='coerce')

# Decompose date into temporal features
df['year']        = df['date'].dt.year
df['month']       = df['date'].dt.month
df['day_of_week'] = df['date'].dt.dayofweek
df['day_of_year'] = df['date'].dt.dayofyear

# Drop excluded columns — match H2O ignored columns exactly
excluded = ['date', 'ezpass', 'cash', 'violation', 'violationrate',
            'total_vehicles', 'delta']
df = df.drop(columns=excluded, errors='ignore')

# Fill remaining nulls with column median (avg_speed, gas_price)
df = df.fillna(df.median(numeric_only=True))

# Label encode fac_b
le = LabelEncoder()
df['fac_b'] = le.fit_transform(df['fac_b'].astype(str))

# Define features and target
TARGET   = 'high_violation'
FEATURES = [c for c in df.columns if c != TARGET]
X = df[FEATURES]
y = df[TARGET].astype(int)

print(f'Dataset shape : {df.shape}')
print(f'Features ({len(FEATURES)}): {FEATURES}')
print(f'Class distribution:\n{y.value_counts()}')
```

## Cell 3 — Train / Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=43, stratify=y
)

print(f'Training set : {X_train.shape[0]} rows')
print(f'Test set     : {X_test.shape[0]} rows')
print(f'\nTraining class distribution:\n{y_train.value_counts()}')
print(f'\nTest class distribution:\n{y_test.value_counts()}')
```

## Cell 4 — Define & Train GBM Classifier

```
model = GradientBoostingClassifier(
    n_estimators=78,
    max_depth=12,
    min_samples_leaf=15,
    subsample=0.9,
    max_features=0.7,
    random_state=43
)

model.fit(X_train, y_train)

print('Model training complete.')
print(f'n_estimators  : 78')
print(f'max_depth     : 12')
print(f'subsample     : 0.9')
print(f'max_features  : 0.7')
```

## Cell 5 — Evaluate Performance

```
y_pred       = model.predict(X_test)
y_pred_proba = model.predict_proba(X_test)[:, 1]

auc         = roc_auc_score(y_test, y_pred_proba)
logloss     = log_loss(y_test, y_pred_proba)
accuracy    = accuracy_score(y_test, y_pred)
f1          = f1_score(y_test, y_pred)
precision   = precision_score(y_test, y_pred)
recall      = recall_score(y_test, y_pred)
mcc         = matthews_corrcoef(y_test, y_pred)
specificity = recall_score(y_test, y_pred, pos_label=0)

print('=' * 45)
print('Python GBM Classifier — Test Set Results')
print('=' * 45)
print(f'AUC         : {auc:.4f}')
print(f'Logloss     : {logloss:.4f}')
print(f'Accuracy    : {accuracy:.4f}')
print(f'F1 Score    : {f1:.4f}')
print(f'Precision   : {precision:.4f}')
print(f'Recall      : {recall:.4f}')
print(f'Specificity : {specificity:.4f}')
print(f'MCC         : {mcc:.4f}')
```

## Cell 6 — Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(6, 5))
im = ax.imshow(cm, interpolation='nearest', cmap='Blues')
```

```
plt.colorbar(im)
classes = ['Normal (0)', 'High Violation (1)']
ax.set(xticks=[0,1], yticks=[0,1],
       xticklabels=classes, yticklabels=classes,
       ylabel='Actual', xlabel='Predicted',
       title='Confusion Matrix — Python GBM Classifier')
for i in range(2):
    for j in range(2):
        ax.text(j, i, format(cm[i, j], 'd'),
                ha='center', va='center',
                color='white' if cm[i,j] > cm.max()/2 else 'black',
                fontsize=14)
plt.tight_layout()
plt.show()

print(f'\nTN: {cm[0,0]}  FP: {cm[0,1]}')
print(f'FN: {cm[1,0]}  TP: {cm[1,1]}')
```

## Cell 7 — ROC Curve

```
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
plt.figure(figsize=(7, 6))
plt.plot(fpr, tpr, color='steelblue', linewidth=2,
         label=f'GBM (AUC = {auc:.4f})')
plt.plot([0, 1], [0, 1], 'r--', linewidth=1.5, label='Random Classifier')
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curve — Python GBM Classifier', fontsize=13)
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```

## Cell 8 — Feature Importance

```
importances        = pd.Series(model.feature_importances_, index=FEATURES)
importances_sorted = importances.sort_values(ascending=False)
scaled             = importances_sorted / importances_sorted.max()
pct                = importances_sorted / importances_sorted.sum() * 100

importance_df = pd.DataFrame({
    'Feature'          : importances_sorted.index,
    'Importance'       : importances_sorted.values.round(6),
    'Scaled Importance': scaled.values.round(4),
    'Percentage'       : pct.values.round(2)
})
print(importance_df.to_string(index=False))

# Chart
top15 = importances_sorted.head(15).sort_values(ascending=True)
plt.figure(figsize=(8, 6))
top15.plot(kind='barh', color='steelblue')
plt.xlabel('Feature Importance', fontsize=12)
plt.title('Feature Importance — Python GBM Classifier', fontsize=13)
plt.tight_layout()
plt.show()
```

# Model 3 — Time Series Forecasting

*AutoTS: Horizontal Ensemble — Forecast 2025 through 2032*

## Cell 1 — Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from autots import AutoTS
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.stats.stattools import durbin_watson
from statsmodels.tsa.seasonal import seasonal_decompose
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

print('All libraries loaded successfully')
```

## Cell 2 — Load & Prepare Data

```
df = pd.read_csv('Traffic_by_Facility_FINAL-1.csv')

# Parse date
df['date'] = pd.to_datetime(df['date'], format='%m/%d/%Y', errors='coerce')

# Keep only what we need
df = df[['date', 'fac_b', 'total_vehicles']].dropna()

# Sort by facility and date
df = df.sort_values(['fac_b', 'date']).reset_index(drop=True)

print(f'Dataset shape  : {df.shape}')
print(f'Date range     : {df["date"].min().date()} to {df["date"].max().date()}')
print(f'Facilities     : {sorted(df["fac_b"].unique())}')
print(f'Missing values : {df.isnull().sum().sum()}')
print(f'\nRows per facility:')
print(df['fac_b'].value_counts().sort_index())
```

## Cell 3 — Yearly Aggregation & Raw Time Series

```
# Aggregate to yearly total vehicles per facility
yearly = df.groupby([df['date'].dt.year,
'fac_b'])['total_vehicles'].sum().reset_index()
yearly.columns = ['year', 'fac_b', 'total_vehicles']

print('Yearly Total Vehicles per Facility:')
print(yearly.pivot(index='year', columns='fac_b',
values='total_vehicles').to_string())

facilities = sorted(df['fac_b'].unique())
fig, axes = plt.subplots(4, 2, figsize=(16, 20))
axes = axes.flatten()

for i, fac in enumerate(facilities):
    data = yearly[yearly['fac_b'] == fac]
```

```
    axes[i].plot(data['year'], data['total_vehicles'],
                 marker='o', linewidth=2, color='steelblue', markersize=6)
    axes[i].axvline(x=2020, color='red',    linestyle='--', alpha=0.7,
label='COVID-19')
    axes[i].axvline(x=2025, color='orange', linestyle='--', alpha=0.7,
label='Congestion Pricing')
    axes[i].set_title(f'{fac} — Yearly Traffic Volume', fontsize=12,
fontweight='bold')
    axes[i].set_xlabel('Year')
    axes[i].set_ylabel('Total Vehicles')
    axes[i].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'{x:,.0f}'))
    axes[i].legend(fontsize=8)
    axes[i].grid(True, alpha=0.3)

plt.suptitle('Historical Yearly Traffic Volume by Facility (2015-2025)',
             fontsize=16, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
```

## Cell 4 — Year-over-Year Growth Rates

```
yearly_pivot = yearly.pivot(index='year', columns='fac_b',
values='total_vehicles')
yearly_full  = yearly_pivot[yearly_pivot.index <= 2024].copy()
yoy_growth   = yearly_full.pct_change() * 100

print('Year-over-Year Growth Rate (%) per Facility:')
print(yoy_growth.round(2).to_string())

fig, ax = plt.subplots(figsize=(14, 6))
for fac in yearly_full.columns:
    ax.plot(yoy_growth.index, yoy_growth[fac], marker='o',
            linewidth=2, markersize=5, label=fac)

ax.axhline(y=0, color='black', linestyle='-',  linewidth=0.8)
ax.axvline(x=2020, color='red',    linestyle='--', alpha=0.7, label='COVID-19')
ax.axvline(x=2025, color='orange', linestyle='--', alpha=0.7, label='Congestion
Pricing')
ax.set_title('Year-over-Year Traffic Growth Rate by Facility (2016-2024)',
             fontsize=14, fontweight='bold')
ax.set_xlabel('Year')
ax.set_ylabel('Growth Rate (%)')
ax.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=9)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

## Cell 5 — Seasonal Decomposition

```
facilities_to_decompose = ['GWB Upper', 'Bayonne', 'Holland']
titles = {
    'GWB Upper': 'GWB Upper — Busiest Facility, Stable Long-Term Growth',
    'Bayonne'  : 'Bayonne — Unique Upward Trend, Outlier Behavior',
    'Holland'  : 'Holland Tunnel — Steepest COVID Impact & Strongest Recovery'
}

for fac in facilities_to_decompose:
    fac_data = df[df['fac_b'] == fac].copy()
    fac_data = fac_data.set_index('date')['total_vehicles']
    fac_data = fac_data[~fac_data.index.duplicated(keep='first')]
    full_idx = pd.date_range(start=fac_data.index.min(),
```

```
                              end=fac_data.index.max(), freq='D')
    fac_data = fac_data.reindex(full_idx).ffill()

    decomp = seasonal_decompose(fac_data, model='additive',
                                period=365, extrapolate_trend='freq')

    fig, axes = plt.subplots(4, 1, figsize=(14, 10))
    decomp.observed.plot(ax=axes[0], color='steelblue')
    axes[0].set_title('Observed', fontweight='bold')
    axes[0].set_ylabel('Vehicles')
    axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'{x:,.0f}'))
    decomp.trend.plot(ax=axes[1], color='orange')
    axes[1].set_title('Trend', fontweight='bold')
    axes[1].set_ylabel('Vehicles')
    axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'{x:,.0f}'))
    decomp.seasonal.plot(ax=axes[2], color='green')
    axes[2].set_title('Seasonality', fontweight='bold')
    axes[2].set_ylabel('Vehicles')
    axes[2].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'{x:,.0f}'))
    decomp.resid.plot(ax=axes[3], color='red', alpha=0.7)
    axes[3].set_title('Residuals (Unexplained Variation)', fontweight='bold')
    axes[3].set_ylabel('Vehicles')
    axes[3].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'{x:,.0f}'))
    plt.suptitle(f'Seasonal Decomposition — {titles[fac]}',
                 fontsize=13, fontweight='bold', y=1.01)
    plt.tight_layout()
    plt.show()
    print('-' * 80)
```

## Cell 6 — Stationarity Tests (ADF & KPSS)

```
print('=' * 70)
print('STATIONARITY TESTS — ADF & KPSS (Yearly Data per Facility)')
print('=' * 70)

results = []
for fac in sorted(df['fac_b'].unique()):
    fac_yearly = yearly[yearly['fac_b'] == fac].dropna()
    series = fac_yearly['total_vehicles'].values
    if len(series) < 4:
        continue

    # ADF Test — null hypothesis: series is non-stationary
    adf_result     = adfuller(series, autolag='AIC')
    adf_stat       = adf_result[0]
    adf_pvalue     = adf_result[1]
    adf_conclusion = 'Stationary' if adf_pvalue < 0.05 else 'Non-Stationary'

    # KPSS Test — null hypothesis: series is stationary
    kpss_result     = kpss(series, regression='c', nlags='auto')
    kpss_stat       = kpss_result[0]
    kpss_pvalue     = kpss_result[1]
    kpss_conclusion = 'Stationary' if kpss_pvalue > 0.05 else 'Non-Stationary'

    if adf_conclusion == 'Stationary' and kpss_conclusion == 'Stationary':
        overall = 'STATIONARY'
    elif adf_conclusion == 'Non-Stationary' and kpss_conclusion == 'Non-
Stationary':
        overall = 'NON-STATIONARY'
    else:
        overall = 'INCONCLUSIVE'
```

```
    results.append({'Facility': fac, 'ADF Stat': round(adf_stat,4),
                    'ADF p-value': round(adf_pvalue,4), 'ADF Result':
adf_conclusion,
                    'KPSS Stat': round(kpss_stat,4), 'KPSS p-value':
round(kpss_pvalue,4),
                    'KPSS Result': kpss_conclusion, 'Overall': overall})

    print(f'\nFacility : {fac}')
    print(f'  ADF  — Stat: {adf_stat:.4f} | p-value: {adf_pvalue:.4f} |
{adf_conclusion}')
    print(f'  KPSS — Stat: {kpss_stat:.4f} | p-value: {kpss_pvalue:.4f} |
{kpss_conclusion}')
    print(f'  Overall: {overall}')

print('\n' + '=' * 70)
results_df = pd.DataFrame(results)
print(results_df[['Facility','ADF Result','KPSS
Result','Overall']].to_string(index=False))
```

## Cell 7 — AutoTS Model Training

```
# Prepare data for AutoTS
df_autots = df[['date', 'fac_b', 'total_vehicles']].copy()

# Train on full data 2015-2024
# AutoTS internally withholds last 2 periods for validation (backwards method)
train = df_autots[df_autots['date'].dt.year <= 2024].copy()

print(f'Training set : {train["date"].min().date()} to
{train["date"].max().date()}')
print(f'Training rows: {len(train)}')

forecast_horizon = 365

model = AutoTS(
    forecast_length     = forecast_horizon,
    frequency           = 'infer',
    prediction_interval = 0.95,
    ensemble            = ['simple', 'horizontal-min'],
    model_list          = 'fast',
    max_generations     = 10,
    num_validations     = 2,
    validation_method   = 'backwards',
    no_negatives        = True,
    n_jobs              = -1,
)

model = model.fit(
    train,
    date_col  = 'date',
    value_col = 'total_vehicles',
    id_col    = 'fac_b'
)

print('\nModel training complete.')
print(model)
```

## Cell 8 — Generate Forecast (2025-2032)

```
# Generate forecast for 2025-2032 (8 years = 2922 days)
prediction     = model.predict(forecast_length=2922)
forecast_daily = prediction.forecast
```

```
forecast_upper = prediction.upper_forecast
forecast_lower = prediction.lower_forecast

print('Forecast shape:', forecast_daily.shape)
print('Forecast date range:', forecast_daily.index.min(), 'to',
forecast_daily.index.max())
print('\nFirst few rows of forecast:')
print(forecast_daily.head())
```

## Cell 9 — Export Best Model Template

```
# Export best model template to CSV
model.export_template(
    'Model3_TimeSeries_BestModels.csv',
    models='best',
    max_per_model_class=1,
    include_results=True
)

print('Best model template exported to Model3_TimeSeries_BestModels.csv')
```

## Cell 10 — Aggregate Forecast to Yearly Table

```
# Reshape forecast daily to long format
forecast_long = forecast_daily.reset_index()
forecast_long = forecast_long.melt(id_vars='index',
                                   var_name='fac_b',
                                   value_name='forecast_vehicles')
forecast_long.rename(columns={'index': 'date'}, inplace=True)
forecast_long['year'] = pd.to_datetime(forecast_long['date']).dt.year

# Aggregate to yearly
yearly_forecast =
forecast_long.groupby(['year','fac_b'])['forecast_vehicles'].sum().reset_index()
yearly_forecast = yearly_forecast[yearly_forecast['year'] <= 2032]

# Pivot for display
forecast_table = yearly_forecast.pivot(index='year', columns='fac_b',
                                       values='forecast_vehicles')
forecast_table_fmt = forecast_table.applymap(lambda x: f'{x:,.0f}' if pd.notna(x)
else 'N/A')

print('Yearly Forecast — Total Vehicles per Facility (2025-2032):')
print(forecast_table_fmt.to_string())
```

## Cell 11 — Per-Facility Forecast Visualization

```
yearly_forecast_num = yearly_forecast.copy()
yearly_actual       = yearly[yearly['year'] <= 2024]

fig, axes = plt.subplots(8, 1, figsize=(14, 40))

for i, fac in enumerate(sorted(yearly_forecast_num['fac_b'].unique())):
    actual   = yearly_actual[yearly_actual['fac_b'] == fac]
    forecast = yearly_forecast_num[yearly_forecast_num['fac_b'] == fac]

    axes[i].plot(actual['year'], actual['total_vehicles'],
                 marker='o', color='steelblue', linewidth=2, label='Historical')
```

```
    axes[i].plot(forecast['year'], forecast['forecast_vehicles'],
                 marker='s', color='orange', linewidth=2,
                 linestyle='--', label='Forecast (2025-2032)')
    axes[i].axvline(x=2024.5, color='gray', linestyle=':', alpha=0.7,
label='Forecast Start')
    axes[i].set_title(f'{fac}', fontsize=13, fontweight='bold')
    axes[i].set_xlabel('Year')
    axes[i].set_ylabel('Total Vehicles')
    axes[i].set_xticks(list(range(2015, 2033)))
    axes[i].tick_params(axis='x', rotation=45)
    axes[i].yaxis.set_major_formatter(
        mticker.FuncFormatter(lambda x, _: f'{x:,.0f}'))
    axes[i].legend(fontsize=9)
    axes[i].grid(True, alpha=0.3)

plt.suptitle('Historical vs Forecast — Yearly Traffic by Facility (2015-2032)',
             fontsize=15, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
```

## Cell 12 — Network-Wide Forecast

```
# Total across all facilities by year
network_actual   = yearly[yearly['year'] <=
2024].groupby('year')['total_vehicles'].sum()
network_forecast = yearly_forecast_num.groupby('year')['forecast_vehicles'].sum()

fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(network_actual.index, network_actual.values,
        marker='o', color='steelblue', linewidth=2, label='Historical')
ax.plot(network_forecast.index, network_forecast.values,
        marker='s', color='orange', linewidth=2,
        linestyle='--', label='Forecast (2025-2032)')
ax.axvline(x=2024.5, color='gray', linestyle=':', alpha=0.7, label='Forecast
Start')
ax.axvline(x=2020,   color='red',  linestyle='--', alpha=0.5, label='COVID-19')
ax.set_title('Port Authority Network — Total Yearly Traffic (2015-2032)',
             fontsize=14, fontweight='bold')
ax.set_xlabel('Year')
ax.set_ylabel('Total Vehicles (All Facilities)')
ax.set_xticks(list(range(2015, 2033)))
ax.tick_params(axis='x', rotation=45)
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{x:,.0f}'))
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('\nNetwork-Wide Yearly Total:')
combined = pd.concat([
    network_actual.rename('Historical'),
    network_forecast.rename('Forecast')
], axis=1)
print(combined.applymap(lambda x: f'{x:,.0f}' if pd.notna(x) else
'N/A').to_string())
```